



Manual do Programador

Tipo de Documento	Manual
Número do Documento	D2.A
Título do Documento	Manual do Programador
Autor Principal	Igor Coelho Instituto de Telecomunicações
Versão / Estado	1 Final
Data	1421/07/2026
Nível de Distribuição	PU (public)



arte



INFORMAÇÃO GERAL

DETALHES DO PROJETO

Acrônimo	DT4MOB
Título	Digital Twins for Mobility
Website	https://www.it.pt/Projects/Index/4934
Coordenador	Instituto de Telecomunicações
Referência	2025.00153.DT4ST

CONTRIBUIDORES

Name	Organization
Bernardo Figueiredo	Instituto de Telecomunicações
Cecília Santos	Instituto de Telecomunicações
Igor Coelho	Instituto de Telecomunicações
João Capucho	Instituto de Telecomunicações
Tomás Victal	Instituto de Telecomunicações
Zakhar Kruptsala	Instituto de Telecomunicações

Financiado pela União Europeia - NextGenerationEU. No entanto, os pontos de vista e opiniões expressos são exclusivamente do(s) autor(es) e não refletem necessariamente os pontos de vista e opiniões da União Europeia ou da Comissão Europeia. Nem a União Europeia nem a Comissão Europeia são responsáveis por eles.

ÍNDICE

1	INTRODUÇÃO	13
2	FIRE-SIMULATOR	13
2.1	Arquitetura do Sistema	13
2.2	Visão Geral	13
2.3	Mapa de Ficheiros do Projeto	14
2.3.1	Diretoria Raiz	14
2.3.2	Preparação de Dados (data_preparation/)	14
2.3.3	Worker de Ingestão (ingestion/) — Aplicação Principal	14
2.3.4	Modelos (ingestion/models/)	15
2.3.5	Definições (ingestion/settings/)	15
2.3.6	Serviços (ingestion/services/)	16
2.3.7	Helm Chart (chart/)	16
2.4	Sequência de Arranque	17
2.5	Fluxo de Eventos de Ignição (Detalhado)	17
2.6	Análise Detalhada de Módulos	17
2.6.1	pipeline.py — Orquestrador do Pipeline	17
2.6.2	cone_management.py — Geometria do Cone de Vento	18
2.6.3	wind.py — Dados Meteorológicos e Interpolação IDW	18
2.6.4	quadkeys.py — Geohashing QuadTile	18
2.6.5	landscape.py — Construtor de NetCDF de Paisagem	18
2.6.6	forefire_runner.py — Script e Execução ForeFire	19
2.6.7	gltf_exporter.py — Exportação 3D GLB	19
2.6.8	s3_uploader.py — Upload S3	19
2.6.9	ditto/ditto_class.py — Cliente Ditto	19
2.6.10	auth/__init__.py — Serviço de Autenticação	20
2.7	Modelos de Dados (ingestion/models/)	20
2.7.1	fire_incident.py	20
2.7.2	ditto_body_maker.py	20
2.7.3	constants.py	21
2.8	Arquitetura de Configuração	21
2.9	Schema da Thing Ditto	21
2.10	Arquitetura Helm Chart	22
2.10.1	Recursos	22
2.10.2	Padrão Init Container	23
2.10.3	Configuração S3 via CRDs SeaweedFS	23
2.10.4	Mapeamento Env: values.yaml → Contentor	23
2.11	Comparação Docker Compose vs Helm	24
2.12	Visão Geral da Arquitetura	25
2.13	Visão Geral	25
2.14	Estrutura de Diretórios	25
2.15	Fluxo de Dados	27
2.16	Detalhes dos Módulos	27
2.16.1	app/main.py — Ponto de Entrada	27

2.16.2	app/routers/events.py — Endpoints API	28
2.16.3	app/services/events_service.py — Lógica de Negócio	29
2.16.4	app/models/ditto_events.py — Modelo de Base de Dados	30
2.16.5	app/models/paths_response.py — Modelo de Resposta	31
2.16.6	app/models/util.py — Enum Action	31
2.16.7	app/database_engine/__init__.py — Singleton Engine	31
2.16.8	app/database_engine/TimeScaleEngineManager.py — Wrapper do Engine	31
2.16.9	app/settings/init.py — Configuração Global	32
2.16.10	app/settings/auth.py — Configuração de Autenticação	32
2.16.11	app/settings/timescale.py — Configuração do Timescale	32
2.17	Fluxo de Autenticação	32
2.18	Estratégia de Tratamento de Erros	33
2.19	Dependências	33
2.19.1	Pacotes Python (de pyproject.toml)	33
2.19.2	Serviços Externos	34
2.20	Deployment	34
2.20.1	Dockerfile	34
2.20.2	Helm Chart	34
2.20.3	Imagens dos Contentores	35
2.21	Visão Geral	35
2.22	Stack Tecnológica	35
2.23	Estrutura do Projeto	35
2.24	Arquitetura e Fluxo de Dados	36
2.25	Detalhes de Implementação	37
2.25.1	Singleton Pattern	37
2.25.2	Ciclo de Vida do WebSocket & Token	37
2.25.3	Correlation-id Matching	37
2.25.4	Pagination	37
2.25.5	Loop de Recolha de Lixo	37
2.26	Dependências	38
2.26.1	Diretas	38
2.27	Configuração	38
2.28	Configuração de Desenvolvimento	38
2.29	Build	39
2.30	Kubernetes e Helm	39
3	HISTORICAL-WRITER	40
3.1	Visão Geral	40
3.2	Estrutura do Projeto	40
3.3	Sequência de Inicialização	41
3.4	Fluxo de Mensagens (Ponta a Ponta)	42
3.5	Detalhes dos Módulos	42
3.5.1	Settings (src/settings/)	42
3.5.2	KafkaConsumer (src/services/KafkaConsumer/kafka_consumer.py)	43
3.5.3	MessageProcessor (src/services/MessageProcessor/message_processor.py)	43
3.5.4	Modelo DittoEvent (src/models/ditto_events.py)	43
3.5.5	TimeScaleDBEngineManager (src/database_engines/TimeScaleEngineManager.py)	44
3.5.6	DittoEventsManager (src/services/DittoEventsManager/ditto_events_manager.py)	44

3.6	Dependências	44
3.7	Build Docker	45
3.8	Deployment com Kubernetes	45
3.8.1	Estrutura do Chart	45
3.8.2	Writer Deployment (writer-deployment.yaml)	45
3.8.3	Writer ConfigMap (writer-config.yaml)	46
3.8.4	TimescaleDB Cluster (timescale-cluster.yaml)	46
3.8.5	KafkaTopic (kafka-topic.yaml)	46
3.8.6	Como Tudo se Liga	46
4	LEVELAMS LOADER	47
4.1	Visão Geral	47
4.2	Architecture Overview	47
4.2.1	Design do Sistema	47
4.3	Local Configuration	49
4.3.1	Project Structure	49
4.3.2	Scripts and Commands	50
4.3.3	Deployment	51
4.3.4	Environment Variables	51
4.3.5	Code Conventions	52
5	GANTRY SERVICE	53
5.1	Visão Geral	53
5.2	Architecture Overview	53
5.3	Configuração Local	56
5.3.1	Project Structure	57
5.4	Scripts e Comandos	58
5.4.1	Key Implementation Details	58
5.5	Deployment	59
6	SENSOR DATA CONVERSION SERVICE	60
6.1	Visão Geral	60
6.2	Architecture Overview	60
6.3	Instalação e Ligação	62
6.4	Local Configuration	62
6.5	Estrutura do Projeto	63
6.6	Scripts e Comandos	63
6.7	Detalhes Chave de Implementação	64
6.8	Deployment	65
6.9	Visão Geral	66
6.10	Arquitetura do Sistema	66
6.11	Autenticação e Configuração	68
6.12	Aquisição de Dados	69
6.12.1	REST Acquisition (UDittoService)	69
6.12.2	Quadkey Geographic Math	70
6.12.3	WebSocket Acquisition (UWSService and UEntityUpdateDaemon)	70

6.13	Conversão de Coordenadas Geográficas	71
6.14	Modelos de Dados de Entidade	72
6.15	Representação de Entidade (ATempUIActor)	72
6.15.1	Components	72
6.15.2	Initialisation	73
6.15.3	Geographic Positioning (SetLocation)	73
6.15.4	Live Update Routing (HandleEntityUpdate)	74
6.15.5	Smooth Position Interpolation	74
6.15.6	Terrain Snap (SnapToGround)	75
6.15.7	GLB Model Loading	75
6.15.8	Terrain Exclusion Polygon	75
6.15.9	Entity Expiry	75
6.16	Entity Factory and Tile-Based Streaming	77
6.16.1	UDT4MOBEntityFactory	77
6.16.2	ADT4MOBGamemode and Tile-Based Streaming	77
6.17	Sistema de Navegação e Câmara	79
6.17.1	AUnifiedPawn	80
6.17.2	AUnifiedController	81
6.18	Gestores e Lógica de Aplicação	81
6.18.1	USelectionManager (LocalPlayer Subsystem)	81
6.18.2	UUIManager (LocalPlayer Subsystem)	81
6.18.3	UActorRegistryService (GameInstance Subsystem)	82
6.18.4	UPlacementManager (LocalPlayer Subsystem)	82
6.19	Interface do Utilizador	82
6.19.1	URootHUDWidget	82
6.19.2	UEntityWindowWidget	83
6.19.3	UOutlinePanelWidget	84
6.19.4	UI Theming	85
6.20	GLB Model Service (UGlbModelService)	85
6.21	Fluxo de Interação do Utilizador	85
6.22	Referência de Configuração	88
7	DATA BRIDGE	88
7.1	Visão Geral	88
7.2	Estrutura do Código	92
7.3	Singletons	92
7.4	Interfaces	93
7.4.1	Hono Interface	94
7.5	Interface HonoMock	94
7.6	Interface IPMA	94
7.7	Interface Waze	94
7.8	Dispositivos	94
7.9	Estratégias	95
7.10	Estratégia Meteorológica	97
7.11	Estratégia de Avisos Meteorológicos	98
7.12	Estratégia de Tráfego	99
7.13	Estratégia GeoJSON	99

8	METEO POPULATOR	101
8.1	Visão Geral	101
8.2	Estrutura do Código	105
8.3	Modelos de Dados	106
8.4	Configuração	107
8.5	Interfaces	107
8.5.1	OIDC Interface	108
8.5.2	IPMA Interface	108
8.5.3	Ditto Interface	108
8.6	Utilitários	109
9	NAMESPACE REMOVER	109
9.1	Visão Geral	109
9.2	Estrutura do Código	111
9.3	Modelos de Dados	112
9.4	Configuração	113
9.5	Argumentos de CLI	114
9.6	Interfaces	115
9.6.1	OIDC Interface	115
9.6.2	Search Interface	115
9.6.3	Delete Interface	116
9.7	Orquestração	116
10	TEXTURE CREATION	117
10.1	Visão Geral	117
10.2	Estrutura do Código	120
10.3	Modelos de Dados	120
10.4	Interface API	121
10.5	Interface S3	122
10.6	Processamento de Malha	123
10.7	Cálculo de Cores	123
10.8	Pipeline de Cores de Vértice	124
10.8.1	From Colored Points to Textured Mesh	125
10.9	Pipeline Principal	126
11	SIMULAÇÃO DA FAIXA RESERVADA A5	127
11.1	Visão Geral	127
11.2	Fluxo de Execução	127
11.3	Estrutura do código	128
11.4	Configuração	128
11.5	Modelos de Dados	129
11.5.1	SimulationRequest	129
11.5.2	ScenarioResult	129
11.5.3	SimulationResponse	130
11.5.4	JobStatus	130
11.5.5	JobSubmitted	130
11.5.6	JobStatusResponse	130

11.6	API	131
11.7	Simulação	131
11.8	Gestão de Trabalhos	131
11.9	Execução dos Cenários	131
11.10	Recolha de Estatísticas	132
11.11	Execução	132
12	VCI TRAFFIC SIMULATION	132
12.1	Visão Geral	132
12.2	Estrutura	133
12.3	Estrutura do Projeto	133
12.4	Rede	133
12.5	Geração das rotas	134
12.6	Processo de Calibração	134
12.7	Execução	134
13	MODELOS DE PREVISÃO DE FLUXO DE TRÁFEGO	135
13.1	Visão Geral	135
13.2	Arquitetura de Machine Learning	135
13.2.1	Pré-processamento (preprocess.py)	135
13.2.2	Imputação de Dados (impute.py)	136
13.2.3	Treino dos Modelos (models.py)	136
13.2.4	Otimização de Hiperparâmetros	137
13.3	Métricas de Avaliação	137
13.4	Artefactos Produzidos	138
13.5	Execução	138
14	MODELO AGÊNTICO	138
14.1	Visão Geral	138
14.2	Arquitetura	138
14.3	Fluxo de Execução	139
14.4	MCP Server	139
14.5	Modelos de Previsão	139
14.6	Configuração e Execução	139

LISTA DE FIGURAS

Figura 1: Arquitetura do Fire Simulator.....	13
Figura 2: Diagrama de fluxo de dados entre componentes.....	47
Figura 3: Diagrama de sequência de interações externas do LevelAMS Loader.....	48
Figura 4: Diagrama de arquitetura de alto nível do Gantry Service.....	53
Figura 5: Diagrama de sequência de interações entre componentes do Gantry Service.....	54
Figura 6: Diagrama de sequência do pipeline de dados do Serviço de Conversão de Dados de Sensor.....	60
Figura 7: Arquitetura em camadas da plataforma de visualização do DT4MOB, mostrando as cinco camadas e a direção do fluxo de informação desde o Eclipse Ditto até aos widgets UMG.....	68
Figura 8: Fluxo de arranque OAuth2 desde o ecrã de login até à obtenção de token e ligação WebSocket.....	69
Figura 9: Como um ponto geográfico é mapeado para um intervalo inteiro de geotile que é enviado como filtro RSqI para a API REST do Ditto.....	70
Figura 10: Percurso de atualização em tempo real desde o evento WebSocket do Ditto até ao ator de entidade e à atualização da UI.....	71
Figura 11: Ciclo de vida do ATempUIActor desde a instanciação pela fábrica até à destruição....	77
Figura 12: Ciclo de streaming baseado em tiles mostrando a grelha 3×3 centrada no tile da câmara e o debounce de 0,75 s a prevenir atualizações espúrias durante movimento rápido.....	79
Figura 13: Comparação do modo RTS (braço de mola aéreo, globo totalmente visível) e do modo FreeFly (braço de mola colapsado, perspetiva ao nível do solo com orientação vertical ao globo)81	
Figura 14: Visão geral da aplicação com a barra de ferramentas no topo, uma janela de inspeção de entidade aberta e o painel de outline à direita.....	83
Figura 15: Close-up de uma janela de entidade aberta mostrando a barra de cabeçalho com thingId e badge de tipo, os quatro botões de separador e o separador Info.....	84
Figura 16: Fluxograma de ponta a ponta desde o login até à autenticação, carregamento inicial de entidades, atualização WebSocket ao vivo, sobreposição, seleção e exibição da janela.....	88
Figura 17: Grafo de fluxo de controlo do Data Bridge.....	89
Figura 18: Diagrama de sequência do Data Bridge.....	92
Figura 19: Grafo de dependências e estrutura de código do Data Bridge.....	92
Figura 20: Diagrama de sequência da Estratégia Meteorológica.....	97
Figura 21: Diagrama de sequência da Estratégia de Avisos.....	98
Figura 22: Diagrama de sequência da Estratégia de Tráfego.....	99
Figura 23: Diagrama de sequência da Estratégia GeoJSON.....	100

Figura 24: Grafo de fluxo de controlo do Meteo Populator	102
Figura 25: Diagrama de sequência do Meteo Populator	104
Figura 26: Grafo de dependências e estrutura de código do Meteo Populator	105
Figura 27: Grafo de fluxo de controlo do Namespace Remover.....	110
Figura 28: Diagrama de sequência do Namespace Remover	111
Figura 29: Grafo de dependências e estrutura de código do Namespace Remover	112
Figura 30: Grafo de fluxo de controlo do Texture Creation.....	118
Figura 31: Diagrama de sequência do Texture Creation	120
Figura 32: Grafo de dependências e estrutura de código do Texture Creator	120
Figura 33: Pipeline de criação de malha.....	126

LISTA DE TABELAS

Tabela 1 - Caminhos — Diretoria Raiz	14
Tabela 2 - Caminhos — Preparação de Dados	14
Tabela 3 - Caminhos — Worker de Ingestão.....	15
Tabela 4 - Caminhos — Modelos	15
Tabela 5 - Caminhos — Definições	16
Tabela 6 - Caminhos — Serviços	16
Tabela 7 - Caminhos — Helm Chart.....	17
Tabela 8 - Modelo — fire_incident.py	20
Tabela 9 - Constante — constants.py.....	21
Tabela 10 - Recurso — Recursos.....	23
Tabela 11 - Aspeto — Comparação Docker Compose vs Helm # Ditto Events API.....	25
Tabela 12 - Função de Endpoint — app/routers/events.py	29
Tabela 13 - Propriedade — app/models/ditto_events.py	30
Tabela 14 - Cenário — Estratégia de Tratamento de Erros	33
Tabela 15 - Pacotes Python.....	34
Tabela 16 - Serviços Externos.....	34
Tabela 17 - Stack Tecnológica	35
Tabela 18 - Dependências Diretas	38
Tabela 19 - Configuração da hipertabela Timescale	44
Tabela 20 - Dependências.....	45
Tabela 21 - Visão Geral da Arquitetura	49
Tabela 22 - Estrutura do Projeto.....	50
Tabela 23 - Variável de Ambiente para configuração do LevelAMS Loader	52
Tabela 24 - Visão Geral da Arquitetura	54
Tabela 25 – Diferentes componentes do LevelAMS Loader	55
Tabela 26 - Estrutura do Projeto.....	58
Tabela 27 - Tecnologias utilizadas no LevelAMS Loader.....	61
Tabela 28 - Estrutura do Projeto.....	63

Tabela 29 - Roteamento de Atualizações em Tempo Real (HandleEntityUpdate)	74
Tabela 30 - Referência de Configuração	88
Tabela 31 - Variáveis de ambiente para configuração do Namespace Remover	114
Tabela 32 - Argumentos de CLI para utilização do Namespace Remover	114
Tabela 33 - Cálculo de cores para a criação da textura	124
Tabela 34 - Modelo Alvo — Arquitetura de Machine Learning	Error! Bookmark not defined.

1 INTRODUÇÃO

Este é o manual de programação do projeto DT4MOB. Documenta todos os serviços, componentes e microserviços que constituem a plataforma de digital twin do DT4MOB, abrangendo a arquitetura, o fluxo de dados, a configuração, a implementação e os detalhes de API de cada componente.

O documento está organizado por serviço, descrevendo a estrutura de diretorias, a sequência de arranque, o fluxo de eventos e a análise detalhada de cada módulo. Inclui também a configuração de infraestrutura, como a arquitetura de Helm Chart e a comparação entre ambientes de desenvolvimento Docker Compose e produção Kubernetes.

Destina-se a desenvolvedores e integradores de sistema que trabalham na plataforma DT4MOB, fornecendo toda a informação necessária para compreender, configurar, implantar e estender cada componente da plataforma.

2 FIRE-SIMULATOR

2.1 Arquitetura do Sistema

2.2 Visão Geral

O Fire-Simulator é um worker de ingestão que processa eventos de ignição de incêndios recebidos do Eclipse Ditto. Executa simulações ForeFire, carrega os resultados no SeaweedFS e atualiza os digital twins. É o motor de simulação central do pipeline de monitorização de incêndios do DT4MOB.

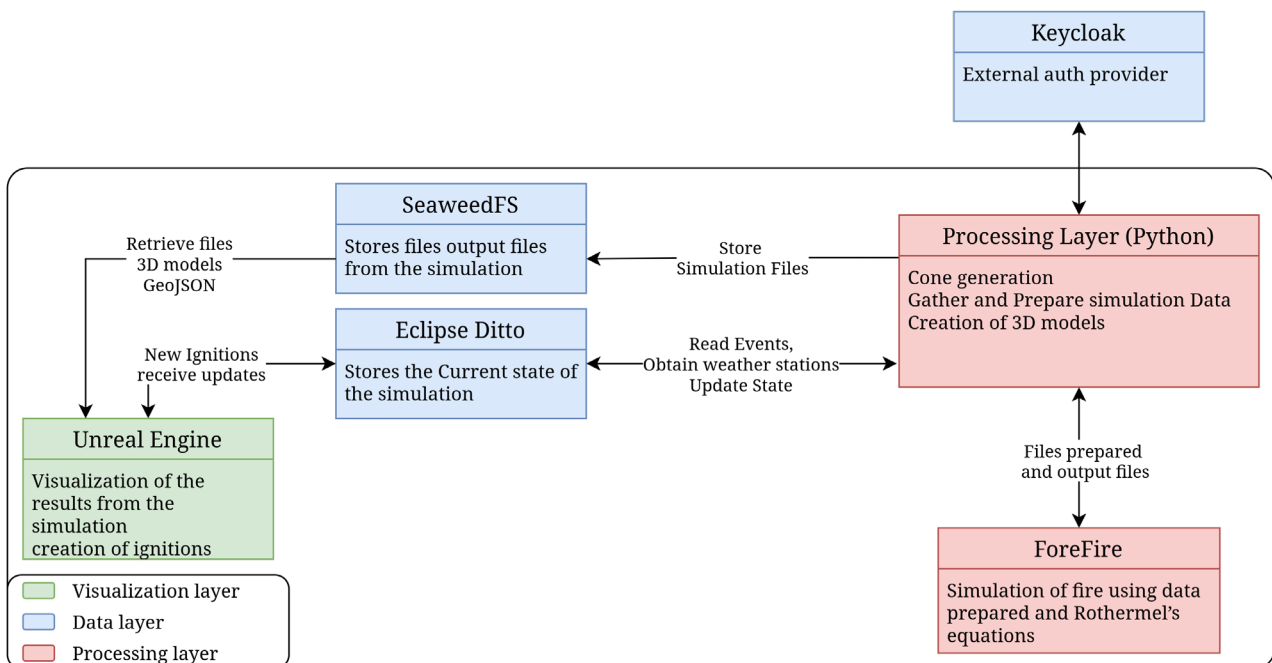


Figura 1: Arquitetura do Fire Simulator

O worker autentica-se via Keycloak (OIDC password grant), obtém um JWT e utiliza-o para abrir um WebSocket para o Eclipse Ditto. O Ditto transmite eventos de incêndio (Things no namespace fire com state == 'new_ignition'). Cada evento desencadeia o pipeline, que pode executar o ForeFire como subprocesso. Os resultados são enviados para o SeaweedFS (compatível com S3) e o twin no Ditto é atualizado com URLs de ficheiros e transições de estado.

2.3 Mapa de Ficheiros do Projeto

2.3.1 Diretoria Raiz

Caminho	Propósito
app_deploy/.env.example.	Modelo para todas as variáveis de ambiente.
app_deploy/docker-compose.yml.	Orquestração Docker Compose para desenvolvimento local.
app_deploy/data/	Volume de dados partilhado (rasters, simulações) — bind-mounted.

Tabela 1 - Caminhos — Diretoria Raiz

2.3.2 Preparação de Dados (data_preparation/)

Caminho	Propósito
data_preparation/prepare_rasters.py.	Script único: descarrega rasters DEM + COSc, reprojecta, extrai zonas de risco do S3.
data_preparation/Dockerfile.	Imagem Python 3.11-slim com GDAL + dependências.
data_preparation/requirements.txt.	6 pacotes Python

Tabela 2 - Caminhos — Preparação de Dados

2.3.3 Worker de Ingestão (ingestion/) — Aplicação Principal

Caminho	Propósito
ingestion/main.py	Ponto de entrada. Abre WebSocket do Ditto, escuta eventos de ignição.
ingestion/pipeline.py.	Orquestrador principal. Gera cone, verifica risco, executa pipeline ForeFire, carrega, limpa.
ingestion/cone_management.py.	Geometria do cone de propagação do vento (Haversine, 4 horizontes)
ingestion/wind.py	Consultas a estações meteorológicas (via pesquisa espacial Ditto) + interpolação IDW.

Caminho	Propósito
ingestion/quadkeys.py.	Geohashing QuadTile para filtragem espacial Ditto.
ingestion/landscape.py.	Construtor de ficheiros NetCDF de paisagem + escritor de CSV de combustíveis.
ingestion/forefire_runner.py.	Gerador de scripts .ff ForeFire + executor de subprocesso + analisador de perímetros.
ingestion/gltf_expert.py.	Exportação de modelo 3D GeoJSON-para-GLB (perímetros + cones)
ingestion/geojson.py.	Auxiliares de I/O de ficheiros GeoJSON.
ingestion/s3_uploader.py.	Carregador S3 SeaweedFS (boto3)
ingestion/Dockerfile.	Multi-stage: compila ForeFire (C++), depois runtime Python.
ingestion/requirements.txt.	16 pacotes Python

Tabela 3 - Caminhos — Worker de Ingestão

2.3.4 Modelos (ingestion/models/)

Caminho	Propósito
fire_incident.py	Modelos Pydantic: Point, enum fireState, ConeSection, schemas de Things Ditto
ditto_body_maker.py	Padrão builder para corpos JSON de Things Ditto.
constants.py	Rótulos de frente de fogo, cores, parâmetros do ponto de ignição.

Tabela 4 - Caminhos — Modelos

2.3.5 Definições (ingestion/settings/)

Caminho	Propósito
__init__.py	Classe Config (pydantic-settings) — singleton de configuração central.

Caminho	Propósito
auth.py	AuthSettings — credenciais Keycloak/OIDC.
s3.py	s3Settings — endpoint S3 + bucket + chaves.
ditto.py	DittoSettings — URLs REST/WS do Ditto.

Tabela 5 - Caminhos — Definições

2.3.6 Serviços (ingestion/services/)

Caminho	Propósito
auth/__init__.py	AuthenticationService - OIDC password grant, cache de tokens + refresh.
ditto/__init__.py	Fábrica singleton ditto_client.
ditto/ditto_class.py.	DittoClient - ouvinte WebSocket + HTTP REST PATCH + pesquisa de estações.
ditto/utils.py	Cabeçalhos de ação, parâmetros de pesquisa baseados em quadkey.

Tabela 6 - Caminhos — Serviços

2.3.7 Helm Chart (chart/)

Caminho	Propósito
Chart.yaml	Metadados do Helm chart (v0.1.0)
values.yaml	Todos os parâmetros configuráveis.
templates/_helpers.tpl.	Modelos nomeados para labels/nomes.
templates/api-deployment.yaml.	Deployment: worker de ingestão principal + contentor init wait-for-rasters.
templates/api-service.yaml.	Service: expõe a porta 8080.
templates/prepare-job.yaml.	Job (Helm hook): executa preparação de rasters uma vez.

Caminho	Propósito
templates/pvc.yaml	PVC de 20Gi para dados partilhados.
templates/secret.yaml.	Secret condicional para credenciais Ditto.
templates/s3-identity-creds.yaml.	CRDs S3Identity + S3Credentials do SeaweedFS.
templates/s3-policy-app.yaml.	Política S3 + binding para o bucket da app (dt4mob-public)
templates/s3-policy-job.yaml.	Política S3 + binding para o bucket do job (fire-sim-briza-zones)

Tabela 7 - Caminhos — Helm Chart

2.4 Sequência de Arranque

```

asyncio.run(main())          — main.py:13
├── os.makedirs(simulations_dir)
├── signal.signal(SIGINT/SIGTERM) — paragem graciosa via asyncio.Event
├── ditto_client.connect("fire", process_ignition)
│   ├── WebSocket.open()      — ws://<ditto>/ws/2 com Bearer token
│   ├── asyncio.create_task(listen_loop(process_ignition))
│   │   └── async for msg in websocket:
│   │       ├── parse JSON
│   │       └── asyncio.to_thread(process_ignition, data)
│   ├── asyncio.create_task(_token_refresh_loop())
│   │   └── a cada 30s: auth_service.refresh(), envia msg de controlo JWT-
TOKEN
│       └── send("START-SEND-EVENTS?namespaces=fire&filter=eq(attributes/state,
'new_ignition')")

```

O ponto de entrada no main.py é minimalista: cria a diretoria de simulação, define os handlers de sinal e delega no DittoClient.connect().

2.5 Fluxo de Eventos de Ignição (Detalhado)

Imagem do fluxo de eventos de ignição.

2.6 Análise Detalhada de Módulos

2.6.1 pipeline.py – Orquestrador do Pipeline

Funções principais:

- `process_ignition(message)` — callback de nível superior. Abrange tudo: cone, verificação de risco, pipeline, upload, limpeza.
- `run_pipeline(occurrence_id, ignition_lat, ignition_lon)` — Síncrona (executa numa thread via `asyncio.to_thread`). Executa o workflow completo do ForeFire: estações -> vento -> paisagem -> combustíveis -> script -> ForeFire -> análise -> GLTF.
- `cleanup_simulation_files(sim_dir)` — Remove a diretoria de trabalho da simulação.

Importante: a `run_pipeline` é síncrona porque o `ForeFire` é uma chamada de subprocesso bloqueante. O pipeline chama `ditto_client.update_fire_incident` (HTTP, utiliza `httpx`) que também é síncrona — o `DittoClient` expõe métodos síncronos e assíncronos.

2.6.2 `cone_management.py` — Geometria do Cone de Vento

Funções principais: - `generate_cone_from_ignition(lat, lng) → (list[ConeSection], wind_dir, wind_speed)` - Obtém a estação meteorológica mais próxima → obtém direção e velocidade do vento - Velocidade de propagação = $0.18 * \text{wind_speed}$ (modelo empírico linear) - Cria 4 horizontes (30, 60, 90, 120 min) como triângulo (primeiro) ou quadrilátero (seguintes) em polígonos WGS84 - Vértice do cone na ignição, alarga a favor do vento com semiângulo de 15° - Usa `_destination_point()` para projeção geodésica de pontos (Haversine) - `cone_intersects_polygons(cone_segments, risk_polygons) → bool` - Verificação Shapely `intersects()` para cada segmento do cone vs cada polígono de risco - `segments_to_wkt(segments) → string WKT MULTIPOLYGON`

2.6.3 `wind.py` — Dados Meteorológicos e Interpolação IDW

Funções principais: - `get_nearest_stations(conn, lat, lon, n=5) → list[dict]` - Utiliza pesquisa REST Ditto com filtro baseado em quadkey - Começa no `zoom=10`, diminui até pelo menos `n` estações serem encontradas - Filtra Things com feature `meteorology` contendo dados de vento válidos - Converte códigos de direção do vento IPMA (1-9) em componentes vetoriais (`u, v`) via `_dir_code_to_uv()` - `get_nearest_station(lat, lon) → dict` — Método de conveniência, retorna a estação mais próxima - `idw_wind_grid(stations, grid_lats, grid_lons, power=2.0) → (u_grid, v_grid)` - Inverse Distance Weighting numa grelha regular de `lat/lon` - Utiliza `pyproj.Geod.inv()` para distâncias geodésicas precisas - O parâmetro `power` controla a decadência do peso (por defeito 2.0) - Casos limite: estações vazias → zeros; estação única → preenchimento constante

2.6.4 `quadkeys.py` — Geohashing QuadTile

- `get_quadkey(lat, lng, zoom) → int` — QuadTile de 64 bits a partir de `lat/lon/zoom` (esquema OSM), empacotado como inteiro com intercalação de bits
- `get_tile_bounds(lat, lng, tile_zoom, max_zoom=31) → (lower, upper)` — Retorna um intervalo de inteiros para todos os tiles em `max_zoom` que caem dentro do tile dado em `tile_zoom`. Usado para pesquisa espacial baseada em filtro Ditto.

2.6.5 `landscape.py` — Construtor de NetCDF de Paisagem

Funções principais: - `_read_reprojected(tif_path, sw_x, sw_y, lx_m, ly_m, resolution_m, resampling) → np.ndarray` - Lê GeoTIFF, reprojecta do seu CRS nativo para coordenadas métricas EPSG:3763 (Portugal TM06) - Utiliza `rasterio.warp.reproject()` com resolução e método de reamostragem especificados - `build_landscape_nc(...) → (nc_path, fuel_lats, fuel_lons, dem_lats, dem_lons)` - Lê DEM a 100m de resolução, tipo de combustível a 10m de resolução - Inverte arrays (eixo Y do rasterio: norte para cima → NetCDF: sul para cima) - Cria ficheiro NetCDF com 4 grupos de variáveis: - altitude (`nt, nz, ny, nx`) — valores DEM - fuel (`ft, fz, fy, fx`) — códigos de tipo de combustível - wind (`2,2,wr,wc`) — componentes `u` e `v`, duplicadas em 2 passos de tempo - coordinates — `lat, lon, fuel_lat, fuel_lon` - Define atributos do domínio: `SWx/SWy=0` (origem métrica local), `Lx/Ly`, `BBoxWSEN`, `WSENLBRT` - `write_fuels_csv(output_path)` — Escreve parâmetros de combustível Rothermel para 13 tipos de combustível COSc (códigos 0, 211–213, 311–313, 321–323, 410, 420, 610). Os parâmetros são constantes definidas no código.

2.6.6 forefire_runner.py — Script e Execução ForeFire

Funções principais: - `build_ff_script(lat, lon, bbox, sim_dir)` → `script_path` - Transforma o ponto de ignição e os limites do domínio de EPSG:4326 para EPSG:3763 - Escreve um script .ff ForeFire com: - units m — modo métrico - Modelo de propagação Rothermel - Tabela de combustível a partir de fuels.csv - Resolução do perímetro 10m - loadData landscape.nc — carrega o netCDF - `startFire <x> <y>` — ponto de ignição em coordenadas métricas - 8 blocos: `goTo 900 (15 min) + print step_NNNN.geojson` - `run_forefire(sim_dir, script_path, timeout=600)` → dict - Executa forefire -i script.ff como subprocesso com CWD=sim_dir - Captura stdout/stderr - Em timeout: envia SIGTERM, retorna erro - Retorna {success, stdout, stderr, returncode} - `parse_perimeters(sim_dir)` → list[dict] - Lê step_0015.geojson até step_0120.geojson - Extrai a primeira feature de cada um (o ForeFire produz uma FeatureCollection com um polígono por passo) - Retorna lista de {"timestep_min": int, "geojson": dict}

2.6.7 gltf_exporter.py — Exportação 3D GLB

Funções principais: - `_triangulate_polygon_with_density(poly, grid_spacing)` → (points, faces) - Densifica o contorno do polígono + grelha interior, triangula via `scipy.spatial.Delaunay` - Filtra faces cujos centroides caem dentro do polígono - `export_perimeters_geojson_to_gltf(sim_id, lat, lon)` → `file_path` - Lê landscape.nc para DEM (RectBivariateSpline para amostragem de altitude) - Para cada um dos 8 perímetros: triangula, amostra Z do DEM, adiciona desvio vertical específico da frente - Converte lon/lat para métrica local (aproximadamente 111000 m/grau); translada para a ignição como origem - Gradiente de cor: dourado → castanho-avermelhado por frente - Adiciona esfera azul no ponto de ignição (raio 5m) - Roda a malha para orientação 3D correta (X -90°, Y +90°) - Exporta via trimesh como GLB - `export_cone_geojson_to_gltf(sim_id, lat, lon, cones)` → `file_path` - Mesmo pipeline mas para os 4 horizontes do cone; lê DEM diretamente de /data/processed/dem.tif

2.6.8 s3_uploader.py — Upload S3

Função principal: - `upload_simulation_dir_to_seaweed_minio(simulationId, type_send)` → dict[str, str] - Configura boto3.client("s3") com endpoint SeaweedFS, assinatura S3v4, endereçamento path-style - Para type_send="cone": carrega cone_horizon.geojson + fire_cone.glb - Para type_send="perimeters": carrega ficheiros correspondentes a step* ou fire_simulation* - Retorna um mapeamento {filename: s3_url} - Tipos MIME: .glb → model/gltf-binary, .geojson → application/geo+json

2.6.9 ditto/ditto_class.py — Cliente Ditto

Métodos principais: - `connect(namespace, process_callable)` — Assíncrono. Abre WebSocket com Bearer token, inicia listen_loop + _token_refresh_loop como tarefas de fundo, envia START-SEND-EVENTS com filtro. - `listen_loop(process)` — Iterador assíncrono sobre mensagens WebSocket. Analisa eventos JSON, envia para process via `asyncio.to_thread`. Lida com ACK e correlation-based request/response. - `_token_refresh_loop()` — A cada 30 segundos, chama `auth_service.refresh()` e envia mensagem de controlo JWT-TOKEN?jwtToken=.... - `update_fire_incident(incident)` — HTTP PATCH síncrono para atualizar uma Thing Ditto (merge-patch). - `get_stations_zoom_sync(lat, lon, zoom)` — HTTP GET síncrono /search/Things com filtro quadkey.

2.6.10 auth/__init__.py — Serviço de Autenticação

- Classe AuthenticationService:
 - Utiliza httpx.Client com verify=False (certificados autoassinados)
 - POST para o endpoint de token do Keycloak com password grant
 - Cache de tokens; renova a metade do tempo de vida do token
 - get_token() retorna um token válido (ou renova primeiro)
 - refresh() força a renovação do token

2.7 Modelos de Dados (ingestion/models/)

2.7.1 fire_incident.py

Define os modelos Pydantic para representar incidentes de incêndio, incluindo coordenadas geográficas, estados do ciclo de vida, geometria do cone de propagação e os schemas completos das Things do Ditto.

Modelo	Campos	Propósito
Point	lat: float, lon: float.	Coordenadas geográficas.
fireState (Enum)	NEW_IGNITION, NO_RISK, SIMULATING, SIMULATED, FAILED.	Máquina de estados do twin.
ConeSection	points: list[Point], horizon: int (minutos)	Polígono de um horizonte do cone.
ConeProperties	cone_path, cone_glb_path (URLs)	Feature Ditto para o cone.
PerimetersProperties.	URLs dos polígonos por passo + URL GLB.	Feature Ditto para os perímetros.
fireIncidentThing	thingId, policyId, attributes (state, ignition, polygon, expiry), features (cone, perimeters)	Payload completo da Thing Ditto.

Tabela 8 - Modelo — fire_incident.py

2.7.2 ditto_body_maker.py

Implementa o padrão builder para construir corpos JSON de Things Ditto de forma programática.

Builder pattern:

```
DittoBodyBuilder()
    .ignition(lat, lon)
    .cones({"cone_horizon.geojson": url, "fire_cone.glb": url})
```

```
.perimeters({...})
.polygon(risk_code, risk_polygon_geojson)
.fire_state(fireState.SIMULATING)
.build()
# → fireIncidentThing com Thing_id auto-gerado "fire:incident_<uuid>"
```

2.7.3 constants.py

Define constantes globais utilizadas nas simulações, incluindo os rótulos das frentes de fogo, o gradiente de cores para o modelo 3D e os parâmetros do ponto de ignição.

Constante	Valor	Propósito
FIRE_FRONTS	["0015", "0030", ..., "0120"]	8 rótulos de passo temporal (a cada 15 min)
FIREFRONT_COLORS	8 códigos hexadecimais de cor.	Gradiente dourado → castanho-avermelhado para modelo 3D.
IGNITION_DOT_RADIUS	5.0	Raio da esfera (metros)
IGNITION_COLOR	[0, 0, 255, 255]	Azul RGBA

Tabela 9 - Constante — constants.py

2.8 Arquitetura de Configuração

Todas as definições utilizam pydantic-settings com análise de prefixo de env:

```
class Config(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_nested_delimiter="__")

    log_level: str = "INFO"
    simulations_dir: str = "/data/simulations"
    risk_areas_dir: str = "/data/raw/brisa_zones.geojson"
    forefire_bin: str = "/usr/local/bin/forefire"
    ditto: DittoSettings = DittoSettings()
    auth: AuthSettings = AuthSettings()
    s3: s3Settings = s3Settings()
```

O delimitador __ mapeia variáveis de ambiente para campos aninhados: - DITTO__API_URL -> config.ditto.api_url - AUTH__USERNAME -> config.auth.username - S3__URL_EXTERNAL -> config.s3.url_external

2.9 Schema da Thing Ditto

Quando o worker atualiza um incidente de incêndio no Ditto, o corpo JSON é:

```
{
  "thingId": "fire:incident_<uuid>",
  "policyId": "dt4mob:policy",
  "attributes": {
```

```

    "ignition": { "lat": 39.5, "lon": -8.5 },
    "state": "SIMULATING",
    "polygon": ["http://s3/...", "http://s3/..."],
    "expiry": "2026-06-30T12:00:00Z"
  },
  "features": {
    "cone": {
      "properties": {
        "cone_horizon.geojson": "http://s3/...",
      }
    },
    "perimeters": {
      "properties": {
        "step_0015.geojson": "http://s3/...",
      }
    }
  }
}

```

2.10 Arquitetura Helm Chart

O chart (chart/) implementa os mesmos componentes no Kubernetes:

2.10.1 Recursos

Recurso	Kind	Descrição
API Deployment	Deployment	Executa o worker de ingestão (main.py). Contentor init wait-for-rasters verifica /data/processed/.done antes do contentor principal iniciar.
API Service	Service	Expõe a porta 8080. Usado pelo Ditto para callback (se necessário) ou para health checks.
Prepare Job	Job (Helm hook)	Executa prepare_rasters.py antes do deployment da API. Eliminado após sucesso via helm.sh/hook-delete-policy.
PVC	PersistentVolumeClaim.	20Gi, partilhado entre o Job e o Deployment.
Auth Secret	Secret	(Condicional) Contém username/password do Ditto. Montado em /run/secrets/dt4mob/creds/.

Recurso	Kind	Descrição
S3 Secret	Secret	Preenchido automaticamente pelo operador SeaweedFS com accessKey/secretKey.
S3 Identity	S3Identity (CRD)	Define um utilizador S3 SeaweedFS para o simulador de incêndio.
S3 Credentials	S3Credentials (CRD)	Liga a identidade ao S3 secret.
S3 Policies + Bindings.	S3Policy + S3PolicyBinding.	Concedem permissões S3 no bucket da app (dt4mob-public) e no bucket do job (fire-sim-briza-zones).

Tabela 10 - Recurso — Recursos

2.10.2 Padrão Init Container

Arranque do Pod:

1. Init Container (wait-for-rasters)
 - └ while ! [-f /data/processed/.done]; do sleep 5; done
2. Main Container (api)
 - └ python main.py

Isto garante que os rasters são preparados antes do worker de ingestão iniciar.

2.10.3 Configuração S3 via CRDs SeaweedFS

O chart utiliza recursos personalizados SeaweedFS em vez de secrets estáticos: - Um S3Identity cria um utilizador no cluster SeaweedFS - Um S3Credentials liga a identidade a um K8s Secret — o operador gera automaticamente accessKey/secretKey - S3Policy + S3PolicyBinding concedem permissões de leitura/escrita nos buckets necessários

Isto significa que não é necessário configurar credenciais S3 estáticas — são geradas dinamicamente pelo operador SeaweedFS.

2.10.4 Mapeamento Env: values.yaml → Contendor

O chart mapeia as chaves do values.yaml para variáveis de ambiente do contendor. Por exemplo:

```
# values.yaml
service:
  config:
    ditto:
      apiUrl: "http://dt4mob-ditto-gateway:8080"
    s3:
      UrlInternal: "http://seaweed-s3.seaweedfs.svc.cluster.local:8333"
```

Torna-se:

```
# api-deployment.yaml (template)
env:
  - name: DITTO__API_URL
    value: {{ .Values.service.config.ditto.apiUrl }}
  - name: S3__URL_INTERNAL
    value: {{ .Values.service.config.s3.UrlInternal }}
```

As credenciais são montadas como ficheiros em vez de variáveis de ambiente (por questões de segurança):

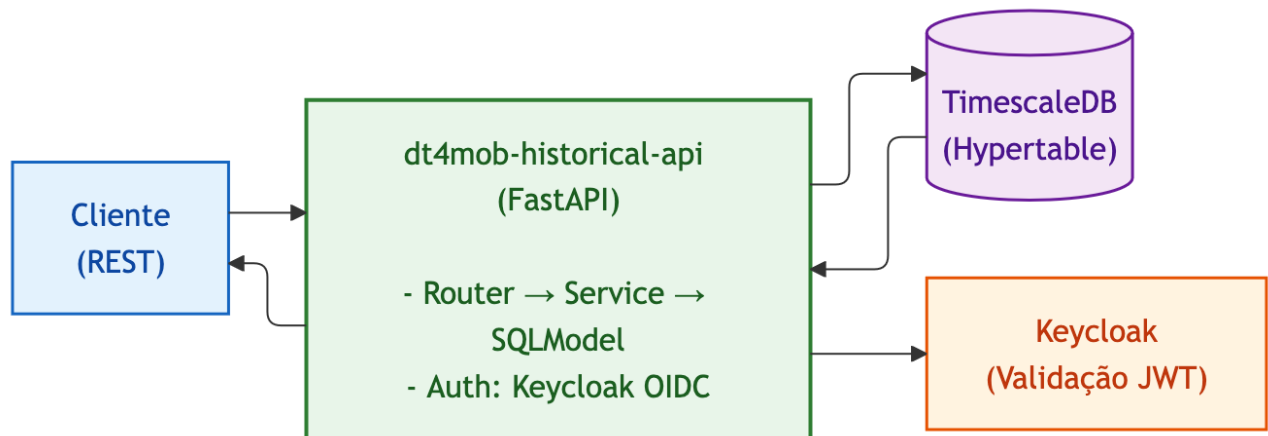
```
- AUTH__USERNAME_FILE=/run/secrets/dt4mob/creds/username
- AUTH__PASSWORD_FILE=/run/secrets/dt4mob/creds/password
```

2.11 Comparação Docker Compose vs Helm

Aspeto	Docker Compose	Helm (K8s)
Rede	network_mode: host	K8s Service (porta 8080)
Volumes	Bind mount ./data:/data.	PVC (20Gi, StorageClass local-path)
Preparação de rasters.	Serviço comentado (manual)	Job hook (executa automaticamente antes do deploy)
Espera por rasters	depends_on (comentado)	Init container (polling .done)
Secrets	Ficheiro .env	K8s Secret (montado como ficheiros)
Credenciais S3	Estáticas em .env	Auto-geradas via CRDs SeaweedFS.
ForeFire	Construído na imagem Docker.	Construído na imagem Docker.

Tabela 11 - Aspeto — Comparação Docker Compose vs Helm

2.12 Visão Geral da Arquitetura



A API é uma camada REST de leitura/escrita sobre TimescaleDB. Partilha o mesmo esquema de base de dados que o dt4mob-historical-writer (o serviço de ingestão Kafka), mas os dois são processos independentes.

2.13 Visão Geral

A Ditto Events API é uma API REST (FastAPI) que disponibiliza acesso de leitura e escrita a eventos históricos do Ditto armazenados no TimescaleDB. Partilha o mesmo esquema de base de dados que o serviço historical-writer. Disponibiliza endpoints para consultar, inserir e eliminar eventos, com projeção JSONPath e agregação temporal por intervalos. A autenticação é gerida via Keycloak OIDC.

2.14 Estrutura de Diretórios

```

dt4mob-historical-api/
├── app/
│   ├── __init__.py           # Vazio
│   └── main.py               # Ponto de entrada da aplicação FastA
├── PI
├── dependencies.py           # Vazio (reservado)
├── database_engine/
│   ├── __init__.py           # Singleton engine + get_session()
│   └── TimeScaleEngineManager.py # Wrapper SQLAlchemy engine
├── models/
│   ├── __init__.py           # Vazio
│   ├── ditto_events.py       # Tabela SQLAlchemy DittoEvent
│   ├── paths_response.py     # Modelo Pydantic PathResponseObject
│   └── util.py               # Enum Action
├── routers/
│   ├── __init__.py           # Vazio
│   └── events.py             # Todos os endpoints REST

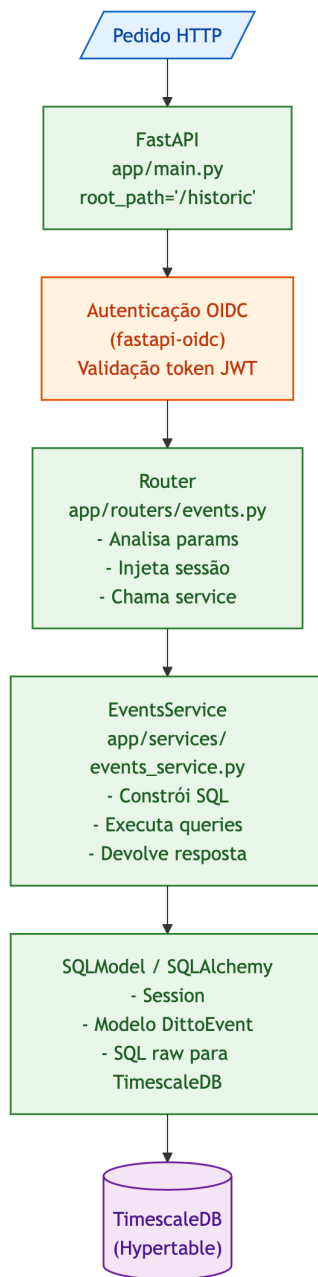
```

```

|   |   | services/
|   |   | |   __init__.py           # Vazio
|   |   | |   events_service.py    # Lógica de negócio / construção de qu
eries
|   |   | settings/
|   |   | |   __init__.py           # Settings global (combina auth + tim
escale)
|   |   | |   auth.py               # AuthSettings (Keycloak)
|   |   | |   timescale.py          # TimeScaleSettings (conexão DB)
|   |   | static/                   # Assets Swagger UI personalizados
|   |   | |   swagger-ui-bundle.js
|   |   | |   swagger-ui-standalone-preset.js
|   |   | |   swagger-ui.css
|   |   | |   swagger-initializer.js
|
|   dockerfile                      # Imagem Docker de produção
|   pyproject.toml                  # Dependências Python
|   uv.lock                          # Ficheiro de lock
|   USER_MANUAL.en.md
|   USER_MANUAL.pt.md
|   DEVELOPER_MANUAL.en.md
|   DEVELOPER_MANUAL.pt.md

```

2.15 Fluxo de Dados



2.16 Detalhes dos Módulos

2.16.1 app/main.py — Ponto de Entrada

Cria a aplicação FastAPI com: - root_path="/historic" — todos os routes são prefixados com /historic - default_response_class=ORJSONResponse — serialização JSON rápida via orjson - Swagger UI personalizado servido em /historic/ usando assets estáticos - Sem docs/redoc URLs automáticos (apenas Swagger personalizado)

```

app = FastAPI(
    root_path=root_path,
    docs_url=None,
    redoc_url=None,

```

```

    default_response_class=ORJSONResponse,
    title="Ditto Events API",
    description="API to manage historical events from Eclipse Ditto",
)

```

2.16.2 app/routers/events.py — Endpoints API

Este ficheiro define todos os endpoints REST e as suas dependências.

Dependências principais:

- `auth`: Instância devolvida por `get_auth()` do `fastapi-oidc`. Configura a validação OIDC com as definições
- `get_events_service(session)`: Dependência FastAPI que cria um `EventsService` com uma sessão de base de dados
- `check_role(roles)`: Devolve uma função de dependência que inspeciona o `resource_access` do token Keycloak para verificar as roles necessárias

Lógica de verificação de roles (`check_role`):

```

def check_role(roles: list[str]):
    def check_role_inner(token: KeycloakToken):
        if settings.auth.client_id not in token.resource_access:
            raise HTTPException(403)
        token_roles = token.resource_access[settings.auth.client_id].roles
        if not any(role in token_roles for role in roles):
            raise HTTPException(403)
    return check_role_inner

```

Modelo de token (`KeycloakToken`):

```

class KeycloakToken(IDToken):
    resource_access: dict[str, ClientResourceAccess] = {}

```

`ClientResourceAccess`:

```

class ClientResourceAccess(BaseModel):
    roles: list[str] = []

```

Lista de endpoints (todos sob `APIRouter` com `dependencies=[Depends(auth)]`):

Função do Endpoint	Path	Método	Método do Service
<code>read_available_Things.gs.</code>	<code>/Things</code>	GET	<code>get_available_Things()</code>
<code>read_event_paths</code>	<code>/events/{Thing_id}/paths.</code>	GET	<code>get_event_paths_with_action()</code>
<code>read_events</code>	<code>/events</code>	GET	<code>get_events()</code>
<code>read_events_by_Thing.gs.</code>	<code>/events/{Thing_id}</code>	GET	<code>get_events_by_Thing()</code>

Função do Endpoint	Path	Método	Método do Service
insert_events	/events	POST	insert_events()
delete_events	/events/{Thing_id}	DELETE	delete_events()
read_jsonpath_projection.	/events/projection/{Thing_id}.	GET	get_jsonpath_projection()
read_events_custom_time_buckets.	/events/time-buckets/{Thing_id}.	GET	get_events_custom_time_buckets() ou get_events_custom_time_buckets_with_path()

Tabela 12 - Função de Endpoint — *app/routers/events.py*

2.16.3 app/services/events_service.py — Lógica de Negócio

A classe `EventsService` encapsula todas as operações de base de dados. Recebe uma `Session` do `SQLModel` por injeção no construtor.

2.16.3.1

Métodos:

- `get_available_Things(Thing_id_prefix)` - SQL: `SELECT DISTINCT Thing_id FROM dittoevent [WHERE Thing_id LIKE prefix%]` - Devolve lista de strings `Thing_id` distintas
- `get_event_paths_with_action(Thing_id, since, until)` - SQL: `SELECT DISTINCT path, action FROM dittoevent WHERE Thing_id = :id AND time BETWEEN :since AND :until` - Devolve `list[PathResponseObject]`.
- `get_events(since, until, Thing_id_prefix, path_prefix, action, limit)` - SQL: `SELECT * FROM dittoevent WHERE time BETWEEN :since AND :until [AND Thing_id LIKE prefix%] [AND action = :action] [AND path LIKE prefix%] ORDER BY time DESC [LIMIT :limit]` - Devolve `list[DittoEvent]`.
- `get_events_by_Thing(Thing_id, since, until, path_prefix, action)` - Igual a `get_events` mas filtrado por `Thing_id` exato.
- `insert_events(events)` - Itera sobre os `events`, adiciona cada um à sessão, faz `commit`. - Devolve `None`.
- `delete_events(Thing_id, since, until)` - SQL: `DELETE FROM dittoevent WHERE Thing_id = :id AND time BETWEEN :since AND :until` - Devolve contagem de linhas apagadas.
- `get_jsonpath_projection(json_path, Thing_id, since, path_prefix, until)` - Usa a função PostgreSQL `jsonb_path_query()` para extrair valores que correspondem a uma expressão `JSONPath`. - SQL (simplificado): `SELECT jsonb_path_query(value, CAST(:json_path AS JSONPATH)) FROM dittoevent WHERE Thing_id = :id AND time BETWEEN :since AND :until [AND path LIKE prefix%]`

- `get_events_custom_time_buckets(json_path, since, until, bucket_minutes, agg_type, Thing_id)` - Agrupa events em intervalos temporais e aplica uma função de agregação sobre um valor extraído via JSONPath. - Modo all-time (`bucket_minutes <= 0` ou `None`): Devolve um único intervalo com `MIN(time)` como rótulo. - Modo intervalo: Usa aritmética `EXTRACT(epoch FROM time) / bucket_seconds` para calcular os limites dos intervalos, depois `to_timestamp()` para converter de volta. - Funções de agregação: `count`, `sum`, `avg`, `min`, `max`. - Para agregações numéricas (`sum`, `avg`, `min`, `max`), o valor extraído é convertido para `Float`. - Tratamento de erros: - `DataError` com “cannot cast jsonb object to double precision” → 400 com mensagem explicativa. - `ProgrammingError` (sintaxe JSONPath inválida) → 400. - Ambos acionam `session.rollback()`.
- `get_events_custom_time_buckets_with_path(json_path, since, until, path_prefixes, bucket_minutes, agg_type, Thing_id)` - Igual ao anterior mas também agrupa por `DittoEvent.path`. - `path_prefixes` é uma string separada por vírgulas dividida num array e usada numa condição OR via `sqlalchemy.or_()`. - Devolve resultados segmentados por path.

2.16.4 app/models/ditto_events.py — Modelo de Base de Dados

```
class DittoEvent(SQLModel, table=True):
    time: datetime = Field(primary_key=True)           # Coluna de partição (
hypertable)
    index: Optional[int] = Field(..., Identity(...))  # Auto-incremento, tam
bém PK
    Thing_id: str = Field(index=True)                 # Indexado para filtra
gem
    action: Action = Field(sa_column=Column(SAEnum))  # created/modified/del
eted/merged
    revision: PositiveInt | None
    path: str                                           # Caminho da feature D
itto
    value: dict = Field(sa_type=JSONB)                 # Payload JSON
```

Configuração TimescaleDB (definida via `__timescale_config__` e um listener de evento `after_create`):

Propriedade	Valor
<code>time_column</code>	<code>time</code>
<code>chunk_time_interval</code>	1 dia
<code>compress</code>	Sim
<code>compress_segmentby</code>	<code>Thing_id</code>
<code>compress_orderby</code>	<code>time DESC</code>
<code>compress_interval</code>	7 dias (política de compressão)

Tabela 13 - Propriedade — `app/models/ditto_events.py`

A função `create_timescale_features()` (registada via `@event.listens_for(...,`

"after_create")) executa três comandos SQL quando a tabela é criada: 1. `SELECT create_hypertable(...)` — converte a tabela numa hipertabela 2. `ALTER TABLE ... SET (timescaledb.compress, ...)` — ativa compressão nativa 3. `SELECT add_compression_policy(...)` — define a agenda de compressão automática.

Este modelo está duplicado no `dt4mob-historical-writer` para manter os serviços independentes.

2.16.5 `app/models/paths_response.py` — Modelo de Resposta

```
class PathResponseObject(BaseModel):
    path: str
    action: Action
    model_config = {"from_attributes": True}
```

Usado pelo endpoint `/events/{Thing_id}/paths`.

2.16.6 `app/models/util.py` — Enum Action

```
class Action(str, Enum):
    MODIFIED = "modified"
    CREATE = "created"
    DELETE = "deleted"
    MERGED = "merged"
```

Usado como tipo de coluna e como parâmetro de filtro em queries.

2.16.7 `app/database_engine/__init__.py` — Singleton Engine

```
timescale_engine = TimescaleDBEngineManager()
timescale_engine.create_db_tables()
```

- Cria o engine SQLAlchemy uma vez aquando da importação do módulo.
- Chama `SQLModel.metadata.create_all(engine)` — isto cria a tabela `dittoevent` e aciona o evento `after_create` que configura a hipertabela `TimescaleDB`.
- **Importante: Importa o modelo `DittoEvent` antes de `create_db_tables()` para garantir que o modelo está registado em `SQLModel.metadata`.**
- `get_session()` é um gerador que produz uma `Session`, faz `rollback` em caso de exceção e fecha no `finally`.

2.16.8 `app/database_engine/TimeScaleEngineManager.py` — Wrapper do Engine

```
class TimescaleDBEngineManager:
    def __init__(self):
        self.engine = create_engine(
            url=settings.timescale.get_connection(),
            echo=(settings.log_level == "DEBUG"),
        )

    def create_db_tables(self):
        SQLModel.metadata.create_all(self.engine)
```

- `get_connection()` substitui `postgresql://` por `postgresql+psycopg://` para usar o driver `psycopg`.
- Engine echo ativado apenas quando `LOG_LEVEL=DEBUG`.

2.16.9 App/settings/__init__.py — Configuração Global

```
class Settings(BaseSettings):
    log_level: LogLevel = Field(validation_alias="LOG_LEVEL", default="INFO")
    auth: AuthSettings
    timescale: TimeScaleSettings

    model_config = SettingsConfigDict(
        env_nested_delimiter="__", # Permite sintaxe AUTH__CLIENT_ID
        extra="ignore",
    )
```

- Singleton: `settings = Settings()` criado ao nível do módulo.
- Lê de variáveis de ambiente ou ficheiro `.env`.
- O delimitador `__` permite aninhamento: `AUTH__CLIENT_ID` → `settings.auth.client_id`.

2.16.10 app/settings/auth.py – Configuração de Autenticação

```
class AuthSettings(BaseModel):
    client_id: str
    audience: str = "account"
    server_uri: str
    issuer: str | list[str]
    signature_cache_ttl: PositiveInt = 3600
    read_role: list[str] = ["historical-read"]
    write_role: list[str] = ["historical-write"]
```

2.16.11 app/settings/timescale.py — Configuração do Timescale

```
class TimeScaleSettings(BaseModel):
    database_timezone: str = "Europe/Lisbon"
    connection: str

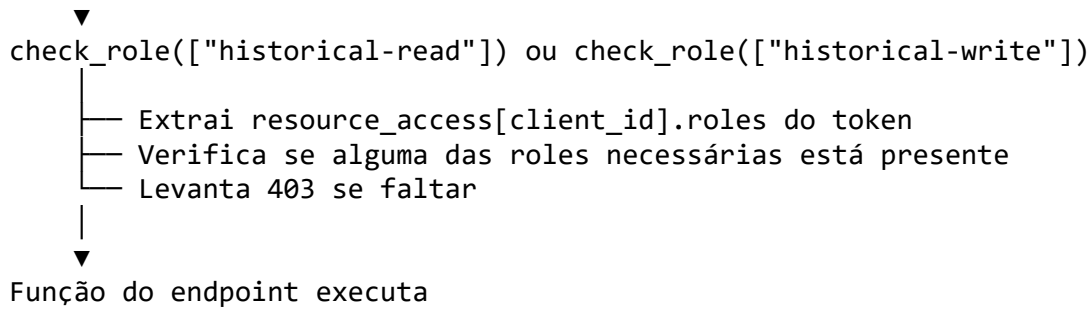
    def get_connection(self):
        return self.connection.replace("postgresql", "postgresql+psycopg")
```

2.17 Fluxo de Autenticação

Pedido do Cliente (com Bearer JWT)

▼
Middleware `fastapi-oidc`

- Valida assinatura JWT (cache por `signature_cache_ttl` segundos)
- Verifica issuer contra `AUTH__ISSUER`
- Decodifica token para modelo `KeycloakToken`



A biblioteca `fastapi-oidc` é configurada com `token_type=KeycloakToken` para que o token decodificado esteja disponível como um modelo Pydantic tipado. A dependência personalizada `check_role()` é usada por endpoint através de `Depends()`.

2.18 Estratégia de Tratamento de Erros

Cenário	Handler	HTTP Status	Detalhe da Resposta
JWT ausente/inválido.	<code>fastapi-oidc</code>	401/403	Automático
Role em falta no token.	<code>check_role()</code>	403	"Missing role"
Erro de cast JSONB→Float (time-buckets)	<code>events_service.py</code>	400	Sugere usar count ou corrigir o JSONPath.
Sintaxe JSONPath inválida.	<code>events_service.py</code>	400	Devolve a mensagem de erro do PostgreSQL.
Erro genérico de BD	<code>SQLAlchemy</code>	500	Propagado

Tabela 14 - Cenário — Estratégia de Tratamento de Erros

Na camada de serviço, erros que ocorrem durante a execução de queries acionam `session.rollback()` para reverter quaisquer alterações não confirmadas antes de levantar a exceção HTTP.

2.19 Dependências

2.19.1 Pacotes Python (de `pyproject.toml`)

Pacote	Versão	Propósito
<code>fastapi[standard]</code>	≥0.128.0	Framework web
<code>fastapi-oidc</code>	≥0.0.11	Middleware de autenticação Keycloak OIDC.

Pacote	Versão	Propósito
orjson	≥3.11.7	Serialização JSON rápida.
psycopg[binary]	≥3.3.2	Adaptador PostgreSQL.
pydantic-settings	≥2.12.0	Configuração baseada em ambiente.
sqlmodel	≥0.0.31	Integração Pydantic + SQLAlchemy ORM.

Tabela 15 - Pacotes Python

2.19.2 Serviços Externos

Serviço	Protocolo	Propósito
TimescaleDB	PostgreSQL (psycopg)	Persistência de dados (hipertabela)
Keycloak	HTTPS (OIDC)	Validação de tokens JWT.

Tabela 16 - Serviços Externos

2.20 Deployment

2.20.1 Dockerfile

- **Imagem base:** python:3.14-slim
- **Gestor de pacotes:** uv 0.9.27 (copiado de [ghcr.io/astral-sh/uv:0.9.27](https://github.com/astral-sh/uv))
- **Ficheiros estáticos:** Copiados explicitamente de ./app/static
- **Comando:** fastapi run app/main.py --port 8080 --host 0.0.0.0

2.20.2 Helm Chart

O chart em `charts/dt4mob-historical/` implanta a API e o writer em conjunto:

- **API Deployment:** Executa a imagem `data-lake-api` na porta 8080.
- **TimescaleDB Cluster:** Gerido pelo operador Cloud Native PostgreSQL (CNPG) com extensão TimescaleDB.
- **Kafka Topic:** Gerido pelo operador Strimzi.

O chart faz o deploy de todos os recursos necessários e inclui um `values.yaml` que permite personalizar variáveis de ambiente. O deploy é automático com o helm chart da infraestrutura, pois o chart do histórico é dependência do mesmo.

2.20.3 Imagens dos Contentores

Registry: atnog-harbor.av.it.pt/dt4mob/ - API: data-lake-api - Writer: data-gateway. # Garbage-Collector

2.21 Visão Geral

O Garbage Collector é um job batch Python que se liga ao Eclipse Ditto via WebSocket e REST API, pesquisa por digital twins cujo `attributes/expiry_ts` expirou e elimina-os através do envio de comandos delete do protocolo Ditto. Foi concebido para ser executado como um Kubernetes CronJob.

2.22 Stack Tecnológica

Componente	Tecnologia
Linguagem	Python 3.13
Gestor de pacotes	uv (Astral)
Runtime assíncrono	asyncio (stdlib)
Cliente WebSocket	websockets 16.0
Cliente HTTP	httpx 0.28.1 (async)
Configuration	pydantic-settings 2.13.0.
Validação de dados	pydantic 2.12.5
Deployment	Docker (Alpine) + Helm (Kubernetes CronJob)

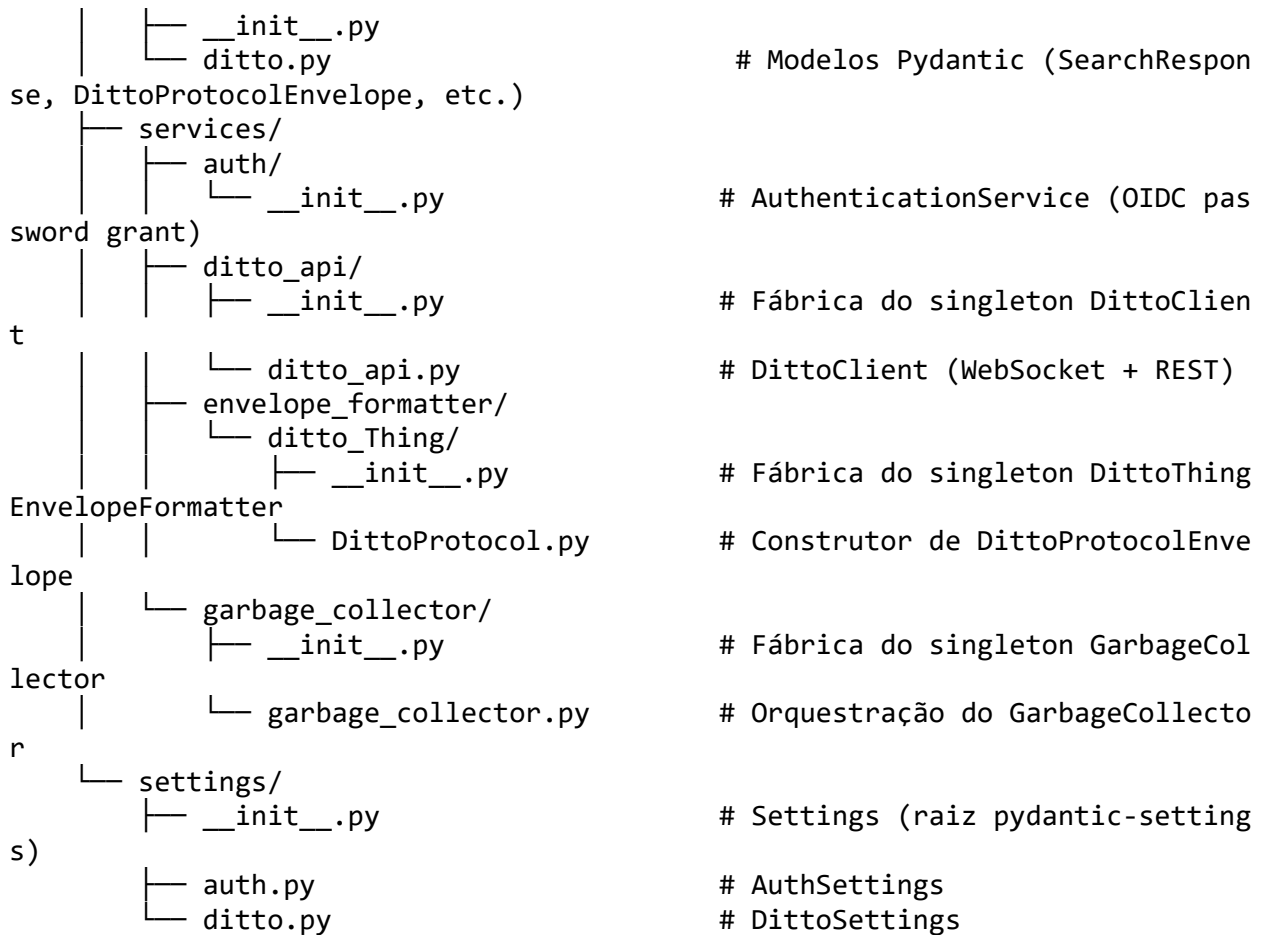
Tabela 17 - Stack Tecnológica

2.23 Estrutura do Projeto

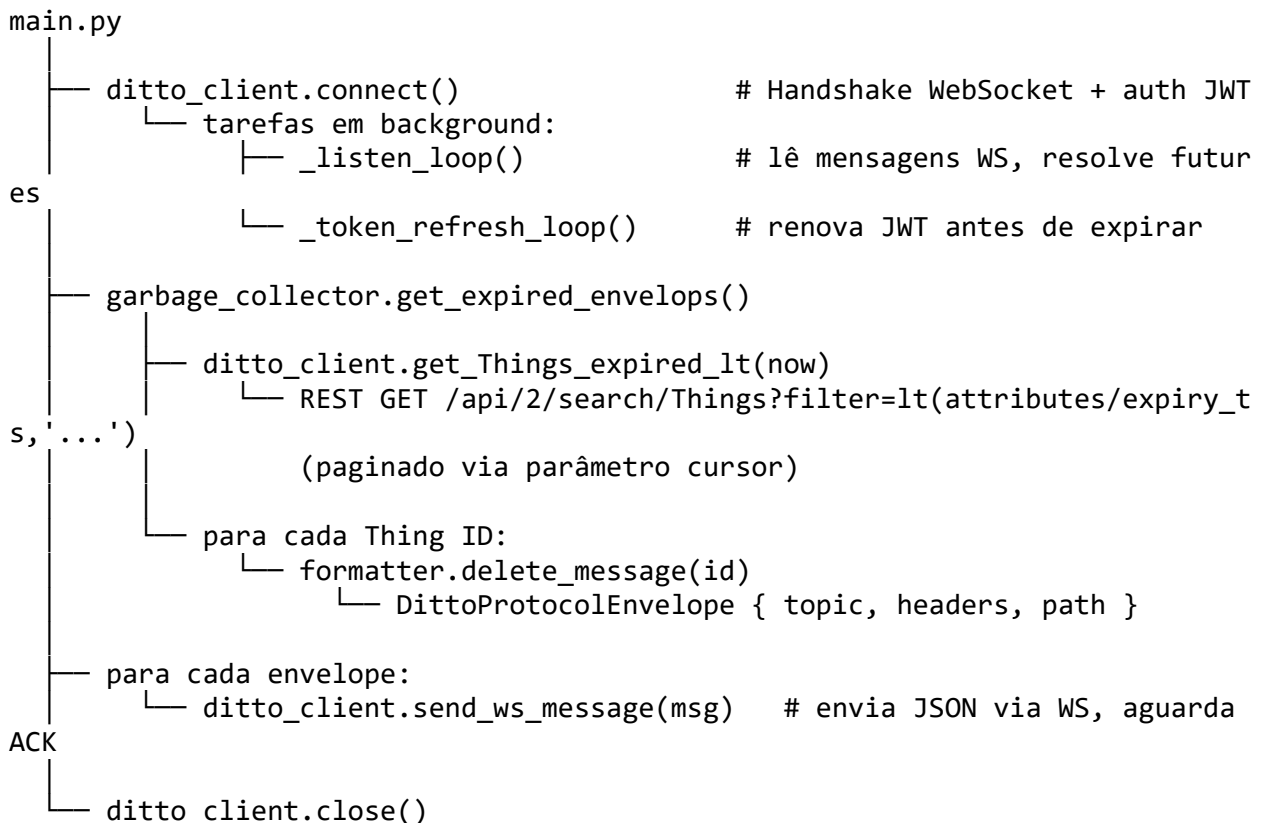
```

dt4mob-garbage-collector/
├── main.py                # Ponto de entrada
├── pyproject.toml         # Metadados do projeto e dependê
ncias
├── uv.lock                # Versões bloqueadas das dependê
ncias
├── Dockerfile             # Build do contentor
├── .python-version        # Versão do Python
├── README_DEV.md
├── README_DEV_PT.md
├── README_USER.md
├── README_USER_PT.md
├── src/
│   └── models/

```



2.24 Arquitetura e Fluxo de Dados



2.25 Detalhes de Implementação

2.25.1 Singleton Pattern

Cada módulo de serviço expõe um singleton ao nível do módulo através de uma fábrica `_new_instance()`:

- `src/services/auth/__init__.py` → `auth_service`
- `src/services/ditto_api/__init__.py` → `ditto_client`
- `src/services/envelope_formatter/ditto_Thing/__init__.py` → `envelop_formatter`
- `src/services/garbage_collector/__init__.py` → `garbage_collector`

As dependências são resolvidas no momento da construção e passadas aos consumidores downstream.

2.25.2 Ciclo de Vida do WebSocket & Token

1. `DittoClient.connect()` abre um WebSocket para `{ws_url}/?x-csrf-token=no-csrf-check`.
2. Envia a mensagem de controlo `START-SEND-EVENTS`.
3. Inicia duas tarefas `asyncio` em background:
 - `_listen_loop()`: lê todas as mensagens recebidas. Se a mensagem for uma resposta JSON com um `correlation-id`, resolve a future correspondente em `_responses`. Mensagens de controlo não-JSON são registadas.
 - `_token_refresh_loop()`: calcula o tempo de vida restante do token, espera até 5 segundos antes da expiração, obtém um novo token do endpoint OIDC e envia-o como mensagem de controlo: `JWT-TOKEN?jwtToken=<new_token>`.

2.25.3 Correlation-id Matching

Cada envelope enviado contém um UUID em `headers.correlation-id`. O remetente armazena uma future indexada por este UUID. O loop de escuta associa respostas recebidas pelo mesmo `correlation-id` e resolve a future, permitindo uma semântica fiável de enviar-e-aguardar sobre WebSocket.

2.25.4 Pagination

`get_Things_expired_lt()` executa uma pesquisa paginada usando o mecanismo de cursor do Ditto. Solicita `DittoSettings.search_size` itens por página (200 por omissão, máximo 500 hard-coded). O loop continua até não haver cursor `returnsdo`.

2.25.5 Loop de Recolha de Lixo

Em `main.py`, após obter a lista de envelopes de eliminação, o script itera sobre eles enviando cada um via WebSocket. O ciclo `while` exterior volta a pesquisar Things expiradas após cada lote e continua até restarem menos de 5 envelopes, capturando quaisquer novas expirações desde o início do script.

2.26 Dependências

2.26.1 Diretas

Pacote	Versão	Propósito
httpx	>=0.28.1	Cliente HTTP assíncrono para a REST API do Ditto.
pydantic	>=2.12.5	Validação de dados e modelos.
pydantic-settings	>=2.13.0	Configuração baseada em variáveis de ambiente.
websockets	>=16.0	Cliente WebSocket assíncrono.

Tabela 18 - Dependências Diretas

2.27 Configuração

Todas as definições são carregadas via `pydantic-settings` a partir de variáveis de ambiente utilizando `__` como delimitador aninhado. Consulte `README_USER_PT.md` para a tabela completa de variáveis de ambiente.

2.28 Configuração de Desenvolvimento

```
# Clone e entre no diretório
cd services/dt4mob-garbage-collector

# Instalar uv se não estiver instalado (https://docs.astral.sh/uv/)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Instalar dependências
uv sync --locked

# Executar
uv run main.py
```

Para execução local, fornecer variáveis de ambiente (via ficheiro `.env` ou inline):

```
DITTO__API_URL=http://localhost:8080 \
AUTH__TOKEN_ENDPOINT=https://keycloak/auth/realms/dt4mob/protocol/openid-connect/token \
AUTH__USERNAME=myuser \
AUTH__PASSWORD=mypass \
uv run main.py
```

2.29 Build

```
docker build -t dt4mob-garbage-collector:latest .
```

O Dockerfile usa `alpine:edge` com Python 3.13, copia o `uv` de `astral/uv:latest`, instala dependências de build, executa `uv sync --locked` e define CMD `["uv", "run", "main.py"]`.

2.30 Kubernetes e Helm

O deploy deste serviço é realizado através de um Helm Chart localizado neste repositório em `/charts/dt4mob-garbage-collector/`.

O chart define um **CronJob** que executa periodicamente de acordo com a expressão cron configurada no ficheiro `values.yaml`. Além disso, inclui dois templates:

- `cronjob` – define o recurso Kubernetes CronJob responsável pela execução periódica do serviço;
- `secret` – cria o Secret que contém as credenciais utilizadas para autenticação no Keycloak.

Uma vez que este serviço é um componente essencial da infraestrutura, o seu Helm Chart está definido como dependência do Helm Chart da infraestrutura. Como resultado, o serviço é automaticamente instalado e atualizado sempre que a infraestrutura é provisionada ou atualizada.

O ficheiro `values.yaml` permite configurar os principais parâmetros do serviço:

```
image:
  repository: "atnog-harbor.av.it.pt/dt4mob/garbage-collector"
  tag: "latest"
  imagePullPolicy: "Always"

config:
  schedule: "* */5 * * * *"
  logLevel: "INFO"
  ditto:
    apiUrl: "ditto-gateway:8080"
    baseApiPath: "/api/2"
    baseWsPath: "/ws/2"
  auth:
    tokenEndpoint: "https://dt4mob-keycloak:8443/auth/realms/dt4mob/protocol/
openid-connect/token"
    # username: ""
    # password: ""
    existingSecret: ""
    caSecret: ""
```

Os parâmetros configuráveis incluem:

- `image` – especifica a imagem do contentor, a respetiva versão (tag) e a política de obtenção da imagem (`imagePullPolicy`);
- `config.schedule` – expressão cron que define a periodicidade de execução do CronJob;
- `config.logLevel` – nível de detalhe dos logs produzidos pelo serviço;
- `config.ditto` – configura os endpoints utilizados para comunicar com o Eclipse Ditto;
- `config.auth` – configura os parâmetros necessários para autenticação no Keycloak, incluindo o endpoint para obtenção do token e os Secrets utilizados para disponibilizar as

credenciais e o certificado da autoridade certificadora (CA).

3 HISTORICAL-WRITER

3.1 Visão Geral

O Historical Writer é um serviço Python que consome mensagens do protocolo Ditto a partir do Kafka e persiste-as no TimescaleDB. Atua como o pipeline de ingestão para dados históricos de eventos e partilha o esquema de base de dados DittoEvent com o serviço de API histórica.

3.2 Estrutura do Projeto

```

dt4mob-historical-writer/
├── main.py                                # Ponto de entrada
├── pyproject.toml                        # Dependências (confluen
t-kafka, sqlalchemy, etc.)
├── dockerfile                            # Imagem Docker de desen
volvimento (Alpine + uv)
├── dockerfile.prod                       # Imagem Docker de produ
ção (sem dev deps)
├── .python-version                       # Python 3.12
├── .env                                  # Configuração local (gi
tignored)
├── uv.lock                               # Lockfile
├── USER_MANUAL_EN.md
├── USER_MANUAL_PT.md
├── DEVELOPER_MANUAL_EN.md
├── DEVELOPER_MANUAL_PT.md
├── src/
│   ├── settings/
│   │   ├── __init__.py                # Configurações globais (pydantic-settings),
carregadas do .env
│   │   ├── kafka.py                  # KafkaSettings – broker, SSL, grupo de cons
umidores, tópico
│   │   └── timescale.py              # TimeScale – string de conexão, fuso horári
o
│   ├── models/
│   │   ├── __init__.py
│   │   └── ditto_events.py           # DittoEvent SQLAlchemyModel, enum Action, config h
ipertabela Timescale
│   ├── database_engines/
│   │   ├── __init__.py               # Singleton engine + criação de tabelas ao i
mportar
│   │   └── TimeScaleEngineManager.py # Engine SQLAlchemy, fábrica de sessõe
s, create_db_tables
│   └── services/
│       ├── __init__.py
│       ├── KafkaConsumer/
│       │   ├── __init__.py           # Re-exporta KafkaConsumer
│       │   └── kafka_consumer.py     # Loop de poll do Kafka, decodificação JS
ON, dispatch
│       └── MessageProcessor/
│           ├── __init__.py

```



```

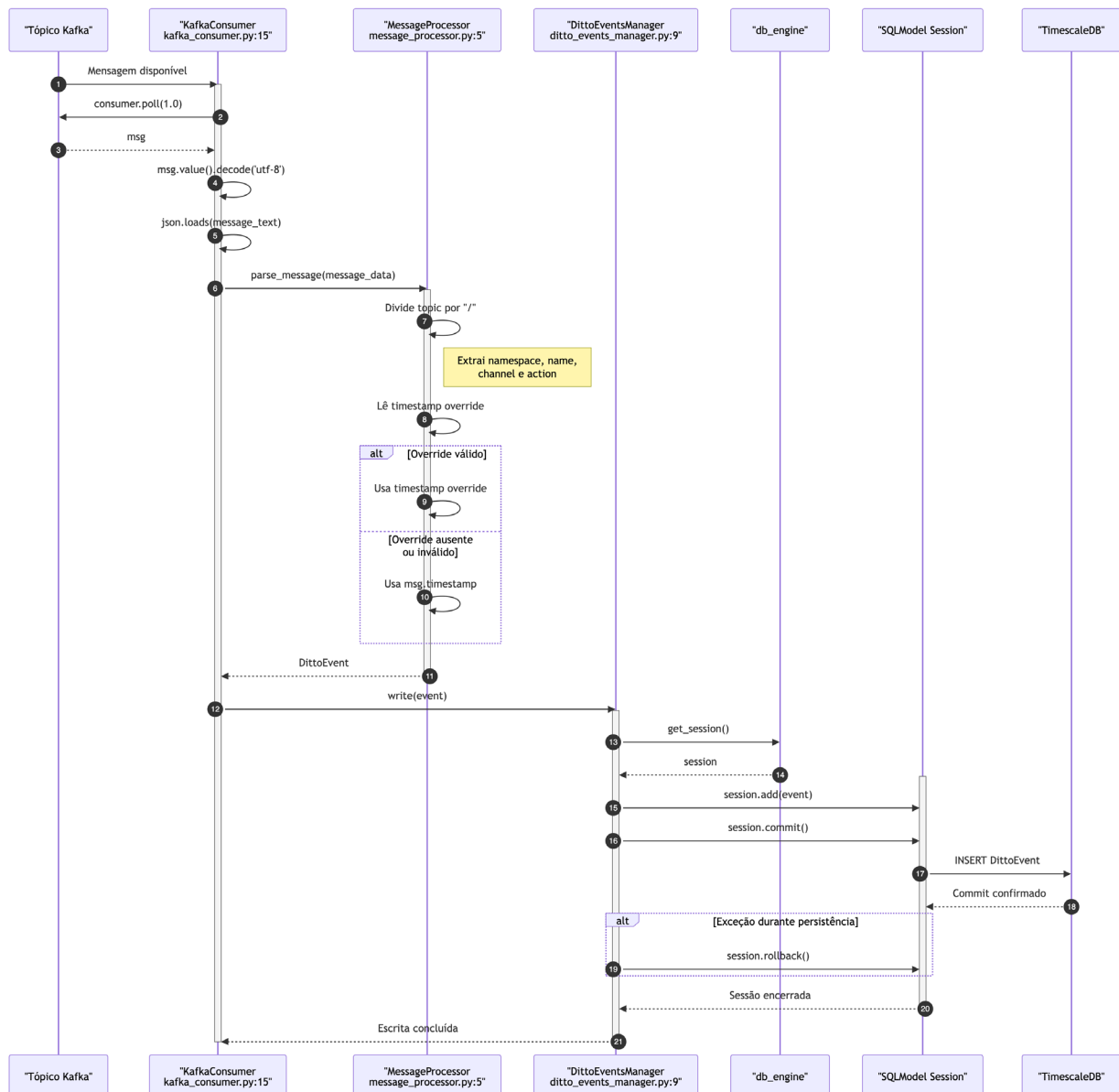
    |   └─ message_processor.py # Interpreta mensagens Ditto para objet
os DittoEvent
    └─ DittoEventsManager/
        └─ __init__.py # Cria singleton DittoEventsManager com o en
gine
        └─ ditto_events_manager.py # write(event) – session.add + commi
t

```

3.3 Sequência de Inicialização

- `main.py:13` — `if __name__ == "__main__": main()` invoca `main()`
- `src/database_engines/__init__.py:4-5` — ao importar, cria o singleton `TimescaleDBEngineManager()` e chama `create_db_tables()`, que executa `SQLModel.metadata.create_all(self.engine)`, emitindo `CREATE TABLE` e o evento `after_create`
- `src/services/DittoEventsManager/__init__.py:2-4` — ao importar, instancia o singleton `DittoEventsManager(db_engine=timescale_engine)`
- `main.py:6` — `KafkaConsumer()` subscreve o tópico Kafka configurado
- `main.py:9` — `consumer.consume(ditto_event_manager)` entra no loop infinito de polling

3.4 Fluxo de Mensagens (Ponta a Ponta)



3.5 Detalhes dos Módulos

3.5.1 Settings (src/settings/)

Usa **pydantic-settings** com `SettingsConfigDict(env_file=".env")`. O singleton settings global é construído ao importar (`src/settings/__init__.py:28`).

`KafkaSettings.as_dict()` (`src/settings/kafka.py:20`) converte os campos Pydantic para o formato de chaves pontuadas exigido pelo `confluent_kafka.Consumer`: `- bootstrap.servers`, `security.protocol`, `ssl.ca.location`, `ssl.certificate.location`, `ssl.key.location`, `group.id`, `auto.offset.reset` - `enable.auto.commit` está definido como `False` — os commits são manuais após cada mensagem processada com sucesso.

`TimeScale.get_connection()` (`src/settings/timescale.py:14`) substitui `postgresql://` por `postgresql+psycopg://` para que o SQLAlchemy use o driver psycopg.

3.5.2 KafkaConsumer (src/services/KafkaConsumer/kafka_consumer.py)

- `__init__`: Cria um `confluent_kafka.Consumer` com `KafkaSettings.as_dict()` e subscreve o tópico configurado.
- `consume(manager)`: Loop infinito que chama `consumer.poll(1.0)`. Em cada mensagem:
 1. Descodifica e interpreta o JSON.
 2. Chama `parse_message()` para construir um `DittoEvent`.
 3. Chama `manager.write(event)` para persistir.
 4. Chama `self.consumer.commit(asynchronous=False)` — **commit síncrono manual** garante entrega pelo menos uma vez. Se o processo falhar entre `write` e `commit`, a mensagem será reentregue, podendo causar duplicados.

3.5.3 MessageProcessor (src/services/MessageProcessor/message_processor.py)

`parse_message(msg)` extrai dados estruturados de uma mensagem Ditto:

4. **Interpretação do tópico**: `msg["topic"]` é dividido por `"/"`. Formato esperado: `{namespace}:{name}/Things/{channel}/{action}`. O `Thing_id` é remontado como `namespace:name`, e a `action` é o 6º elemento (índice 5).
5. **Substituição de timestamp**: Se o cabeçalho `dt4mob-historic-timestamp-override` estiver presente e for uma string ISO-8601 válida, substitui o timestamp original da mensagem. Caso contrário, o `msg["timestamp"]` original é usado.
6. Retorna um `DittoEvent` com `time`, `Thing_id`, `action`, `path`, `revision`, `value`.

3.5.4 Modelo DittoEvent (src/models/ditto_events.py)

Enum Action: `modified`, `created`, `deleted`, `merged`.

`DittoEvent SQLAlchemy (tabela dittoevent)`: - Chave primária composta: `time` (datetime, 1ª parte) + `index` (auto-incremento Identity, 2ª parte) - `Thing_id`: indexado para performance de consultas - `action`: armazenado como enum nativo PostgreSQL (`action_enum`) - `value`: armazenado como JSONB para dados aninhados flexíveis

Configuração da hipertabela Timescale (timescale_config):

Parâmetro	Valor	Descrição
<code>time_column</code>	"time"	Coluna de particionamento
<code>chunk_time_interval</code>	"1 days"	Cada chunk cobre 1 dia
<code>compress</code>	True	Ativa compressão nativa
<code>compress_segmentby</code>	"Thing_id"	Linhas comprimidas segmentadas por Thing
<code>compress_orderby</code>	"time DESC"	Ordenação dentro dos segmentos comprimidos

Parâmetro	Valor	Descrição
compress_interval	"7 days"	Política de compressão aplicada a dados com mais de 7 dias

Tabela 19 - Configuração da hipertabela Timescale

O ouvinte do evento `after_create` (`src/models/ditto_events.py:42`) executa automaticamente estas instruções SQL quando a tabela é criada:

```
SELECT create_hypertable('dittoevent', 'time', chunk_time_interval => INTERVAL '1 days', if_not_exists => TRUE);
```

```
ALTER TABLE dittoevent SET (timescaledb.compress, timescaledb.compress_segmentby = 'Thing_id', timescaledb.compress_orderby = 'time DESC');
```

```
SELECT add_compression_policy('dittoevent', INTERVAL '7 days', if_not_exists => TRUE);
```

3.5.5 TimeScaleDBEngineManager (src/database_engines/TimeScaleEngineManager.py)

- `__init__`: Cria uma Engine SQLAlchemy usando a string de conexão das definições. `echo=True` quando `LOG_LEVEL=DEBUG`.
- `create_db_tables()`: Chama `SQLModel.metadata.create_all(self.engine)` que emite `CREATE TABLE` e aciona o evento `after_create`.
- `get_session()`: Um `@contextmanager` que produz uma `Session`, faz auto-commit em caso de sucesso e auto-rollback em caso de exceção.

3.5.6 DittoEventsManager (src/services/DittoEventsManager/ditto_events_manager.py)

Wrapper leve em torno do motor de base de dados. O método `write(event)` abre uma sessão, adiciona o evento e faz commit. A sessão é obtida através do context manager de `TimeScaleDBEngineManager.get_session()`.

3.6 Dependências

Pacote	Versão	Propósito
confluent-kafka	>= 2.13.0	Consumidor Kafka (C-based, alta performance)
psycopg[binary]	>= 3.3.2	Adaptador PostgreSQL (compatível com async)
pydantic	>= 2.12.5	Validação de dados
pydantic-settings	>= 2.12.0	Gestão de definições a partir

Pacote	Versão	Propósito
		de variáveis de ambiente.
sqlalchemy	>= 2.0.45	Núcleo ORM
sqlmodel	>= 0.0.31	Integração Pydantic + SQLAlchemy, definição de tabelas.

Tabela 20 - Dependências

3.7 Build Docker

Dois Dockerfiles usando **Alpine Edge + uv**:

- `dockerfile (dev)`: executa `uv sync --locked` (instala todas as dependências, incluindo dev).
- `dockerfile.prod (prod)`: define `ENV UV_NO_DEV=1` antes de `uv sync --locked` para excluir dependências de desenvolvimento.

Ambos instalam `build-base`, `python3-dev`, `librdkafka-dev` (para a extensão C do `confluent-kafka`). O CMD é `uv run main.py`.

O contentor espera que os certificados SSL estejam montados em `/certs/` (ou caminhos personalizados).

3.8 Deployment com Kubernetes

Os recursos Kubernetes estão definidos no Helm chart `dt4mob-historical` em `charts/dt4mob-historical/` na infraestrutura do projeto.

3.8.1 Estrutura do Chart

```
charts/dt4mob-historical/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── _helpers.tpl           # Helpers de nomenclatura de templates
│   ├── writer-deployment.yaml # Deployment para este serviço
│   └── writer-config.yaml     # ConfigMap (variáveis de ambiente para o
writer)
│   ├── kafka-topic.yaml      # Strimzi KafkaTopic (opcional)
│   ├── timescale-cluster.yaml # Cluster CloudNativePG com TimescaleDB
│   ├── timescale-image-catalog.yaml # ImageCatalog CNPG
│   └── api-deployment.yaml    # API de leitura complementar (código sepa
rado)
└── api-service.yaml          # Service da API complementar
```

3.8.2 Writer Deployment (`writer-deployment.yaml`)

Aspetos principais:

- **Imagem:** de `values.images.writer` (padrão: `atnog-harbor.av.it.pt/dt4mob/data-gateway`).
- **Variáveis de ambiente** de duas fontes:
 - **ConfigMap** (`writer-config.yaml`): `LOG_LEVEL`, `KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_SECURITY_PROTOCOL`, `KAFKA_SSL_CA_LOCATION`, `KAFKA_SSL_CERTIFICATE_LOCATION`, `KAFKA_SSL_KEY_LOCATION`, `KAFKA_CONSUMER_GROUP`, `KAFKA_TOPIC`, `KAFKA_AUTO_OFFSET_RESET`, `TIMESCALE_DATABASE_TIMEZONE`.
 - **Referência a secret:** `TIMESCALE_CONNECTION` é lida do secret do cluster CNPG (`{release}-cluster-app`, chave `uri`).
- **Montagens de volume:** Dois volumes montam os secrets SSL do Kafka:
 - `/certs` (certificado + chave do utilizador) — de `writer.kafka.userSecretName`
 - `/cluster-ca` (certificado CA) — de `writer.kafka.trustSecretName`

3.8.3 Writer ConfigMap (`writer-config.yaml`)

Mapeia os campos de `values.yaml` para nomes de variáveis de ambiente, construindo os caminhos dos certificados SSL a partir de pontos de montagem conhecidos:

```
KAFKA_SSL_CA_LOCATION: "/cluster-ca/ca.crt"
KAFKA_SSL_CERTIFICATE_LOCATION: "/certs/user.crt"
KAFKA_SSL_KEY_LOCATION: "/certs/user.key"
KAFKA_TOPIC: {{ .Values.kafkaTopic.name }}
```

3.8.4 TimescaleDB Cluster (`timescale-cluster.yaml`)

Usa o operador **CloudNativePG** (`postgresql.cnpg.io/v1`) para provisionar um cluster PostgreSQL com TimescaleDB:

- **Imagem:** `timescale/timescaledb-ha:pg18.0-ts2.23.0` (via `ImageCatalog`)
- **Biblioteca pré-carregada:** `timescaledb` adicionada a `shared_preload_libraries`
- **SQL pós-inicialização:** `CREATE EXTENSION IF NOT EXISTS timescaledb` — garante que a extensão existe antes do evento `after_create` do writer tentar chamar `create_hypertable()`
- **Armazenamento:** `storageClass` e `size` configuráveis (padrão: `local-path`, `10Gi`)

3.8.5 KafkaTopic (`kafka-topic.yaml`)

Cria opcionalmente um recurso personalizado `KafkaTopic` (Strimzi) definido por `kafkaTopic.name`. Controlado pelo booleano `kafkaTopic.create`.

3.8.6 Como Tudo se Liga

- O Helm cria o cluster TimescaleDB (CNPG) e o `KafkaTopic`.
- O operador CNPG gera um secret com o URI da base de dados.
- O Deployment do writer lê:

- O URI do TimescaleDB a partir do secret CNPG
- As restantes configurações a partir do ConfigMap
- O writer inicia, cria a tabela dittoevent + hipertabela, subscreve o tópico Kafka e começa a consumir.

4 LEVELAMS LOADER

4.1 Visão Geral

O LevelAMS Loader é uma aplicação Python assíncrona que sincroniza dados de monitorização geotécnica entre uma API disponibilizada pela LevelAMS e o Eclipse Ditto (plataforma de digital twins) via Eclipse Hono (broker de mensagens MQTT).

4.2 Architecture Overview

4.2.1 Design do Sistema

Fluxo de dados de alto nível entre componentes.

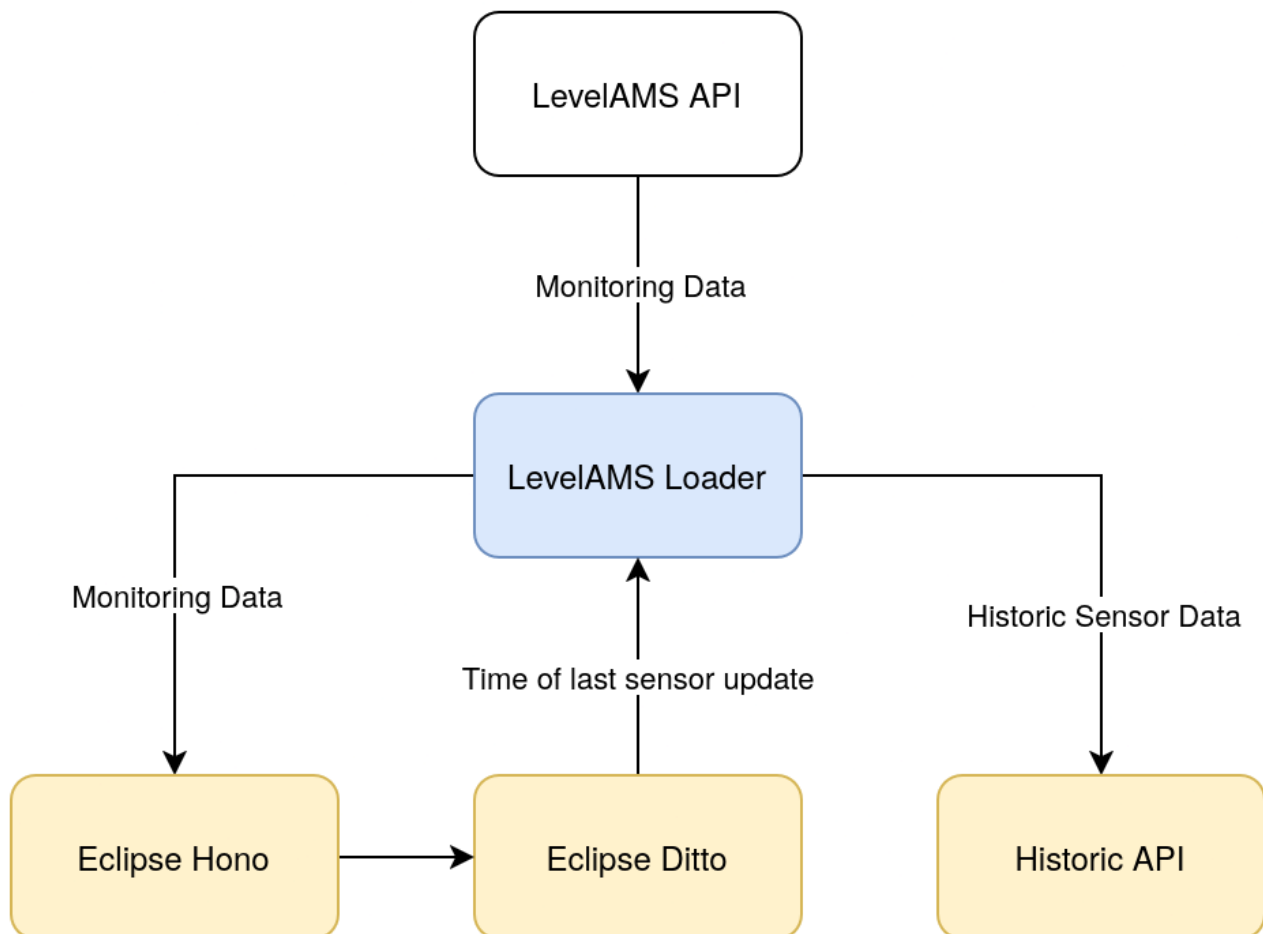


Figura 2: Diagrama de fluxo de dados entre componentes

Interações detalhadas entre componentes.

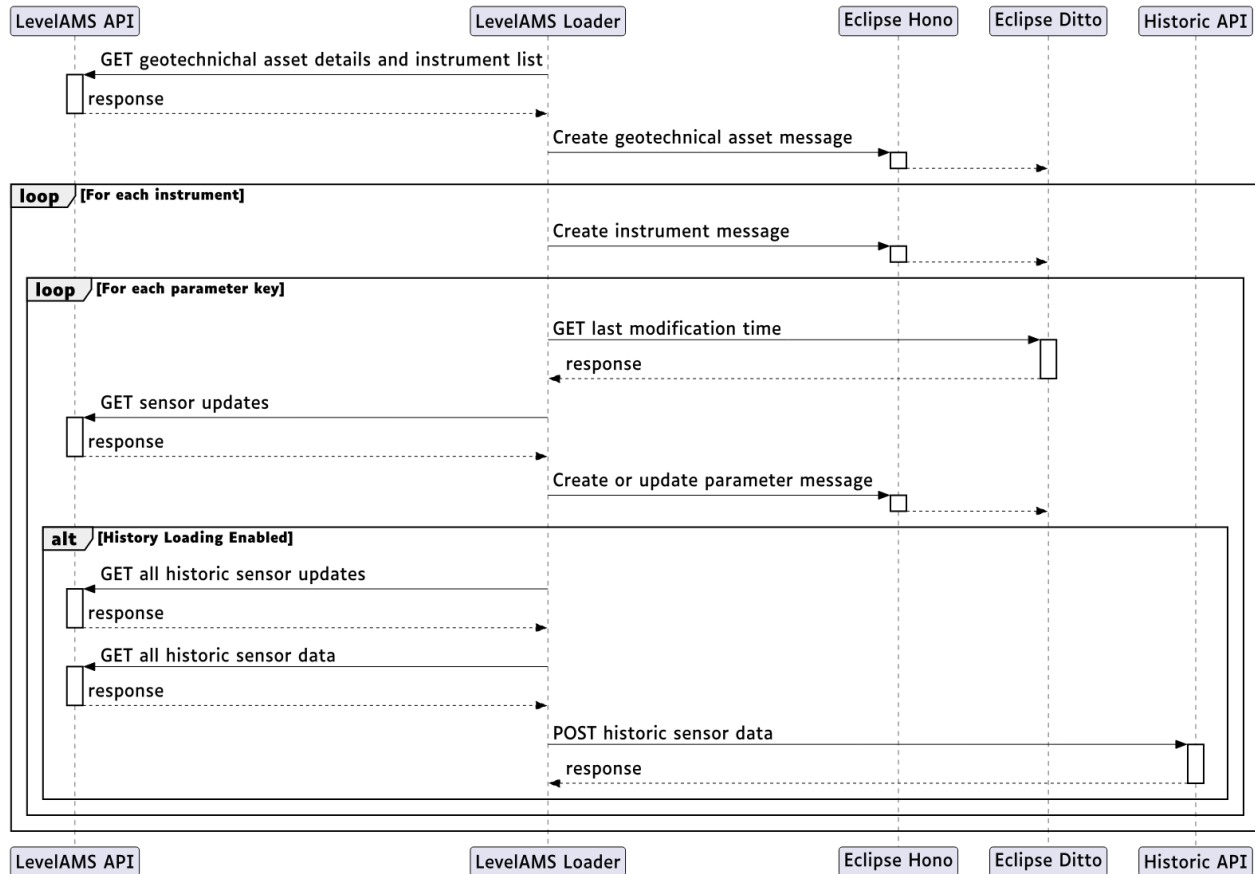


Figura 3: Diagrama de sequência de interações externas do LevelAMS Loader

Data Flow

1. **Obter:** O loader solicita ativos geotécnicos e instrumentos da API LevelAMS
1. **Transformar:** Converte respostas da API em mensagens de protocolo Eclipse Ditto
2. **Publicar:** Envia mensagens para o Eclipse Hono via MQTT
3. **Armazenar:** O Hono roteia mensagens para o Ditto para criação/atualização de Things
4. **Histórico** (opcional): Obtém dados históricos e envia para a API de histórico

Technology Stack

Componente	Tecnologia	Versão	Propósito
Ambiente de execução.	Python	3.13	Ambiente de execução da aplicação.
Gestor de packages	uv	Último	Gestão de dependências.
Cliente HTTP	httpx	==0.28.1	Pedidos HTTP assíncronos.
Cliente MQTT	amqtt	==0.11.3	Suporte protocolo MQTT.

Componente	Tecnologia	Versão	Propósito
<i>Validação de Dados</i>	<i>pydantic</i>	<i>==2.12.5</i>	<i>Validação de modelos.</i>
<i>Definições</i>	<i>pydantic-settings</i>	<i>==2.12.0</i>	<i>Carregamento de configuração TOML/env.</i>
<i>Análise de CLI</i>	<i>pydantic-argparse</i>	<i>==0.10.0</i>	<i>Tratamento de argumentos CLI.</i>
<i>Registo de Logs</i>	<i>loguru</i>	<i>==0.7.3</i>	<i>Registo de logs estruturado.</i>
<i>Coordenadas</i>	<i>pyproj</i>	<i>==3.7.2</i>	<i>ETRS89 TM06 → WGS84</i>
<i>Linting</i>	<i>ruff</i>	<i>==0.12.5</i>	<i>Aplicação de estilo de código.</i>
<i>Type Checking</i>	<i>mypy</i>	<i>==1.19.0</i>	<i>Análise de tipos estática.</i>

Tabela 21 - Visão Geral da Arquitetura

4.3 Local Configuration

1. Pré-requisitos

- Python 3.13+
- Package manager: uv

2. Clonar e instalar.

```
git clone https://github.com/ATNoG/dt4mob-level-ams-loader.git
cd dt4mob-level-ams-loader
uv sync
```

3. Configurar ambiente.

```
cp config.example.toml config.toml
```

Edite o `config.toml` com as definições específicas do seu ambiente. Consulte o ficheiro README para todas as opções.

4. Verificar instalação.

```
#### Verificação de lint
uv run ruff check

#### Verificação de tipos
uv run mypy
```

4.3.1 Project Structure

Ficheiros Principais.

Ficheiro/Diretório	Propósito
main.py	<i>Ponto de entrada. Analisa argumentos CLI, instancia Hono e LevelAMSLoader, executa loader assíncrono.</i>
app/level_ams_loader.py.	<i>Lógica de negócio principal. Orquestra obtenção da API, criação de mensagens Ditto, publicação MQTT.</i>
app/hono_connection.py.	<i>Context manager assíncrono para ciclo de vida de ligação MQTT.</i>
app/settings/__init__.py.	<i>Classe Settings raiz. Carrega config.toml, valida credenciais, configura registo de logs.</i>
app/models/common.py.	<i>Tipos partilhados incluindo Coordinates com conversão ETRS89 TM06 para WGS84.</i>
app/models/	<i>Armazena todos os modelos de dados relevantes.</i>
app/dependencies/	<i>Lógica de ligação para quaisquer dependências de serviços externos.</i>
app/docs/	<i>Ficheiros de documentação.</i>

Tabela 22 - Estrutura do Projeto

4.3.2 Scripts and Commands

Desenvolvimento

```
#### Executar o loader
uv run main.py --geo-asset-id <ASSET_ID>

#### Executar com filtragem de instrumentos
uv run main.py --geo-asset-id <ASSET_ID> --instrument-ids <ID1,ID2>

#### Executar com carregamento de histórico (requer alterações de configuração)
#### 1. Definir history.enabled = true em config.toml
#### 2. Configurar secção [oidc]
uv run main.py --geo-asset-id <ASSET_ID>
```

Qualidade de Código

```
#### Verificação de lint
uv run ruff check

#### Verificação de lint com correção automática
uv run ruff check --fix

#### Verificação de tipos (modo estrito)
uv run mypy
```

Docker

```
#### Construir imagem
docker build -t level-ams-loader .

#### Run container
docker run level-ams-loader --geo-asset-id <ASSET_ID>

#### Run com variáveis de ambiente e substituições de ficheiros externos
docker run \
  -e LEVEL_AMS_LOADER_LOG_LEVEL=DEBUG \ # Pode fornecer outras variáveis de f
orma semelhante
  -v ./config.toml:/app/config.toml:ro \ # Pode fornecer outros ficheiros de
forma semelhante
  level-ams-loader \
  --geo-asset-id <ASSET_ID>
```

4.3.3 Deployment

Build Docker

O Dockerfile constrói uma imagem de estágio único:

2. **Imagem base:** python:3.13-slim
5. **Package manager:** Copia o uv da imagem oficial
6. **Dependências:** Copia pyproject.toml, uv.lock, .python-version primeiro para cache, depois executa `uv sync --frozen --no-cache`
7. **Aplicação:** Copia código fonte
8. **Entry point:** `uv run main.py`

4.3.4 Environment Variables

Todas as definições podem ser substituídas por variáveis de ambiente com o prefixo `LEVEL_AMS_LOADER_` e `__` como delimitador aninhado.

<i>Variável de Ambiente.</i>	<i>Caminho Config</i>	<i>Exemplo</i>
<code>LEVEL_AMS_LOADER_LOG_LEVEL.</code>	<code>log_level</code>	<code>DEBUG</code>
<code>LEVEL_AMS_LOADER_LOADER__BASE_URL.</code>	<code>loader.base_url</code>	<code>https://api.example.com.</code>
<code>LEVEL_AMS_LOADER_LOADER__JWT.</code>	<code>loader.jwt</code>	<code>eyJhbGciOiJIUzI1NiIs...</code>
<code>LEVEL_AMS_LOADER_HONO__HOST.</code>	<code>hono.host</code>	<code>mqtt.example.com</code>
<code>LEVEL_AMS_LOADER_HONO__PORT.</code>	<code>hono.port</code>	<code>8883</code>
<code>LEVEL_AMS_LOADER_HONO__DEVICE.</code>	<code>hono.device</code>	<code>my-device</code>

Variável de Ambiente.	Caminho Config	Exemplo
LEVEL_AMS_LOADER_HONO__TENANT.	hono.tenant	my-tenant
LEVEL_AMS_LOADER_HONO__PASSWORD.	hono.password	secret
LEVEL_AMS_LOADER_DITTO__BASE_URL.	ditto.base_url	https://ditto.example.com/api/2.
LEVEL_AMS_LOADER_DITTO__NAMESPACE.	ditto.namespace	org.example
LEVEL_AMS_LOADER_HISTORY__ENABLED.	history.enabled	true
LEVEL_AMS_LOADER_OIDC__USERNAME.	oidc.username	admin

Tabela 23 - Variável de Ambiente para configuração do LevelAMS Loader

4.3.5 Code Conventions

Type Hints

O projeto utiliza verificação mypy estrita. Todas as funções e métodos devem ter anotações de tipo completas.

Pydantic Models

- Utilizar modelos pydantic v2 para validação de dados
- Utilizar pydantic-settings para carregamento de configuração
- Utilizar pydantic.v1 apenas para compatibilidade com pydantic-argparse

Async/Await

- Utilizar asyncio.TaskGroup para operações concorrentes
- Utilizar httpx.AsyncClient para pedidos HTTP
- Utilizar context managers para gestão de recursos

Registo de Logs

- Utilizar loguru para registo de logs estruturado
- Níveis de log: trace para dados originais, debug para fluxo, info para progresso, success para conclusão, error para falhas

Tratamento de Erros

- Levantar RuntimeError para erros irrecuperáveis
- Utilizar logger.error() antes de levantar para visibilidade

- Deixar exceções propagar através de TaskGroup para limpeza adequada

5 GANTRY SERVICE

5.1 Visão Geral

O Gantry Service é uma aplicação Python assíncrona que faz a ponte entre dados de sensores rodoviários e digital twins do Eclipse Ditto através do Eclipse Hono. Segue uma arquitetura driver/interface em camadas com componentes intercambiáveis que permitem uma integração flexível com diferentes fontes de dados e destinos.

5.2 Architecture Overview

Fluxo de Dados de Alto Nível.

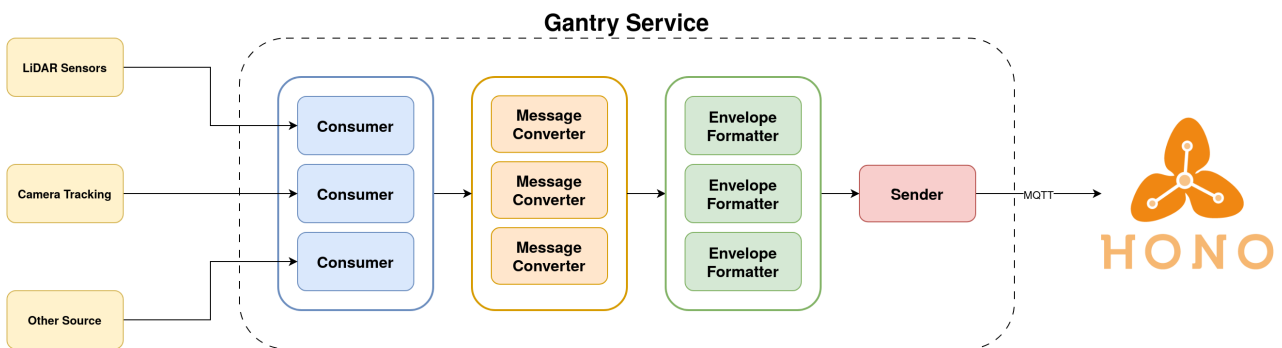


Figura 4: Diagrama de arquitetura de alto nível do Gantry Service

Interações entre Componentes.

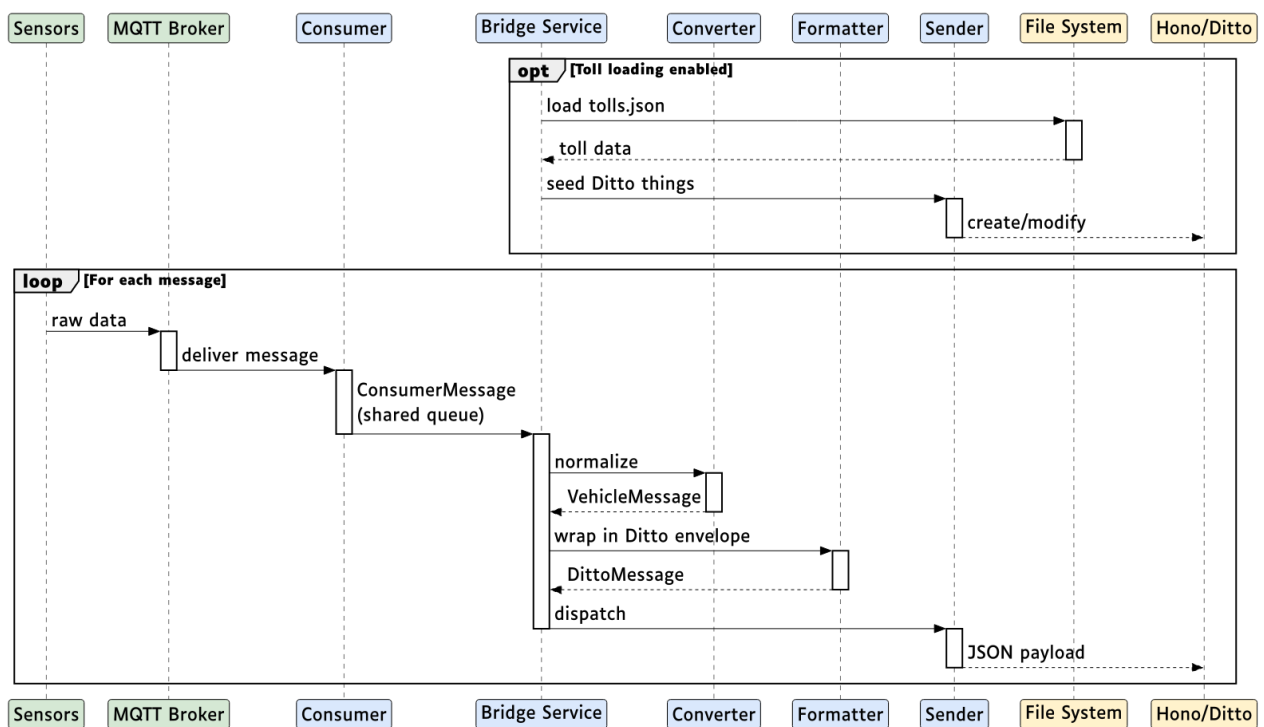


Figura 5: Diagrama de sequência de interações entre componentes do Gantry Service

Technology Stack

Camada	Tecnologia
Linguagem	Python 3.13
Gestor de packages	uv (Astral)
Ambiente de execução assíncrono.	asyncio
MQTT	amqtt (cliente MQTT assíncrono)
Servidor webhook	FastAPI + uvicorn
Cliente HTTP	httpx
Validação de dados	pydantic v2 + pydantic-settings.
Registo de logs	loguru
Type check	mypy (modo estrito com plugin pydantic)
Linting	ruff (com detecção T20 print)
Containerização	Docker (python:3.13-slim) + Docker Compose.

Tabela 24 - Visão Geral da Arquitetura

Padrões de Design

- **Separação Interface/Driver:** Todos os componentes definem classes base abstratas em `app/interface/`, com implementações concretas em `app/drivers/`
- **Unões discriminadas:** As definições utilizam Unões discriminadas Pydantic para configurações multi-backend (ex., `ConsumerSettings` pode ser `MQTTConsumerSettings` ou `WebhookConsumerSettings`)
- **Padrão Factory:** Cada módulo driver expõe uma função `factory` (`get_consumers()`, `get_sender_interface()`, etc.) que lê as definições e instancia os drivers corretos
- **Singleton store:** `MessageConverterStore` mantém instâncias de conversores ativos
- **Fila assíncrona partilhada:** Todos os consumers enviam para uma única `asyncio.Queue[ConsumerMessage]` consumida pelo bridge service

Instalação e Ligação.

Toda a criação de componentes acontece no **momento de importação do módulo**. O singleton `Settings` é construído uma vez, depois cada driver lê-o e instancia o backend correto. `BridgeService` é o componente principal que simplesmente chama as funções `factory` e orquestra o ciclo de vida assíncrono.

Singleton de Definições.

app/settings/___init__.py cria uma instância `settings = Settings()` a nível de módulo. Lê `config.toml` e aplica quaisquer substituições de variáveis de ambiente com o prefixo `GS_`. Cada módulo driver importa esta instância partilhada.

Instanciação do Driver (no momento de importação)

Cada `app/drivers/<component>/___init__.py` executa um `match` no discriminador das definições e importa + instancia condicionalmente o backend correto. Alguns drivers incluem também uma **cláusula de guarda** que levanta `RuntimeError` se o módulo for importado quando o backend não corresponde — serve de rede de segurança contra importações diretas acidentais.

Componente	Função factory	Fonte de verdade	Backends
Consumers	<code>get_consumers()</code>	<code>settings.consumers[]</code> . backend.	mqtt, webhook
Sender	<code>get_sender_interface()</code>	<code>settings.sender.backend</code> .	disabled, mqtt, http.
Conversores de Mensagem	<code>get_message_converters()</code>	<code>settings.message_converters</code> .	a-to-be, oversight
Formatadores de Envelope.	<code>get_envelope_formatters()</code>	<code>settings.envelope_formatters</code> .	toll-feature, ditto-Thing
Carregador de Portagens.	<code>get_toll_loader()</code>	<code>settings.toll_loader</code> . backend.	disabled, json-file

Tabela 25 – Diferentes componentes do LevelAMS Loader

Padrão da cláusula de guarda (usado por drivers de sender e toll loader):

```
#### app/drivers/sender/mqtt/___init__.py
if settings.sender.backend != SenderBackend.MQTT:
    raise RuntimeError("MQTT Sender driver instantiated but backend isn't MQTT")

mqtt_sender_interface = MQTTSenderInterface(mqtt_settings=settings.sender)
```

Isto garante que um módulo driver só pode ser instanciado quando o backend correspondente está ativo. O `___init__.py` pai só acede a este código através do ramo `match`, pelo que o guarda é uma medida defensiva contra uso indevido.

Ciclo de processamento principal (dentro do método `run()`):

3. Lê `ConsumerMessage` da fila partilhada
9. Despacha para o `MessageConverterInterface` correto com base no `expected_message_type`
10. Para cada `EnvelopeFormatterBackend` em `allowed_ditto_formats`, chama `format()` → adiciona a `DittoMessage` resultante na fila de envio

11. `_sender_loop` apanha mensagens e chama `sender.send()`

Adicionar um Novo Backend.

Para adicionar uma nova implementação para qualquer componente:

4. **Interface:** definir uma classe base abstrata em `app/interface/<component>.py` com os métodos necessários
12. **Definições:** adicionar um novo valor enum ao enum backend relevante em `app/settings/<component>.py` e criar um modelo de definições Pydantic para os campos do novo backend
13. **Driver:** criar `app/drivers/<component>/<backend>/` com a implementação concreta, aceitando as definições tipadas em `__init__`
14. **Ligação:** adicionar um caso match em `app/drivers/<component>/__init__.py` que importa e instancia o driver quando o backend é selecionado; adicionar uma cláusula guarda seguindo o padrão existente
15. **Factory:** a função `get_*` existente irá retornar o novo driver automaticamente — não são necessárias alterações aí a menos que o tipo de retorno mude

5.3 Configuração Local

Clonar o repositório.

```
git clone https://github.com/ATNoG/dt4mob-gantry-service.git
cd dt4mob-gantry-service
```

Instalar uv (se não estiver instalado)

Siga as instruções em <https://docs.astral.sh/uv/getting-started/installation/>.

Instalar dependências.

```
uv sync
```

Isto instala dependências de runtime e desenvolvimento (mypy, ruff) num ambiente virtual.

Configurar o serviço.

```
cp config.example.toml config.toml
```

Edite `config.toml` com as suas definições locais. No mínimo, configure:

- Definições de identidade [ditto]
- Pelo menos um bloco `[[consumers]]`
- Entradas correspondentes `[[consumers.message_settings]]`
- Backend `[sender]` apontando para a sua instância Hono/MQTT
- `[message_converters]` e `[envelope_formatters]` para o seu pipeline

Configurar dados de portagens (opcional)

```
cp tolls.example.json tolls.json
```

Edite `tolls.json` com as suas definições de portagens.

Run


```
uv run main.py
```

Variáveis de Ambiente.

As definições podem ser substituídas por variáveis de ambiente usando o prefixo `GS_` com `__` como delimitador aninhado:

```
export GS_LOG_LEVEL="DEBUG"
export GS_SENDER__HOST="10.0.0.1"
export GS_SENDER__MQTT__PORT=1884
export GS_DITTO__NAMESPACE="gantries"
```

A ordem de prioridade é: variáveis de ambiente → valores em `config.toml`.

5.3.1 Project Structure

Caminho	Descrição
<code>main.py</code>	<i>Ponto de entrada da aplicação — cria e executa o BridgeService.</i>
<code>app/interface/</code>	<i>Classes base abstratas que definem a interface de programação para os componentes intercambiáveis do bridge service (consumers, senders, conversores, formataadores, carregadores de portagens)</i>
<code>app/drivers/</code>	<i>Implementações concretas de driver para cada interface (consumers MQTT/webhook, senders MQTT/HTTP, etc.)</i>
<code>app/schema/</code>	<i>Modelos de dados Pydantic para envelopes de protocolo Ditto, mensagens de veículos, definições de portagens, e tipos comuns.</i>
<code>app/settings/</code>	<i>Modelos de configuração pydantic-settings que mapeiam secções de <code>config.toml</code> para objetos Python tipados.</i>
<code>app/bridge_service.py.</code>	<i>Orquestração principal — liga consumers, conversores, formataadores, e o sender num pipeline de processamento assíncrono.</i>
<code>config.example.toml</code>	<i>Configuração de exemplo documentada com todas as opções disponíveis.</i>
<code>tolls.example.json</code>	<i>Ficheiro de exemplo de definições de portagens mostrando o esquema JSON esperado.</i>
<code>compose.yml</code>	<i>Definição de serviço Docker Compose para deploy contentorizado.</i>
<code>Dockerfile</code>	<i>Instruções de build de imagem de contentor (Python 3.13-</i>

Caminho	Descrição
	<i>slim + uv)</i>
<code>pyproject.toml</code>	<i>Metadados do projeto, dependências, e configuração de ferramentas (mypy, ruff)</i>
<code>docs/</code>	<i>Fontes de documentação (manual do utilizador, manual do programador, scripts de build)</i>

Tabela 26 - Estrutura do Projeto

5.4 Scripts e Comandos

Não existe Makefile ou task runner. Todos os comandos utilizam uv diretamente:

Desenvolvimento

```
#### Instalar/sincronizar dependências
uv sync

#### Run o serviço
uv run main.py
```

Type Checking

```
uv run mypy
```

Executa mypy em modo estrito com o plugin pydantic. A configuração está em `pyproject.toml` em `[tool.mypy]`.

Linting

```
uv run ruff check
```

Executa ruff com detecção de declarações print (regra T20). A configuração está em `pyproject.toml` em `[tool.ruff]`.

Docker

```
#### Construir e executar
docker compose up --build

#### Run em segundo plano
docker compose up --build -d

#### Parar
docker compose down
```

5.4.1 Key Implementation Details

Drivers de Consumer

- **Consumer MQTT** (`app/drivers/consumer/mqtt/consumer.py`): Utiliza `amqtt.client.MQTTClient` com `ClientConfig` e `ConnectionConfig`. Subscrive a todos

os tópicos configurados com QOS 0. A correspondência tópico MQTT → `message_settings` é feita comparando a string do tópico MQTT.

- **Consumer Webhook** (`app/drivers/consumer/webhook/consumer.py`): Cria uma aplicação FastAPI com um endpoint POST catch-all `/{{topic:path}}`. Executa via `uvicorn.Server`. Apenas processa tópicos presentes nos `message_settings` configurados.

Conversores de Mensagem.

- **AToBe** (`app/drivers/message_converter/a_to_be/message_converter.py`): Trata sub-tipos `realtime` (→ `VehicleDataMessage`) e `history` (→ `VehicleDeleteMessage`). Valida mensagens recebidas contra esquemas Pydantic.
- **OutSight** (`app/drivers/message_converter/out_sight/message_converter.py`): Utiliza um set para rastrear IDs de mensagens vistas. Trata entregas fora de ordem: mensagens vistas pela primeira vez são adicionadas à fila; mensagens recebidas novamente com `out_alert` produzem o `VehicleDataMessage` final.

Formatadores de Envelope.

- **Toll Feature** (`app/drivers/envelope_formatter/toll_feature/envelope_formatter.py`): Único Ditto Thing por portagem com features por sensor. Utiliza um buffer circular (tamanho 50) para IDs de veículo; entradas mais antigas recebem comandos `delete`. `delete_all()` limpa todas as features de origem
- **Ditto Thing** (`app/drivers/envelope_formatter/ditto_Thing/envelope_formatter.py`): Ditto Thing separado por veículo. Utiliza `deferred_creation_offset` para criar “future Things” que expiram automaticamente. Gera comandos `create`, `modify`, e de expiração,

Drivers de Sender

- **Sender MQTT** (`app/drivers/sender/mqtt/sender.py`): Publica para o Eclipse Hono via `amqtt`. Suporta TLS com certificado CA. Utiliza `reconnect_retries=-1` para reconexão infinita.
- **Sender HTTP** (`app/drivers/sender/http/sender.py`): Envia JSON via `httpx.AsyncClient`. Suporta auth básica e mTLS com certificados de cliente.

5.5 Deployment

Build Docker

O Dockerfile utiliza uma abordagem multi-step:

5. Imagem base: `python:3.13-slim`
16. Copia `uv` da imagem oficial Astral
17. Copia `pyproject.toml`, `uv.lock`, `.python-version` primeiro (para cache de camadas)
18. Executa `uv sync --frozen --no-cache` para instalar dependências
19. Copia código da aplicação
20. Comando de entrada: `uv run main.py`

Deploy em Produção

```
#### Construir a imagem
docker compose build

#### Deploy (com modo destacado)
docker compose up -d

#### Ver logs
docker compose logs -f gantry-service

#### Parar
docker compose down
```

Assegure-se de que todos os ficheiros estão disponíveis em tempo de execução.

6 SENSOR DATA CONVERSION SERVICE

6.1 Visão Geral

O Sensor Data Conversion Service é uma aplicação Python assíncrona que converte mensagens de sensores no formato OutSight para o formato ORT Tracking. Utiliza uma arquitetura driver/interface com backends de consumer intercambiáveis e um pipeline de conversão modular para transformação de coordenadas e mensagens.

6.2 Architecture Overview

Fluxo de Dados de Alto Nível.

6. **Consumer** recebe mensagens JSON no formato da OutSight através de uma subscrição MQTT ou POST requests HTTP para o webhook
21. **Conversor** (OutSightToAToBe) executa transformações de coordenadas usando intermediários ENU (East-North-Up) e operações matriciais numpy, derivando dimensões das caixas delimitadoras, e produz mensagens no formato ORT Tracking
22. **Sender** publica mensagens convertidas num broker MQTT em tópicos configuráveis

Diagrama de sequência que descreve o fluxo.

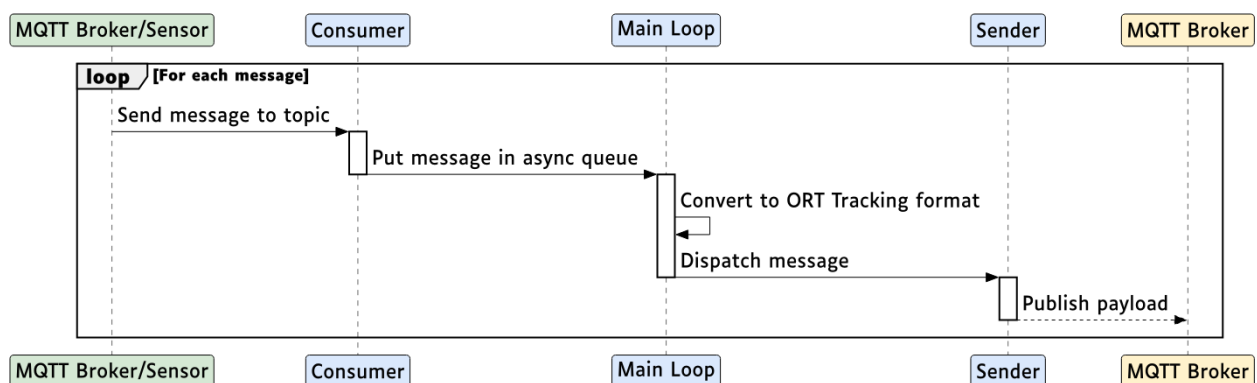


Figura 6: Diagrama de sequência do pipeline de dados do Serviço de Conversão de Dados de Sensor

Technology Stack

Componente	Tecnologia	Versão	Propósito
<i>Ambiente de execução.</i>	<i>Python</i>	<i>3.13</i>	<i>Ambiente de execução da aplicação.</i>
<i>Gestor de packages</i>	<i>uv</i>	<i>latest</i>	<i>Gestão de dependências.</i>
<i>Ambiente de Execução Assíncrono.</i>	<i>asyncio</i>	<i>stdlib</i>	<i>Concorrência</i>
<i>Cliente MQTT</i>	<i>amqtt</i>	<i>==0.11.3</i>	<i>Publicação/subscrição MQTT assíncrona.</i>
<i>Servidor Webhook</i>	<i>FastAPI + uvicorn</i>	<i>==0.129.0</i>	<i>Endpoint HTTP POST para consumer webhook.</i>
<i>Validação de Dados</i>	<i>pydantic</i>	<i>==2.12.3</i>	<i>Análise de mensagens e modelos de definições.</i>
<i>Definições</i>	<i>pydantic-settings</i>	<i>==2.11.0</i>	<i>Carregamento de configuração TOML/env.</i>
<i>Registo de Logs</i>	<i>loguru</i>	<i>==0.7.3</i>	<i>Registo de logs estruturado.</i>
<i>Álgebra Linear</i>	<i>numpy</i>	<i>==2.3.4</i>	<i>Operações matriciais para transformações de coordenadas.</i>
<i>Transformações Geográficas.</i>	<i>pymap3d</i>	<i>==3.2.0 g</i>	<i>Conversão de coordenadas eodésicas ↔ ENU.</i>
<i>Linting</i>	<i>ruff</i>	<i>==0.14.14</i>	<i>Aplicação de estilo de código.</i>
<i>Type Checking</i>	<i>mypy</i>	<i>==1.18.2</i>	<i>Análise de tipos estática (modo estrito)</i>

Tabela 27 - Tecnologias utilizadas no LevelAMS Loader

Tabela 26: Componente — Visão Geral da Arquitetura

Padrões de Desenho

- **Separação Interface/Driver:** O consumer é definido através de uma ConsumerInterface abstrata em app/interfaces/consumer.py, com implementações concretas em app/drivers/consumer/
- **União discriminadas:** ConsumerSettings utiliza uma união discriminada Pydantic no campo backend para selecionar entre MQTTConsumerSettings, WebhookConsumerSettings, ou DisabledConsumerSettings
- **Padrão Factory:** app/drivers/consumer/__init__.py expõe uma função factory

`get_consumer()` que lê as definições e instancia o driver correto

- **Cláusulas guarda:** Cada módulo driver inclui uma guarda que levanta `RuntimeError` se for importado quando o backend correspondente não está ativo
- **Ciclo de vida do context manager:** Tanto o consumer como o sender expõem context managers assíncronos para uma gestão de conexões limpa

6.3 Instalação e Ligação

Toda a criação de componentes acontece no **momento em que um módulo é importado**:

1. **Singleton de definições:** `app/settings/__init__.py` cria uma instância `settings = Settings()` a nível do módulo. Lê o ficheiro `config.toml` e aplica quaisquer substituições de variáveis de ambiente com o prefixo `SENSOR_DATA_CONVERSION_`.
23. **Instanciação do driver:** `app/drivers/consumer/__init__.py` executa um match em `settings.consumer.backend` e importa + instancia condicionalmente o backend correto.
24. **Conexão ao serviço:** `app/service.py` cria o consumer, sender, e conversor, depois orquestra o ciclo de processamento assíncrono.

6.4 Local Configuration

Clonar o repositório.

```
git clone https://github.com/ATNoG/dt4mob-sensor-data-conversion.git
cd dt4mob-sensor-data-conversion
```

Instalar uv (se não estiver instalado)

Siga as instruções em <https://docs.astral.sh/uv/getting-started/installation/> para instalar uv.

Instalar dependências.

```
uv sync
```

Isto instala dependências de runtime e desenvolvimento (mypy, ruff) num ambiente virtual.

Configurar o serviço.

```
cp config.example.toml config.toml
```

Edite `config.toml` com as suas definições locais. No mínimo, configure:

- `atb_realtime_topic` e `atb_history_topic` para tópicos de saída
- Backend [consumer] com detalhes de conexão
- URL [sender] apontando para o seu broker MQTT
- Pontos de referência [converter] para as coordenadas do seu deployment

Run

```
uv run main.py
```

Variáveis de Ambiente.

As definições podem ser substituídas por variáveis de ambiente usando o prefixo `SENSOR_DATA_CONVERSION_` e com `__` como delimitador aninhado:

```
export SENSOR_DATA_CONVERSION_LOG_LEVEL="DEBUG"
export SENSOR_DATA_CONVERSION_SENDER__URL="mqtt://broker:1883"
export SENSOR_DATA_CONVERSION_CONSUMER__BACKEND="mqtt"
export SENSOR_DATA_CONVERSION_CONSUMER__URL="mqtt://sensor-broker:1883"
export SENSOR_DATA_CONVERSION_ATB_REALTIME_TOPIC="vehicle/realtime"
```

A ordem de prioridade é: variáveis de ambiente → valores no ficheiro config.toml.

6.5 Estrutura do Projeto

Ficheiros Principais.

Caminho	Descrição
main.py	Ponto de entrada da aplicação – cria e executa o Service.
app/service.py	Orquestração principal – liga consumer, conversor, e sender num pipeline de processamento assíncrono.
app/sender.py	Sender MQTT – publica mensagens convertidas via amqtt.
app/interfaces/	Classes base abstratas que definem a interface de programação para componentes.
app/drivers/	Implementações concretas de driver para cada interface.
app/lib/	Lógica de conversão principal.
app/lib/convert.py	Classe OutSightToAToBe – transformações de coordenadas, cálculos de dimensões, conversão de mensagens.
app/models/	Modelos de dados Pydantic para mensagens de input/output e tipos partilhados.
app/settings/	Modelos de configuração pydantic-settings.
config.example.toml	Configuração de exemplo documentada com todas as opções disponíveis.
docs/	Fontes de documentação (manual do utilizador, manual do programador)

Tabela 28 - Estrutura do Projeto

6.6 Scripts e Comandos

Não existe Makefile ou task runner. Todos os comandos utilizam uv diretamente:

Desenvolvimento

```
#### Instalar/sincronizar dependências
uv sync
```

```
#### Run o serviço
uv run main.py
```

Type Checking

```
uv run mypy
```

Executa mypy em modo estrito com o plugin pydantic. A configuração está em `pyproject.toml` em `[tool.mypy]`.

Linting

```
uv run ruff check
```

Executa ruff com detecção de declarações print (regra T20). A configuração está em `pyproject.toml` em `[tool.ruff]`.

Docker

```
#### Construir e executar
docker compose up --build
```

```
#### Run em segundo plano
docker compose up --build -d
```

```
#### Parar
docker compose down
```

6.7 Detalhes Chave de Implementação

Consumer Drivers

- **Consumer MQTT** (`app/drivers/consumer/mqtt/consumer.py`): Utiliza `amqtt.client.MQTTClient` com `ClientConfig` e `ConnectionConfig`. Subscrive a todos os tópicos configurados com QOS 0. Expõe um iterador assíncrono que produz bytes de mensagens originais.
- **Consumer Webhook** (`app/drivers/consumer/webhook/consumer.py`): Cria uma aplicação FastAPI com um endpoint POST catch-all no `base_path` configurado. Executa via `uvicorn.Server` como uma tarefa `asyncio`. Os request bodies recebidos são colocados numa `asyncio.Queue[bytes]`.
- **Factory** (`app/drivers/consumer/__init__.py`): Utiliza uma declaração `match` em `settings.consumer.backend` para importar e instalar condicionalmente o driver correto. Levanta `RuntimeError` se nenhum backend válido estiver configurado.

Conversor (`app/lib/convert.py`)

A classe `OutSightToAToBe` executa todas as transformações de coordenadas e conversão de mensagens.

Inicialização (executa uma vez no arranque):

1. Converte a origem do plano de referência `OutSight` e o ponto do eixo horizontal de geodésicas para coordenadas ENU relativas à origem do plano de referência `AToBe` usando

`pymap3d.geodetic2enu`

25. Calcula a matriz de transformação OutSight-to-ENU a partir dos vetores unitários horizontais e verticais OutSight
26. Calcula a matriz de transformação ENU-to-AToBe a partir dos vetores unitários horizontais e verticais AToBe

Conversão por mensagem:

- `calculate_coords(data: Alert)` – Transforma o canto [3] da caixa delimitadora OutSight (o ponto de referência) através da matriz OutSight-to-ENU, depois calcula coordenadas absolutas através do `pymap3d.enu2geodetic`, depois transforma através da matriz ENU-to-AToBe para coordenadas locais. Nota: o x e y são intencionalmente invertidos na saída final devido à especificação do ORT Tracking.
- `calculate_dimensions(data: Alert)` – Calcula o comprimento e a largura fazendo a média das distâncias entre lados opostos da caixa delimitadora usando `numpy.linalg.norm`. A altura é retirada diretamente da mensagem OutSight.
- `convert_outsight_to_ort_tracking_realtime_message()` – Produz uma única `ORTTrackingRealtimeMessage` a partir dos dados `out_alert` (preferido) ou `in_alert`.
- `convert_outsight_to_ort_tracking_history_message()` – Produz uma `ORTTrackingHistoryMessage` contendo pontos de detecção in e out como itens de caminho. Returns None se não houver `out_alert` ou `end_timestamp`.

Sender (app/sender.py)

- Utiliza `amqtt.client.MQTTClient` com `reconnect_retries=-1` para reconexão infinita
- Expõe um context manager assíncrono `client` que liga ao entrar e desliga ao sair
- `send(topic, payload)` publica bytes originais no tópico MQTT especificado

Orquestração do Serviço (app/service.py)

A classe `Service` junta tudo:

1. Cria consumer, sender, e conversor a partir das definições
27. Abre ambas as ligações consumer e sender via context managers assíncronos
28. Itera sobre mensagens do consumer, converte cada uma, e publica os resultados
29. Trata `ValidationError` do `pydantic` (registra aviso e ignora mensagens inválidas)
30. Trata encerramento gracioso em `KeyboardInterrupt` / `CancelledError`

6.8 Deployment

Build Docker

O Dockerfile utiliza uma build de estágio único:

1. **Imagem base:** `python:3.13-slim`
31. **Package manager:** Copia o `uv` da imagem oficial Astral (`ghcr.io/astral-sh/uv:latest`)
32. **Dependências:** Copia `pyproject.toml`, `uv.lock`, `.python-version` primeiro para cache de camadas, depois executa `uv sync --frozen --no-cache`

33. **Aplicação:** Copia código fonte

34. **Comando de entrada:** `uv run main.py`

Deploy em Produção

```
#### Construir a imagem
docker compose build

#### Deploy (com modo destacado)
docker compose up -d

#### Ver logs
docker compose logs -f conversion-service

#### Parar
docker compose down
```

Configuração do Docker Compose.

O `compose.yml` monta o `config.toml` como um volume read-only e expõe a porta 8000 para o consumer webhook. O hostname `host.docker.internal` é mapeado para o gateway do anfitrião, permitindo ao serviço aceder a um broker MQTT a funcionar na máquina anfitriã.

Para deploys em produção, assegure-se de que:

- `config.toml` está configurado com os URLs corretos do broker e coordenadas dos planos de referência
- O broker MQTT é acessível em rede a partir do contentor
- A porta 8000 é exposta apenas se o consumer webhook estiver em uso # Data Visualization Platform

6.9 Visão Geral

A Data Visualization Platform é uma aplicação C++ construída com Unreal Engine 5.6 que disponibiliza a camada de visualização 3D interativa para o digital twin do DT4MOB. Utiliza o plugin Cesium for Unreal para renderização geoespacial precisa ao nível do globo. Recebe estado vivo de entidades do Eclipse Ditto via WebSocket e apresenta-o num ambiente 3D geoespacialmente preciso e continuamente atualizado.

A plataforma foi desenvolvida inteiramente em C++, em vez do sistema de scripting visual do Unreal Engine (Blueprints), exceto para o layout de widgets e animações, que são definidos em Blueprints mas suportados por classes de lógica em C++. Esta separação permite que a autenticação de serviços backend complexos, a gestão de WebSocket, a análise de JSON e a matemática geográfica sejam implementadas de forma rigorosa, mantendo a flexibilidade de design visual do Unreal Motion Graphics (UMG).

6.10 Arquitetura do Sistema

A plataforma de visualização segue uma arquitetura em camadas. As responsabilidades são distribuídas por cinco camadas que comunicam através de delegados e eventos, mantendo as camadas desacopladas.

Service Layer — Singletons `UGameInstanceSubsystem` que são automaticamente criados e destruídos pelo motor com a instância de jogo. Todos os serviços partilham o mesmo ciclo de vida e são globalmente acessíveis a partir de qualquer objeto do jogo. A camada de serviço trata da comunicação externa (HTTP, WebSocket), conversão de coordenadas geográficas,

descarregamento de modelos GLB, registo de atores e instanciação de entidades.

Entity Layer — Instâncias individuais de ATempUIActor que existem no mundo do Unreal. Cada ator representa exatamente um Ditto Thing e é responsável por manter uma cópia atualizada do JSON desse Thing, redesserializá-lo numa struct C++ tipada, posicionar-se no globo e refletir o seu estado visual (sobreposição, seleção, malhas personalizadas).

Gamemode Layer — O ADT4MOBGamemode coordena o arranque: inicia a obtenção REST inicial de todos os Things conhecidos, aciona obtenções baseadas em tiles à medida que a câmara se move e trata da instanciação sob demanda quando chegam eventos WebSocket para Things que ainda não estão no mundo.

Manager Layer — Singletons ULocalPlayerSubsystem com âmbito no jogador humano, que gerem o estado de interação (seleção, sobreposição), coordenação de visibilidade da UI, posicionamento de entidades e o registo de atores.

User Interface Layer — Hierarquia de widgets UMG renderizada no ecrã, disponibilizando janelas de inspeção de entidades, um painel de outline da cena, uma barra de ferramentas com ferramentas de posicionamento e um subsistema de temas.

O fluxo de informação através destas camadas segue um percurso fixo: os dados externos entram pela camada de serviço, são traduzidos em atores de entidade pelo gamemode e pela factory, e são refletidos na interface do utilizador através de delegados dos managers. Nenhuma camada endereça diretamente uma camada da qual não depende.

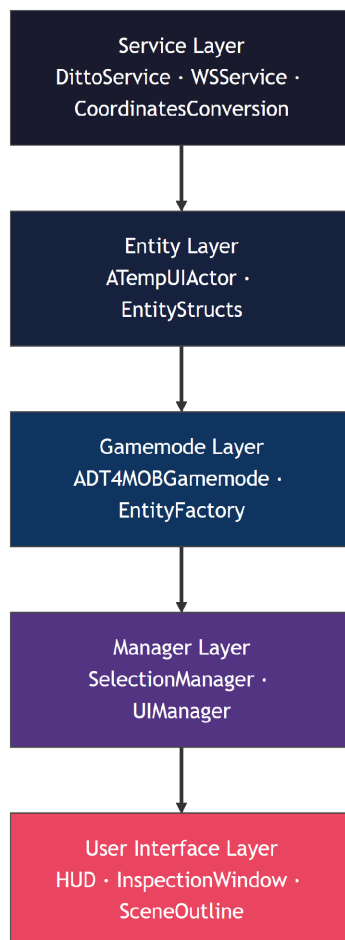


Figura 7: Arquitetura em camadas da plataforma de visualização do DT4MOB, mostrando as cinco camadas e a direção do fluxo de informação desde o Eclipse Ditto até aos widgets UMG

6.11 Autenticação e Configuração

O utilizador insere todas as credenciais e parâmetros de ligação através de um ecrã de login apresentado no arranque da aplicação. O ecrã de login recolhe o hostname do Ditto, o nome de utilizador, a palavra-passe e a seleção do modo de autenticação, e escreve-os nas variáveis de configuração em tempo de execução consumidas pelo UDittoService antes de qualquer pedido de rede ser emitido. Uma flag de configuração seleciona a autenticação OAuth2 ou HTTP Basic, juntamente com definições opcionais de HTTPS/WSS e a mensagem de início do WebSocket.

O subsistema UDittoService lê esta configuração durante o Initialize(). Quando o OAuth2 é selecionado, emite imediatamente um pedido POST para o endpoint de token do Keycloak utilizando um password grant. A resposta contém um access_token, um refresh_token e um valor expires_in. O serviço armazena os tokens e agenda uma renovação proativa de token 30 segundos antes de o access token expirar, utilizando o FTimerHandle do Unreal. Se não estiver disponível nenhum refresh token, recai sobre uma reautenticação completa com palavra-passe.

Quaisquer chamadas à API HTTP que cheguem antes de o primeiro token ser obtido são armazenadas numa fila PendingRequests. Assim que o token chega, a fila é esvaziada atómicamente. Isto garante que nenhuma chamada à API é silenciosamente descartada durante a janela de arranque antes de a autenticação ser concluída.

O delegado OnAuthHeaderReady é emitido quando um cabeçalho Authorization válido fica disponível pela primeira vez e novamente em cada renovação subsequente de token. O UWSService subscreve este delegado para adiar a sua ligação WebSocket até que a autenticação seja confirmada, evitando tentativas de ligação com credenciais inválidas ou ausentes.

Quando a autenticação Basic é configurada em vez de OAuth2, o cabeçalho Authorization é calculado imediatamente a partir do nome de utilizador e palavra-passe armazenados, e emitido sincronamente durante o Initialize(). O UWSService deteta este caso verificando se o cabeçalho de autenticação já está preenchido no momento das subscrições e ligando-se imediatamente se estiver.

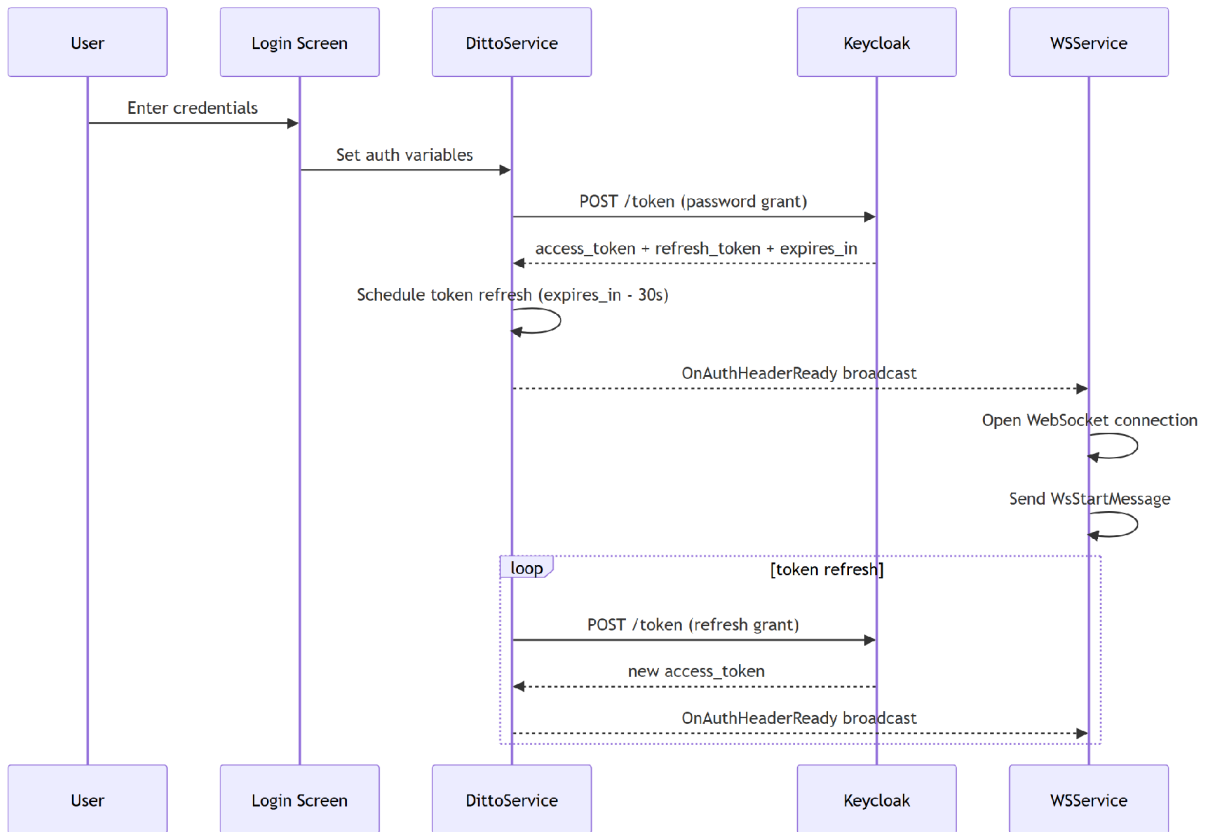


Figura 8: Fluxo de arranque OAuth2 desde o ecrã de login até à obtenção de token e ligação WebSocket

6.12 Aquisição de Dados

A aquisição de dados é realizada através de dois mecanismos complementares: pedidos HTTPS REST contra a API do Eclipse Ditto e uma subscrição WebSocket persistente ao stream de eventos do Ditto. Em conjunto, estes mecanismos permitem que a aplicação reconstrua o estado atual do Digital Twin no arranque e se mantenha sincronizada com as alterações à medida que ocorrem.

6.12.1 REST Acquisition (UDittoService)

O UDittoService é um UGameInstanceSubsystem que encapsula o módulo HTTP do Unreal para comunicar com a API REST do Ditto versão 2. Disponibiliza quatro operações públicas:

- **GetAllThings** — Obtém todos os Things cujo thingId contém a substring geo, utilizando o endpoint de pesquisa do lado do servidor do Ditto (/api/2/search/Things). Os resultados são paginados em cinquenta itens por página. Esta operação é utilizada apenas para fins de depuração e não é chamada durante o funcionamento normal da aplicação.
- **GetThingsByGeotile** — Obtém Things cujo campo numérico attributes/geotile está dentro de um intervalo correspondente a um tile geográfico num determinado nível de zoom. O valor geotile é um quadkey inteiro calculado no nível de zoom 31 (a resolução mais alta). Para consultar um tile de menor resolução, o serviço calcula o quadkey do tile no nível de zoom solicitado e deriva um intervalo [Lower, Upper) deslocando a chave para a esquerda por $2 \times (31 - \text{zoom})$ bits. Este intervalo é passado ao Ditto como um filtro RSqL: `and(ge(attributes/geotile, Lower), le(attributes/geotile, Upper))`. Os resultados são paginados em duzentos itens por página.

- **GetThingById** — Obtém um único Thing pelo seu identificador completo a partir de `/api/2/Things/<id>`.
- **PutThing** — Cria ou substitui um Thing via HTTP PUT, utilizado quando o utilizador coloca uma nova entidade interativamente (ver Secção 12).

Todos os pedidos são enviados através do `SendAuthenticatedRequest()`, que anexa o cabeçalho `Authorization` atual e coloca o pedido em fila se o token ainda não estiver disponível.

6.12.2 Quadkey Geographic Math

O serviço contém funções utilitárias estáticas para aritmética de quadkeys estilo OSM. O `GetTileXY()` converte um par latitude/longitude WGS-84 para coordenadas de grelha de tiles OSM num determinado nível de zoom, utilizando a projeção Web Mercator. O `GetQuadkeyFromXY()` codifica uma posição de grelha de tile como um único inteiro de 64 bits, intercalando os planos de bits X e Y. O `AltitudeToZoomLevel()` converte uma altitude da câmara em metros para um nível de zoom inteiro utilizando a fórmula $Z = \log_2(10019000 / \text{altitude})$, limitada ao intervalo 0–20. 10019000 é aproximadamente um quarto da circunferência da Terra em metros. É utilizado como ponto de ancoragem para que, no nível de zoom 0 (visão mundial), a fórmula produza uma altitude de câmara que corresponda à escala da projeção Web Mercator.

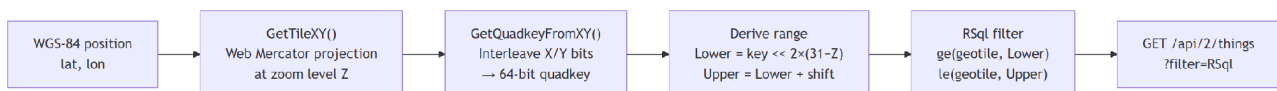


Figura 9: Como um ponto geográfico é mapeado para um intervalo inteiro de geotile que é enviado como filtro RSqL para a API REST do Ditto

6.12.3 WebSocket Acquisition (UWSService and UEntityUpdateDaemon)

O `UWSService` é um `UGameInstanceSubsystem` que gerencia uma única ligação WebSocket persistente ao endpoint da API WebSocket do Ditto. Após abrir a ligação, envia imediatamente uma mensagem de início (definida no ecrã de login) que subscreve a sessão a todos os eventos de modificação de twins. O serviço expõe quatro delegados multicast: `OnConnected`, `OnMessage`, `OnError` e `OnClosed`.

Quando a ligação é interrompida inesperadamente, o serviço agenda uma tentativa de reconexão utilizando back-off exponencial: o atraso base (cinco segundos) é multiplicado por dois elevado ao número atual de tentativas, até um máximo de cinco tentativas consecutivas, após as quais o serviço para de tentar. Chamar `Disconnect()` define manualmente a flag `bManualClose`, que suprime a lógica de reconexão automática, permitindo um encerramento limpo.

O `UEntityUpdateDaemon` é um `UGameInstanceSubsystem` que se situa entre o `UWSService` e os atores de entidade individuais. Subscreve o `UWSService::OnMessage` e analisa cada mensagem recebida como um envelope do protocolo Ditto. O formato do envelope é:

```

{
  "topic": "<namespace>/<subject>/<Thing>/Things/twin/events/modified",
  "path": "/features/lidar-outsight/properties/1",
  "value": {...}
}

```

O daemon extrai o `thingId` juntando os três primeiros segmentos separados por barras do campo `topic` com dois pontos (`namespace:subject:Thing`), utiliza o campo `path` diretamente e serializa o campo `value` de volta para uma string JSON. De seguida, procura o `thingId` num `TMap<FString, TArray<FOnEntityUpdated*>>` interno e transmite o par (`path`, `valueJson`) para cada delegado

registrado para essa chave.

Os atores registam o seu delegado `OnEntityUpdated` no `BeginPlay()` através de `RegisterEntity()` e cancelam o registo no `EndPlay()` através de `UnregisterEntity()`. Se um evento `WebSocket` chegar para um `thingId` sem ator registrado, o evento é silenciosamente descartado.

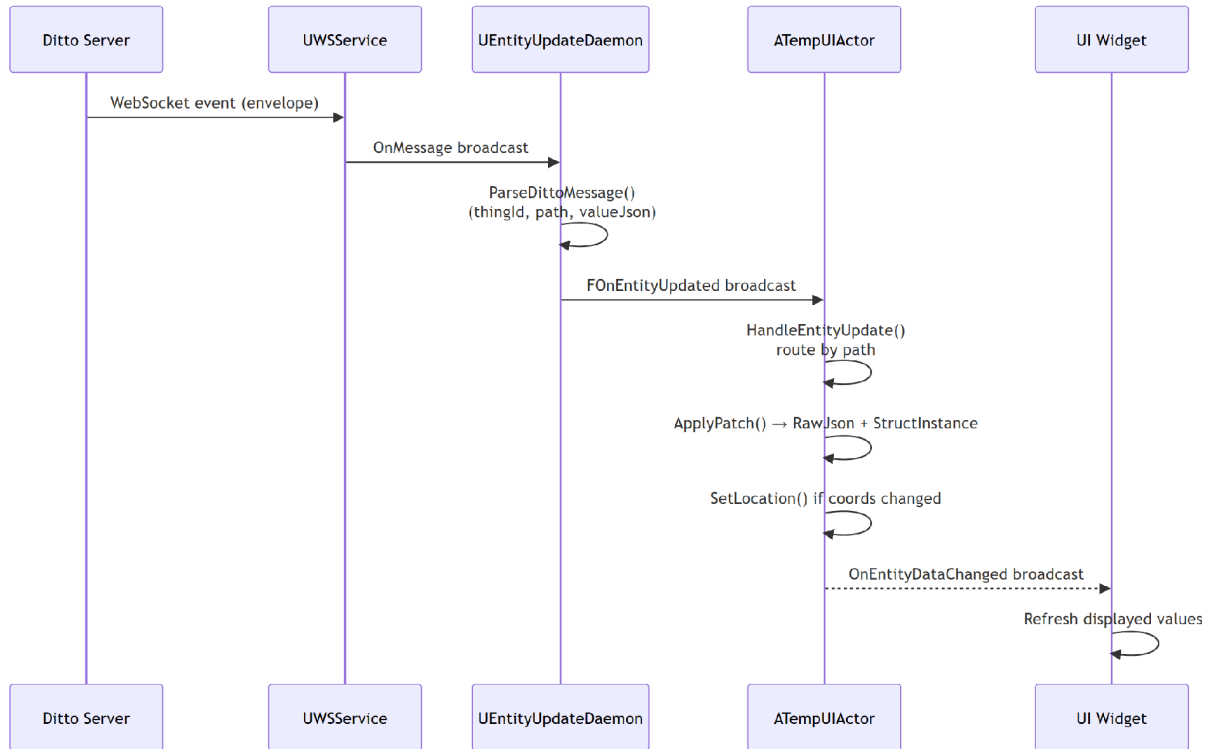


Figura 10: Percurso de atualização em tempo real desde o evento `WebSocket` do Ditto até ao ator de entidade e à atualização da UI

6.13 Conversão de Coordenadas Geográficas

Um requisito central da plataforma de visualização é o posicionamento preciso de entidades no globo. Todos os Ditto Things armazenam a sua posição geográfica em graus decimais WGS-84 (latitude e longitude) com uma altitude opcional em metros acima do nível médio do mar. O Unreal Engine utiliza um sistema de coordenadas cartesiano local plano; portanto, posicionar objetos corretamente requer uma cadeia de conversão desde coordenadas geodésicas até ao espaço do mundo do motor.

Esta conversão é realizada por dois componentes que cooperam: o plugin Cesium for Unreal e um wrapper singleton ligeiro, o `UCoordinatesConversionService`.

O `ACesiumGeoreference` é um ator colocado no nível pelo plugin Cesium. Mantém a relação entre coordenadas Earth-Centered Earth-Fixed (ECEF) e o espaço do mundo do Unreal, e reposiciona a origem dinamicamente à medida que a câmara se afasta da origem da cena para evitar perda de precisão de vírgula flutuante. Todas as transformações de coordenadas devem passar pelo ator de georeferência para que este reposicionamento seja aplicado corretamente.

O `UCoordinatesConversionService` é um singleton ligeiro (protegido contra garbage collection) que expõe um único ponto de entrada de conversão, o `ConvertWSG84ToUELocal()`. Internamente, chama o `ACesiumGeoreference::TransformLongitudeLatitudeHeightPositionToUnreal()` na georeferência padrão do mundo, passando longitude, latitude e altitude em metros. O resultado é a posição no mundo do Unreal em centímetros, que é depois passada para o `UCesiumGlobeAnchorComponent` do ator.

O `UCesiumGlobeAnchorComponent` em cada `ATempUIActor` mantém o ator fixo às suas coordenadas geográficas através de reposicionamentos de origem. Quando `MoveToLongitudeLatitudeHeight()` é chamado na âncora, esta calcula a nova posição ECEF. Mantém o ator visualmente estável à medida que a câmara se move pelo globo, mesmo quando a origem do mundo do Unreal se desloca.

Para o tratamento de altitude, o serviço aplica uma correção de undulação geoidal EGM96 de 53 metros quando está presente uma altitude acima do nível médio do mar. Este valor está calibrado para a área de implantação do projeto (aproximadamente 39° N, 9° O, Portugal central). As entidades sem altitude explícita são instead posicionadas na superfície do terreno renderizado utilizando um processo em tempo de execução descrito na Secção 6.6.

6.14 Modelos de Dados de Entidade

Cada tipo de Ditto Thing na plataforma DT4MOB é representado internamente por uma `USTRUCT` que espelha o esquema JSON para esse tipo. Estas structs são definidas como tipos refletidos do Unreal Engine utilizando a macro `USTRUCT(BlueprintType)`, o que permite tanto a acessibilidade em Blueprint como a reflexão em tempo de execução. A utilidade `FJsonObjectConverter::JsonObjectToUStruct()` é utilizada para desserializar payloads JSON recebidos diretamente em instâncias de struct em tempo de execução, sem necessitar de código de análise escrito manualmente.

Cada struct contém, no mínimo, um campo de string `thingId` e uma sub-struct `attributes`. A struct `attributes` contém a posição geográfica da entidade (quer uma struct aninhada `location/geometry` com campos de latitude/longitude, quer campos planos latitude/longitude diretamente nos `attributes`), juntamente com campos específicos do domínio.

A plataforma de visualização pode criar entidades de qualquer tipo registado através de `UDittoService::PutThing()`. No entanto, apenas as entidades de fogo (`fire: / FIgnitionPointData`) têm tratamento de simulação do lado do servidor: após a colocação, o servidor atualiza os campos `features.cone` e `features.perimeters` da entidade com polígonos de propagação de fogo com horizontes temporais de 30, 60, 90 e 120 minutos. Para todos os outros tipos de entidade, a entidade criada persiste no Ditto mas não recebe processamento do lado do servidor.

6.15 Representação de Entidade (ATempUIActor)

O `ATempUIActor` é a classe central de ator do mundo na plataforma de visualização. Cada objeto Ditto carregado na cena é representado por exatamente uma instância de `ATempUIActor`. A classe encapsula a representação visual do ator, os seus dados atualizados e a sua integração com os sistemas de seleção, atualização e posicionamento.

6.15.1 Components

Cada `ATempUIActor` é construído com quatro subcomponentes padrão:

- **UCesiumGlobeAnchorComponent (GlobeAnchor)** — Fixa o ator a uma coordenada geográfica através do reposicionamento de origem do Cesium. As alterações de posição são sempre aplicadas através do `MoveToLongitudeLatitudeHeight()` deste componente, em vez de através de `SetActorLocation()` diretamente.
- **USceneComponent (SceneRoot)** — A transformação raiz do ator.
- **UBoxComponent (InteractionBox)** — Uma caixa de semi-extensão de 300 cm com colisão ativada apenas no canal personalizado `ECC_GameTraceChannel1`. Este canal é utilizado

exclusivamente para teste de interseção do cursor, prevenindo interferência com a física ou outros traces.

- **UStaticMeshComponent (StaticMesh)** — A representação visual padrão. Por omissão, utiliza a malha Cube integrada no motor. É substituída ou oculta quando um modelo específico de tipo ou GLB é carregado.

Um TMap<FString, UStaticMeshComponent*> chamado MeshLayers permite que componentes de malha adicionais nomeados sejam adicionados em tempo de execução.

6.15.2 Initialisation

Quando o ATempUIActor::Initialize() é chamado pela factory após a instanciação, este:

1. Armazena o tipo de struct e cria uma instância FStructOnScope atualizada a partir dele.
2. Desserializa o payload JSON na instância da struct utilizando FJsonObjectConverter.
3. Extrai o campo thingId da instância da struct através de reflexão de propriedades em tempo de execução.
4. Chama SetLocation() para posicionar o ator no globo.
5. Chama TryApplyExpiry() para agendar a autodestruição se estiver presente um atributo expiry_ts.
6. Verifica a lista de URLs attributes.polygon e inicia uma obtenção assíncrona de modelo GLB se estiver presente.
7. Escala a malha padrão e a caixa de interação a partir de attributes.length/width/height se estiverem presentes.
8. Se o BeginPlay já tiver sido executado, regista-se imediatamente no UEntityUpdateDaemon e no UActorRegistryService.

6.15.3 Geographic Positioning (SetLocation)

O SetLocation() percorre a hierarquia de structs refletida do ator em tempo de execução utilizando o TFieldIterator do Unreal para encontrar valores de latitude e longitude sem depender de um layout de struct específico. A pesquisa segue uma ordem de prioridade:

1. Uma sub-struct attributes.location ou attributes.geometry contendo campos latitude/longitude/altitude.
2. Campos planos attributes.latitude / attributes.longitude.
3. Uma varredura recursiva de todas as subpropriedades de struct de attributes para qualquer propriedade chamada lat ou lon (trata casos como FIgnitionPointAttributes::fire_ignition.lat).
4. Uma varredura de fallback de features.<name>.properties.absoluteCoordinates no JSON bruto, utilizado por entidades de câmaras de portagem cuja posição é armazenada numa propriedade de feature em vez de em attributes.
5. Campos diretos latitude/longitude na raiz de propriedades de qualquer feature, utilizado por entidades de veículos TRACI.

Se for encontrada uma altitude válida, é aplicada uma correção de undulação geoidal EGM96 de 53 metros para converter a altitude acima do nível médio do mar para altura elipsoidal WGS-84, e `bHasExplicitAltitude` é definido para suprimir tentativas subsequentes de snap ao terreno.

6.15.4 Live Update Routing (HandleEntityUpdate)

Quando o `EntityUpdateDaemon` entrega uma atualização ao delegado `OnEntityUpdated` do ator, o `HandleEntityUpdate()` encaminha-a para o manipulador de patch apropriado com base no caminho Ditto:

Padrão de caminho M	anipulador E	feito
/ ou /attributes/...	<code>ApplyFullOrAttributePatch</code> .	Faz merge profundo do valor no <code>RawJson</code> ao nível superior, redesserializa a struct e reexecuta <code>SetLocation()</code>
/features/<name>/properties/<id>.	<code>ApplyFeaturePatch</code>	Escreve o valor em <code>RawJson["features"][nome]["properties"][id]</code> . Extrai <code>absoluteCoordinates</code> , <code>speedKmh</code> e <code>dimensions</code> de atualizações de sensores para atualizar a posição e escala do ator.
/features/<name>/properties.	<code>ApplyFeaturePropertiesPatch</code> .	Substitui todo o bloco de propriedades de uma feature. Trata atualizações de posição de veículos TRACI e atualizações de estado camera-ORT.

Tabela 29 - Roteamento de Atualizações em Tempo Real (HandleEntityUpdate)

O merge profundo (em vez de substituição) é utilizado para patches completos e de atributos, para que campos ausentes do patch, como `thingId`, `policyId` ou atributos de metadados estáticos, sejam preservados no `RawJson` através de atualizações incrementais. Após cada patch, o `OnEntityDataChanged` é emitido, notificando quaisquer widgets de UI abertos de que os dados do ator foram alterados.

6.15.5 Smooth Position Interpolation

Para entidades dinâmicas, como veículos, o ator implementa interpolação de posição por tick entre eventos de atualização `WebSocket`. Em vez de teletransportar o ator para cada nova coordenada à medida que chega, o ator mantém um par `VisualLatitude/VisualLongitude` que é suavemente movido em direção ao `LastLatitude/LastLongitude` alvo a cada frame.

Quando um valor `speedKmh` está presente na atualização, a interpolação avança a posição visual a essa velocidade. Quando não está presente nenhuma velocidade, a posição é distribuída uniformemente pelo intervalo estimado entre atualizações, que é mantido como uma média móvel do tempo medido entre chegadas consecutivas de atualizações.

Para atualizações de veículos TRACI que incluem um ângulo de rumo, o ator aplica `CesiumGeoreference::TransformEastSouthUpRotatorToUnreal()` para rodar a malha visual para enfrentar a direção da bússola reportada, convertendo da convenção norte-zero no sentido horário do TRACI para o referencial East-South-Up do Cesium.

A primeira atualização `WebSocket` ao vivo após um ator ser instanciado teletransporta sempre a posição visual para as novas coordenadas (a flag `bTeleport`), prevenindo uma longa cauda de interpolação desde a posição inicial dos dados REST até à primeira posição ao vivo.

6.15.6 Terrain Snap (SnapToGround)

As entidades sem altitude explícita são periodicamente posicionadas na superfície do terreno renderizado. Uma verificação de visibilidade é executada a cada segundo num temporizador; se o ator estiver a menos de 5000 m da câmara e projetar dentro dos limites do ecrã, o SnapToGround() é chamado.

O processo de snap começa por realizar um trace de linha com múltiplos hits para baixo através do mundo, recolhendo todos os hits ECC_WorldStatic. Seleciona o hit de maior Z voltado para cima a partir da geometria do tile de fotogrametria P3D. Se não estiver disponível geometria de malha estática na localização (por exemplo, os tiles P3D ainda não foram transmitidos), é feito um fallback para UCesiumSampleHeightMostDetailedAsyncAction, que solicita ao Cesium que carregue ativamente o tile de terreno para a posição consultada mesmo que não esteja atualmente visível, retornando a altura elipsoidal WGS-84 amostrada de forma assíncrona.

Após o snap, bSnappedToGround é definido e o temporizador é limpo. O temporizador é rearmado apenas quando o ator se move mais de aproximadamente 50 m da sua última posição de snap.

6.15.7 GLB Model Loading

Certos tipos de entidade possuem o campo attributes.polygon que referencia um ou mais ficheiros .glb alojados num servidor compatível com S3. Este campo pode ser um array JSON de strings URL ou uma única string URL, dependendo do tipo de entidade; quando presente como lista, todos os URLs são obtidos. Quando um URL é detetado durante a inicialização ou um patch subsequente, o ator solicita a malha ao UGlbModelService (Secção 11). Após receção, as malhas são aplicadas como camadas nomeadas; a malha de cubo padrão é ocultada.

6.15.8 Terrain Exclusion Polygon

Para entidades de simulação de incêndio, quando uma malha GLB é carregada, os tiles de terreno P3D subjacentes parecem interseccionar a malha de grande área visualmente. Para evitar isto, o ator instancia um ACesiumCartographicPolygon cuja spline traça o limite da malha e regista-o com um UCesiumPolygonRasterOverlay em cada ACesium3DTileset não relacionado com terreno no nível. A sobreposição indica ao Cesium que ignore a renderização de tiles de terreno que caiam dentro do polígono, criando um recorte visual limpo sob a malha GLB. Isto é aplicado apenas a entidades de fogo porque as suas malhas de grande pegada são o único caso em que o clipping de terreno é visualmente significativo.

O limite do polígono é calculado extraindo as arestas de fronteira da malha (arestas referenciadas por exatamente um triângulo), percorrendo o ciclo de arestas resultante e projetando os vértices no plano XY. Se a extração de fronteira falhar, é usado o anel de perímetro de fogo mais externo de features.perimeters. Um fallback de caixa envolvente alinhada aos eixos é utilizado se todos os outros métodos falharem.

6.15.9 Entity Expiry

Entidades efémeras, como veículos, possuem o timestamp ISO-8601 attributes.expiry_ts. Quando este campo está presente, o ator calcula o tempo de vida restante em segundos e agenda um FTimerHandle que chama Destroy() quando é acionado. Se o timestamp já estiver no passado no momento da instanciação, o ator destrói-se imediatamente.

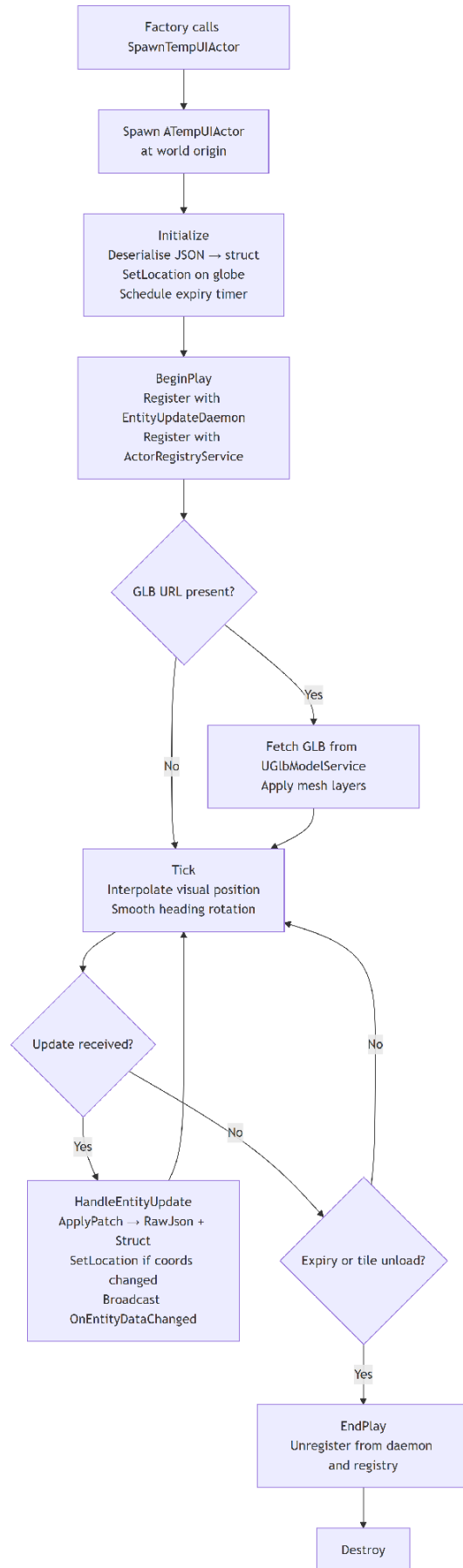


Figura 11: Ciclo de vida do ATempUIActor desde a instanciação pela fábrica até à destruição

6.16 Entity Factory and Tile-Based Streaming

6.16.1 UDT4MOBEntityFactory

O UDT4MOBEntityFactory é um UGameInstanceSubsystem responsável por converter objetos JSON brutos do Ditto em atores do mundo ativos. Mantém um TMap<FString, UScriptStruct*> (ThingStructMap) populado no seu construtor, mapeando substrings de chave de fábrica para tipos de struct.

A resolução de struct utiliza uma estratégia de correspondência mais longa: cada chave registada é testada quanto à contenção dentro do thingId completo do Thing, e a entrada com a chave correspondente mais longa é selecionada. Isto permite que uma chave específica, como .instrument., tenha prioridade sobre uma chave mais ampla, como geo-asset, para thingIds de instrumento que contenham ambas as substrings.

Quando o SpawnTempUIActor() é chamado, a fábrica:

1. Resolve o tipo de struct através da pesquisa de correspondência mais longa. Se não for encontrada correspondência, o JSON é escrito em Project/logs/<thingId>.json para inspeção posterior e é retornado nullptr.
2. Instancia um ATempUIActor na origem do mundo.
3. Chama Initialize() no ator, passando o tipo de struct resolvido e o objeto JSON.
4. Se o tipo resolvido tiver um DefaultMeshPath registado, carrega o asset UStaticMesh e atribui-o ao StaticMeshComponent do ator.
5. Se o thingId específico tiver caminhos de substituição de malha por ID registados (ThingMeshOverrideMap), procura atores de nível existentes por label ou carrega os assets de malha e atribui-os como camadas nomeadas. O cubo padrão é ocultado.
6. Adiciona o ator a SpawnedActors para controlo do ciclo de vida.

Um registo FEntityTypeMeta (TypeMetaMap) armazena nomes de exibição e uma flag bNoServerHandling para cada chave de tipo. Esta informação é consumida pelo menu suspenso de tipos de entidade na UI da barra de ferramentas.

6.16.2 ADT4MOBGamemode and Tile-Based Streaming

O ADT4MOBGamemode orquestra o carregamento inicial de entidades e o carregamento e descarregamento incremental de entidades à medida que a câmara se move.

No BeginPlay(), o gamemode chama UDittoService::GetThingsByGeotile() com um callback que despacha cada página recebida para a thread do jogo através de AsyncTask(ENamedThreads::GameThread, ...) para garantir que a instanciação de atores ocorre na thread que possui o mundo. Isto permite que os callbacks de resposta HTTP (que são executados numa thread de trabalho) interajam com segurança com os objetos do mundo do Unreal.

O gamemode também implementa streaming baseado em tiles através de CheckAndRefreshTiles(). Quando ativo, o sistema:

1. Lê a posição WGS-84 da câmara a partir de AUnifiedPawn através de CesiumGeoreference.

2. Converte a altitude da câmara para um nível de zoom utilizando `AltitudeToZoomLevel()`. A filtragem de tiles não é aplicada abaixo do nível de zoom 7.
3. Calcula uma grelha 3×3 de quadkeys centrada no tile da câmara utilizando `GetNeighborTileKeys()`.
4. Compara o novo conjunto de tiles com o conjunto previamente carregado. Se for detetada uma alteração, um temporizador de debounce de 0,75 segundos é iniciado; a atualização de tiles só é acionada se o conjunto de tiles permanecer estável durante todo o período de debounce, prevenindo obtenções rápidas durante movimento rápido da câmara.
5. No `DoTileRefresh()`, destrói atores para tiles que saíram do conjunto e emite chamadas `GetThingsByGeotile()` para tiles recém-entrados, instanciando atores na thread do jogo à medida que as páginas chegam.

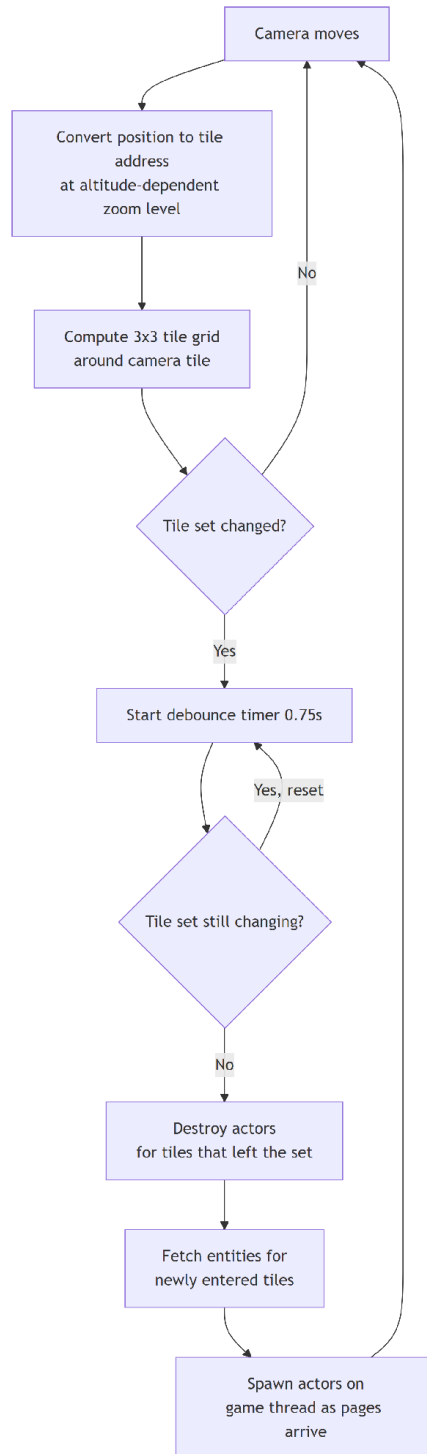


Figura 12: Ciclo de streaming baseado em tiles mostrando a grelha 3×3 centrada no tile da câmara e o debounce de 0,75 s a prevenir atualizações espúrias durante movimento rápido

6.17 Sistema de Navegação e Câmara

A plataforma de visualização disponibiliza dois modos de câmara que operam no mesmo par AUnifiedPawn / AUnifiedController: um modo RTS aéreo para observação geográfica ampla e um modo FreeFly para inspeção ao nível do solo. O modo ativo é controlado pelo enum ECameraMode e pode ser alternado em tempo de execução.

6.17.1 AUnifiedPawn

O AUnifiedPawn estende APawn e possui um USpringArmComponent e um UCameraComponent. Ambos os modos partilham este braço de mola, com diferenças específicas de modo no comprimento do braço e na rotação relativa.

RTS Mode — O braço de mola é definido com uma rotação fixa de $(-90^\circ, -90^\circ, 0^\circ)$ em relação ao pawn, produzindo uma perspetiva diretamente aérea. O comprimento do braço representa o nível de zoom, variando de um mínimo de 1500 unidades Unreal (inspeção aproximada) a um máximo de 750.000.000 unidades Unreal (visão global). Em cada tick, o `UpdateRtsAlignment()` reorienta o corpo do pawn para que os seus eixos X e Y locais estejam alinhados com o Norte e Este geográficos na posição atual do globo, consultando o `CesiumGeoreference` para obter os vetores Up e North ECEF e construindo um quaternião a partir deles.

FreeFly Mode — O comprimento do braço de mola é reduzido a zero, pelo que a câmara se fixa diretamente à raiz do pawn, e o pawn utiliza a rotação do controlador. O `UpdateFreeFlyAlignment()` extrai o vetor up do globo na posição atual e reortogonaliza a rotação do controlador contra ele a cada tick, mantendo a vista vertical na superfície curva sem permitir acumulação de roll.

O `GetGlobeUpVector()` calcula a normal da superfície na posição ECEF atual do pawn normalizando o vetor de posição ECEF. O `GetGlobeNorthVector()` projeta a direção do polo Norte ECEF no plano tangente local definido pelo vetor up do globo. Ao alternar de RTS para FreeFly, o `SnapFreeFlyToGlobeUp()` é chamado para alinhar instantaneamente a rotação do controlador com a superfície do globo, prevenindo uma transição abrupta.

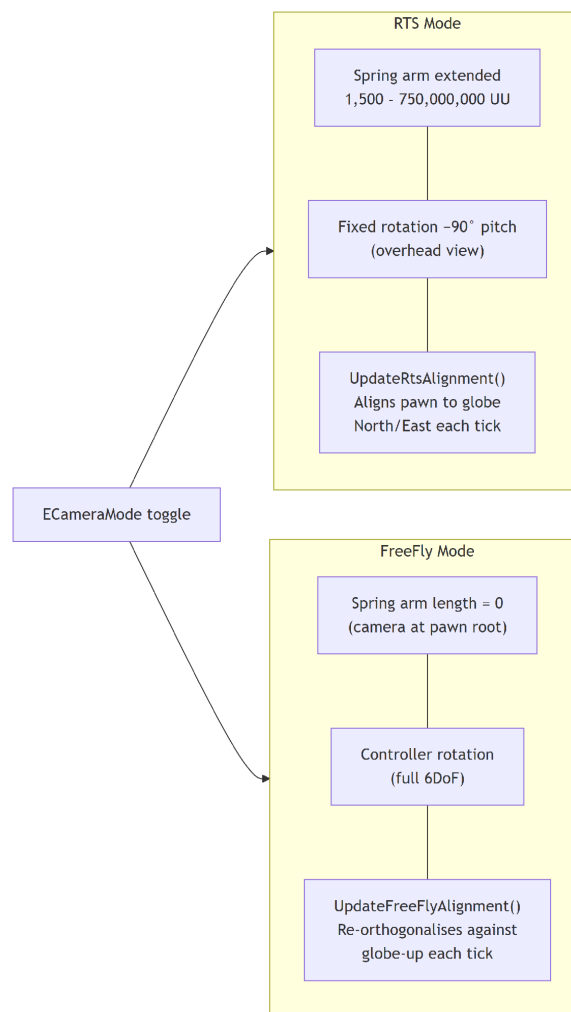


Figura 13: Comparação do modo RTS (braço de mola aéreo, globo totalmente visível) e do modo FreeFly (braço de mola colapsado, perspectiva ao nível do solo com orientação vertical ao globo)

6.17.2 AUnifiedController

O AUnifiedController estende APlayerController e utiliza o sistema Enhanced Input do Unreal. Armazena em cache ponteiros para USelectionManager e UPlacementManager da coleção de subsistemas do jogador local.

Ações de entrada vinculadas:

- **Move** — Reencaminhado para AUnifiedPawn::HandleMove(). No modo RTS, desloca o pawn nas direções Norte/Este do globo; no modo FreeFly, translada para a frente/direita em relação à vista.
- **Look** — Aplica yaw e pitch através da rotação do braço de mola ou do controlador, dependendo do modo. Suprimido no FreeFly quando o rato está destrancado.
- **Zoom** — Ajusta TargetArmLength; o braço interpola suavemente em direção ao alvo a cada tick.
- **VerticalMove** — Move o pawn ao longo do eixo up do globo no modo RTS; para cima/baixo ao longo do eixo up local no FreeFly.
- **ToggleMouseUnlock** — No modo FreeFly, alterna entre o modo de olhar com rato trancado e um modo de interação com cursor visível.
- **LeftClick** — Se o posicionamento estiver em andamento, confirma a colocação da entidade. Caso contrário, realiza um trace de hit do cursor e atualiza o SelectionManager.

Em cada tick, o UpdateHover() traça a partir do cursor ao longo do canal ECC_WorldDynamic e chama USelectionManager::SetHoveredActor() com o resultado. Um trace separado ECC_WorldStatic é realizado quando o gestor de posicionamento está ativo para seguir a superfície do terreno para a pré-visualização fantasma de posicionamento. O controlador suprime a entrada de movimento através de SetMovementInputSuppressed() quando uma janela de UI está a ser arrastada, prevenindo movimento acidental da câmara durante o movimento da janela.

6.18 Gestores e Lógica de Aplicação

6.18.1 USelectionManager (LocalPlayer Subsystem)

O USelectionManager é um ULocalPlayerSubsystem que monitoriza os atores de entidade atualmente selecionados e sob o cursor, aceitando apenas instâncias de ATempUIActor como alvos válidos. As emissões são suprimidas quando o novo valor é igual ao valor atual, garantindo que os consumidores a jusante são apenas notificados quando ocorre uma alteração genuína de estado.

Cada ATempUIActor subscreve OnHoveredActorChanged e OnSelectedActorChanged no BeginPlay(). Quando notificado, chama UpdateHighlight(), que ativa ou desativa a renderização de profundidade personalizada em todos os seus componentes primitivos e define o valor de stencil para 1 (sobreposição) ou 2 (selecionado). Um material de pós-processamento lê o stencil de profundidade personalizado no nível para desenhar o contorno de destaque em torno do ator.

6.18.2 UUIManager (LocalPlayer Subsystem)

O UUIManager é um ULocalPlayerSubsystem que coordena o estado de visibilidade global da UI.

No `Initialize()`, vincula-se a `USelectionManager::OnSelectedActorChanged`. Quando um ator é selecionado, emite `OnEntityWindowVisibilityChanged(true)`; quando a seleção é limpa, emite `OnEntityWindowVisibilityChanged(false)`.

6.18.3 UActorRegistryService (GameInstance Subsystem)

O `UActorRegistryService` mantém um `TMap<FString, ATempUIActor*>` de todos os atores ativos indexados por `thingId`. Os atores registam-se no `BeginPlay()` e cancelam o registo no `EndPlay()`. O `UOutlinePanelWidget` consulta este registo para preencher a lista de outline da cena. O serviço emite os delegados `OnActorRegistered` e `OnActorUnregistered` para que os componentes de UI possam reagir a atores a serem instanciados ou destruídos sem necessidade de polling.

6.18.4 UPlacementManager (LocalPlayer Subsystem)

O `UPlacementManager` gere o fluxo de trabalho de colocação de entidades. Quando o utilizador seleciona um tipo de entidade no menu suspenso da barra de ferramentas e ativa o modo de colocação, o gestor instancia um ator fantasma que segue a superfície do terreno sob o cursor a cada tick. Quando o utilizador clica com o botão esquerdo para confirmar, o controlador lê a posição de hit do terreno, converte-a para coordenadas WGS-84 através do `CesiumGeoreference`, constrói o corpo JSON apropriado, chama `UDittoService::PutThing()` e instancia um ator local imediatamente sem esperar que o PUT seja bem-sucedido.

6.19 Interface do Utilizador

A interface do utilizador é implementada utilizando Unreal Motion Graphics (UMG). As classes C++ definem a lógica dos widgets e expõem propriedades a subclasses Blueprint, que definem o layout visual.

6.19.1 URootHUDWidget

O `URootHUDWidget` é o widget de topo criado por `AUnifiedController::BeginPlay()` e adicionado ao viewport. É o contendor para todos os elementos HUD persistentes:

- **UToolbarWidget** — Uma barra horizontal no topo do ecrã contendo a seleção suspensa de tipo de entidade e um botão de alternância de colocação.
- **UOutlinePanelWidget** — Um painel deslizante que lista todas as entidades atualmente carregadas com um campo de pesquisa para filtrar por `thingId` ou nome de exibição.
- **UCanvasPanel (EntityWindowContainer)** — Uma tela transparente de ecrã inteiro na qual as janelas de inspeção de entidades são instanciadas como widgets filhos.

O `URootHUDWidget` subscreve `USelectionManager::OnSelectedActorChanged`. Quando um ator é selecionado, o `SpawnOrFocusWindow()` é chamado: se já estiver aberta uma janela para esse `thingId`, é trazida para a frente; caso contrário, um novo `UEntityWindowWidget` é instanciado e vinculado ao ator. Múltiplas janelas de entidade podem estar abertas simultaneamente, uma por `thingId`.

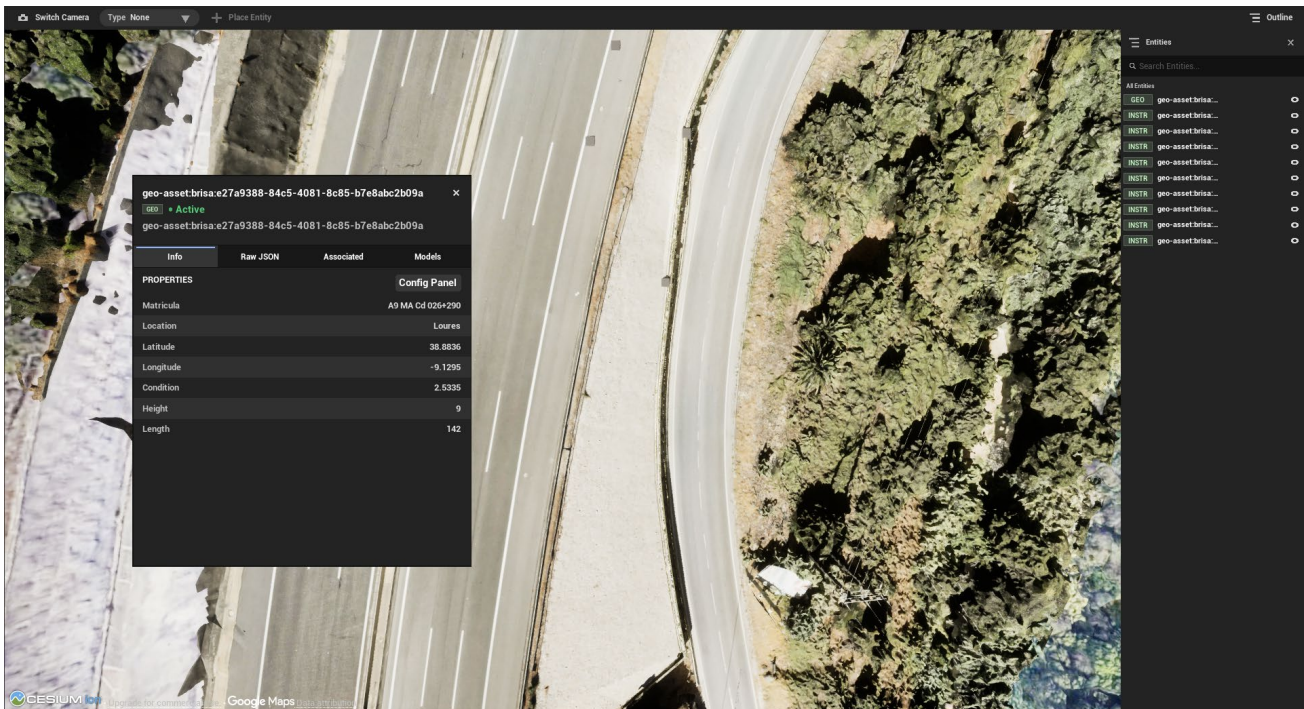


Figura 14: Visão geral da aplicação com a barra de ferramentas no topo, uma janela de inspeção de entidade aberta e o painel de outline à direita

6.19.2 UEntityWindowWidget

O UEntityWindowWidget é o painel de inspeção por entidade. É uma janela flutuante arrastável e redimensionável posicionada através de um UCanvasPanelSlot. O cabeçalho da janela mostra o thingId, um badge de tipo, uma etiqueta de estado e um botão opcional do Grafana.

O corpo da janela é uma interface com separadores com quatro separadores:

- **Info tab (UInfoTabWidget)** — Mostra uma lista configurável de pares label-valor para os campos mais relevantes da entidade. Os campos exibidos são definidos por tipo de entidade através do UInfoFieldRegistry, que armazena entradas FInfoFieldDef (cada uma com um caminho JSON separado por pontos e uma label de exibição). As atualizações ao vivo de ATempUIActor::OnEntityDataChanged atualizam os valores exibidos sem reconstruir a lista de linhas. Os utilizadores podem personalizar quais os campos que aparecem através de um painel de configuração.
- **JSON tab (UJsonTabWidget)** — Mostra o JSON bruto completo da entidade numa vista de árvore recolhível, implementada como um widget Slate personalizado (SJsonViewer). O objeto JSON é achatado num TArray<FJsonViewerRow> para renderização eficiente de lista com controlo de indentação por profundidade.
- **Assoc tab (UAssocTabWidget)** — Mostra entidades associadas obtidas consultando o Ditto por Things que referenciam este thingId nos seus attributes.
- **Models tab (UModelsTabWidget)** — Mostra as camadas de malha nomeadas registadas no ator e fornece alternadores de visibilidade por camada. Para entidades de fogo com múltiplos perímetros de horizonte temporal, este separador permite que fatias individuais de simulação sejam ativadas ou desativadas.

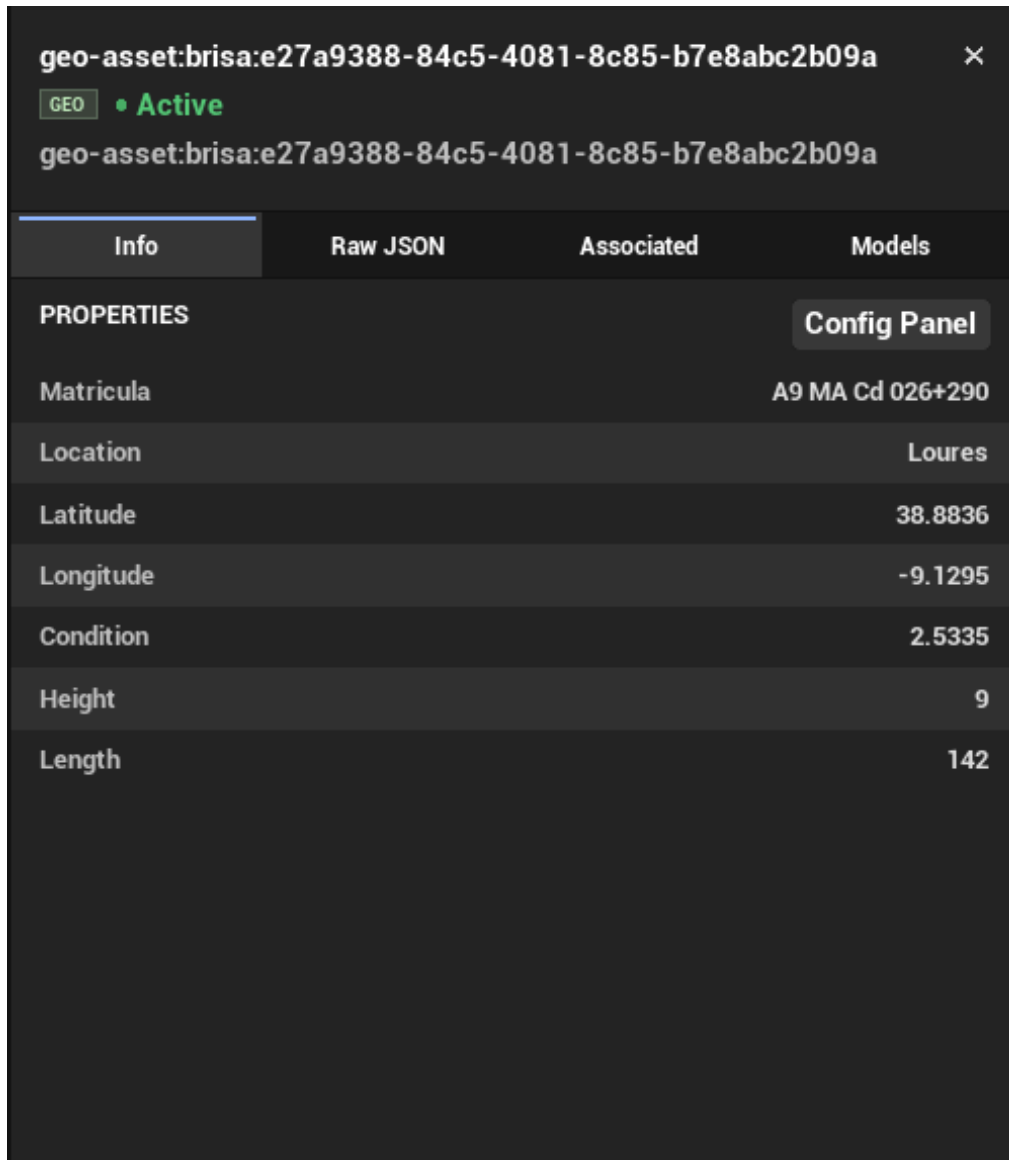


Figura 15: Close-up de uma janela de entidade aberta mostrando a barra de cabeçalho com thingId e badge de tipo, os quatro botões de separador e o separador Info

As interações de arrastar e redimensionar são tratadas em C++ através de `NativeOnMouseDown`, `NativeOnMouseMove` e `NativeOnMouseUp`. O arrasto é detetado quando o rato é pressionado dentro da área da barra de título (72 pixels a partir do topo do widget); o redimensionamento é detetado quando a pressão cai dentro do punho de redimensionamento de 20×20 pixels no canto inferior direito. Durante estas operações, a entrada de movimento no controlador é suprimida.

6.19.3 UOutlinePanelWidget

O `UOutlinePanelWidget` é um painel lateral que desliza para dentro e para fora ao alternar. Subscrive o `UActorRegistryService` e adiciona ou remove dinamicamente entradas `UOutlineRowWidget` à medida que os atores são registados e cancelados. Uma caixa de pesquisa `UEditableText` filtra as linhas exibidas em tempo real. Clicar numa linha abre ou foca a janela de entidade para esse ator.

6.19.4 UI Theming

A plataforma de visualização inclui um subsistema de temas (UIThemeSubsystem, UIThemeData, UIThemeBFL) que propaga cores e definições de tipo de letra a todos os widgets que implementam a classe base UThemedWidget. Os widgets substituem ApplyTheme_Implementation() para aplicar o tema atual aos seus componentes visuais no momento da construção e em atualizações de tema em tempo real.

6.20 GLB Model Service (UGlbModelService)

O UGlbModelService é um UGameInstanceSubsystem responsável por descarregar ficheiros de malha GLB (glTF binário) de endpoints HTTP e convertê-los em assets UStaticMesh em tempo de execução utilizando o plugin glTFRuntime do Unreal.

O serviço implementa uma cache de dois níveis:

- **Session cache** — Um TMap<FString, UStaticMesh*> em memória que mapeia URL para malha. Num hit de cache, o callback é acionado imediatamente sem um pedido de rede.
- **Disk cache** — Os corpos das malhas são guardados num diretório de cache local do projeto no primeiro descarregamento. Um ficheiro etags.json no mesmo diretório armazena o ETag retornado pelo servidor para cada URL.

Num miss de cache, o serviço emite um pedido GET. Se existir um ETag em cache para o URL, é incluído como cabeçalho If-None-Match. Uma resposta 304 Not Modified carrega a malha do disco sem a descarregar novamente. Uma resposta 200 OK substitui o ficheiro em disco e atualiza o armazenamento de ETags.

Múltiplos atores a solicitar o mesmo URL concorrentemente são coalescidos num único pedido HTTP em andamento. Os seus callbacks são armazenados num mapa PendingCallbacks indexado por URL, e todos são acionados juntos quando o pedido é concluído.

6.21 Fluxo de Interação do Utilizador

O fluxo de trabalho completo de interação do utilizador, desde o início da aplicação até à inspeção de dados da entidade, procede da seguinte forma:

1. **Login** — O utilizador introduz as credenciais do Ditto no ecrã de login. As credenciais são escritas nas variáveis de configuração em tempo de execução consumidas pelo UDittoService.
2. **Authentication** — O UDittoService inicia a aquisição de token OAuth2. O UWSService subscreve UDittoService::OnAuthHeaderReady.
3. **Authentication complete** — O UDittoService emite OnAuthHeaderReady. O UWSService abre a ligação WebSocket e envia a mensagem de início, subscrevendo os eventos de twin do Ditto.
4. **Initial entity load** — ADT4MOBGamemode::BeginPlay() emite consultas REST baseadas em tiles. Páginas de objetos JSON são recebidas da API REST do Ditto. Para cada objeto, UDT4MOBEntityFactory::SpawnTempUIActor() instancia e inicializa um ATempUIActor. Cada ator regista-se no UEntityUpdateDaemon e no UActorRegistryService.
5. **Entity appears on globe** — ATempUIActor::SetLocation() localiza latitude/longitude na struct ou no JSON bruto e chama GlobeAnchor->MoveToLongitudeLatitudeHeight(). Se não

for conhecida altitude, o ator é periodicamente posicionado na superfície do terreno assim que entra no frustum da câmara.

6. **Live update arrives** — O evento WebSocket do Ditto é analisado pelo UEntityUpdateDaemon, que extrai o thingId, caminho e valor e despacha a atualização para o ATempUIActor correspondente através do seu delegado registado. O ator atualiza o RawJson, redesserializa a struct e atualiza o seu alvo de interpolação. O OnEntityDataChanged é emitido.
7. **User hovers over an entity** — AUnifiedController::UpdateHover() traça a partir do cursor a cada tick ao longo de ECC_WorldDynamic. Num hit, USelectionManager::SetHoveredActor() notifica o ator alvo para ativar a renderização de profundidade personalizada para o contorno de sobreposição.
8. **User clicks to select** — USelectionManager::SetSelectedActor() emite OnSelectedActorChanged. UUIManager emite OnEntityWindowVisibilityChanged(true). URootHUDWidget chama SpawnOrFocusWindow(), que cria ou foca a janela de inspeção de entidade.
9. **Inspection window displays data** — UEntityWindowWidget::OpenForActor() vincula a janela ao ator e preenche o cabeçalho. UInfoTabWidget constrói as linhas do separador Info a partir das entradas UInfoFieldRegistry para a chave de tipo do ator e subscreve OnEntityDataChanged para atualizações ao vivo.
10. **User places a new entity** — O utilizador seleciona um tipo de entidade na barra de ferramentas e ativa o modo de colocação. Um ator fantasma segue a superfície do terreno sob o cursor. Ao clicar com o botão esquerdo, o controlador constrói um corpo JSON, chama UDittoService::PutThing() assincronamente e instancia imediatamente um ator local para que a colocação seja refletida na cena sem esperar pela viagem de ida e volta ao servidor.

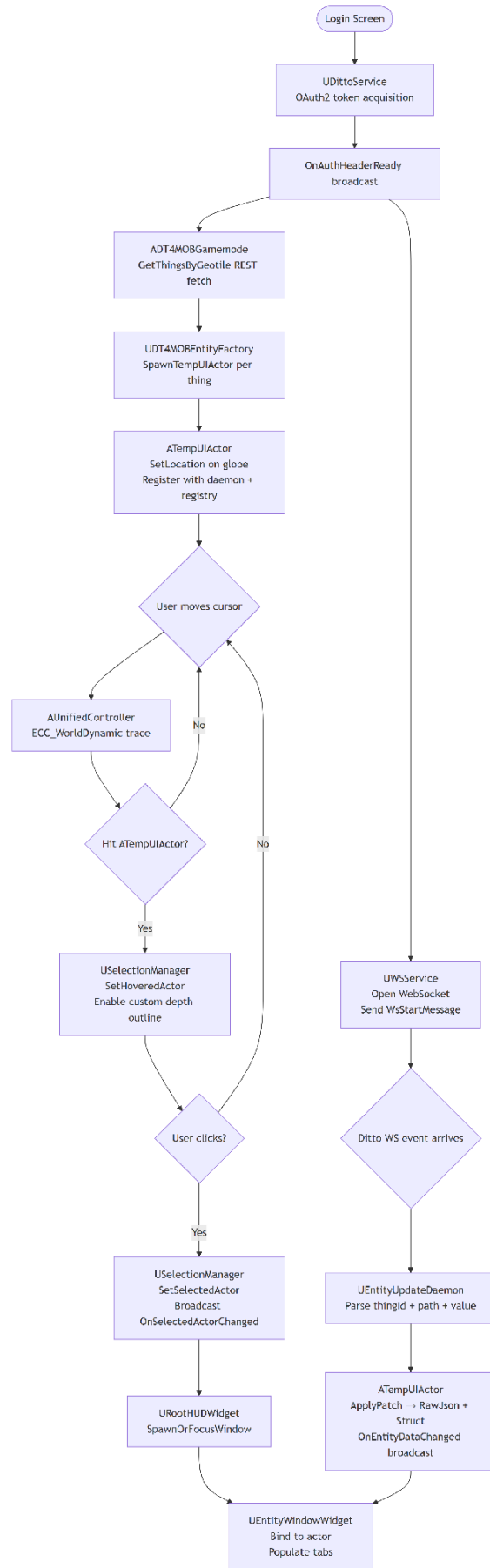


Figura 16: Fluxograma de ponta a ponta desde o login até à autenticação, carregamento inicial de entidades, atualização WebSocket ao vivo, sobreposição, seleção e exibição da janela

6.22 Referência de Configuração

O ecrã de login recolhe todos os parâmetros de ligação necessários no arranque. A tabela seguinte descreve cada campo:

Campo	Descrição
Host	Hostname do servidor Ditto (sem prefixo de protocolo). Utilizado para construir a URL base e a URL WebSocket.
Username	Nome de utilizador para autenticação no Ditto (OAuth2 ou Basic).
Password	Palavra-passe para autenticação no Ditto.
UseHttps	Booleano. Se ativado, liga via HTTPS/WSS; caso contrário, utiliza HTTP/WS.
UseOAuth	Booleano. Se ativado, obtém um token Bearer do Keycloak; caso contrário, utiliza autenticação HTTP Basic.
WsStartMessage	String que é enviada ao Ditto imediatamente após a ligação WebSocket para subscrever eventos do Digital Twin.

Tabela 30 - Referência de Configuração

Valores ausentes ou vazios produzem um aviso no Log de Saída sob a categoria DittoService e resultam em falhas HTTP silenciosas para todas as chamadas subsequentes à API.

7 DATA BRIDGE

7.1 Visão Geral

O Data Bridge é uma aplicação Python que estabelece a ponte entre fontes de dados externas e o Eclipse Ditto através do Eclipse Hono. Utiliza um padrão Strategy para integração extensível de fontes de dados. Suporta dados meteorológicos do IPMA, dados de tráfego do Waze e fontes de ficheiros GeoJSON.

O programa principal, conforme implementado atualmente, funciona da seguinte forma:

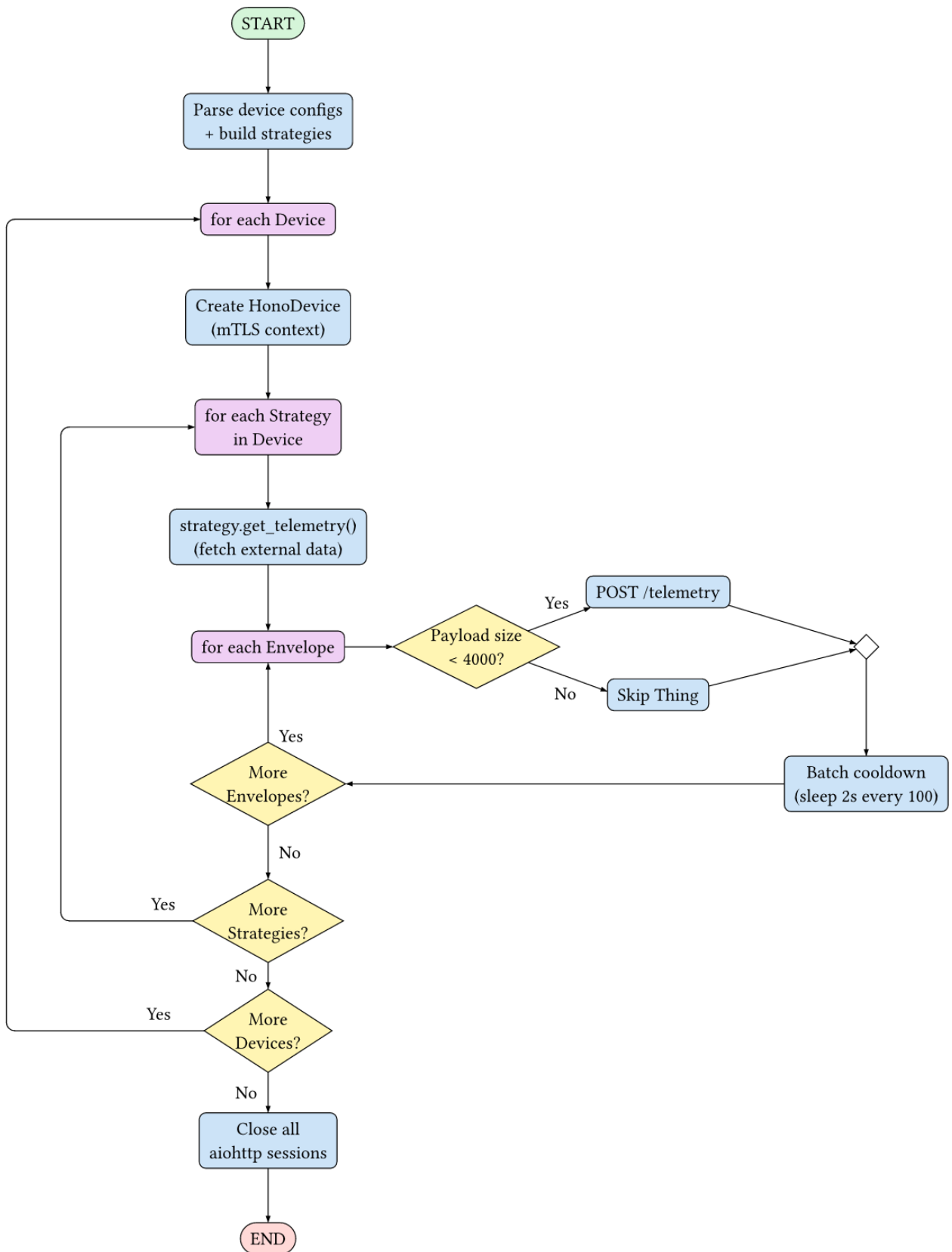


Figura 17: Grafo de fluxo de controlo do Data Bridge

As principais adições podem ser feitas implementando mais estratégias, que podem modificar o comportamento de um determinado dispositivo. Estas estratégias são responsáveis por criar mensagens de Envelope do Protocolo Ditto, que o Device enviará para o Eclipse Hono, que as reencaminhará para o Eclipse Ditto. Este protocolo pode ser consultado na [documentação oficial do Eclipse Ditto](#), e o Modelo de Dados para o encapsulamento deste comando está definido em `models/ditto.py`.

Além disso, o diagrama de sequência do fluxo de execução principal é o seguinte:

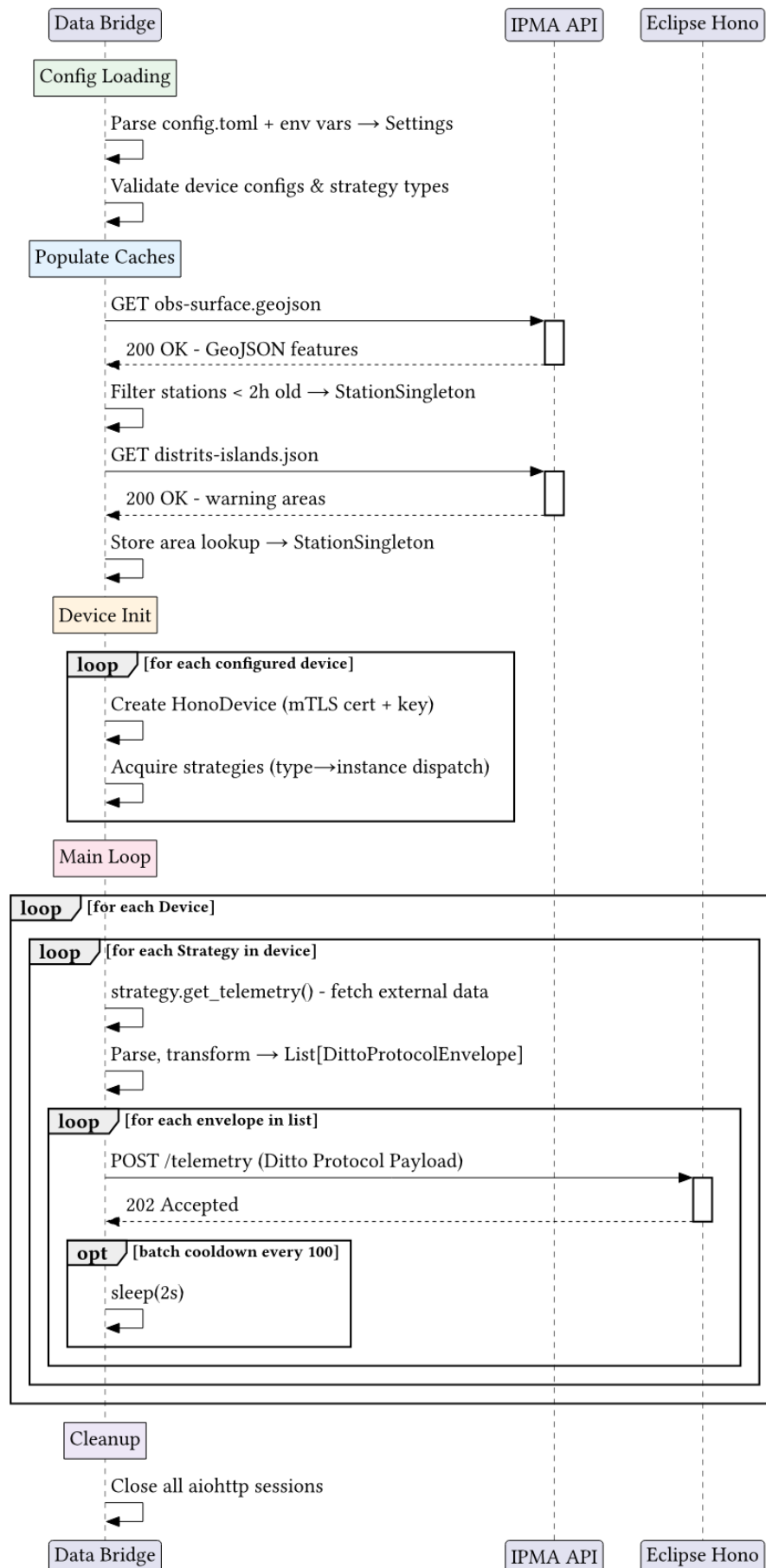


Figura 18: Diagrama de sequência do Data Bridge

7.2 Estrutura do Código

O programa está estruturado logicamente da seguinte forma:

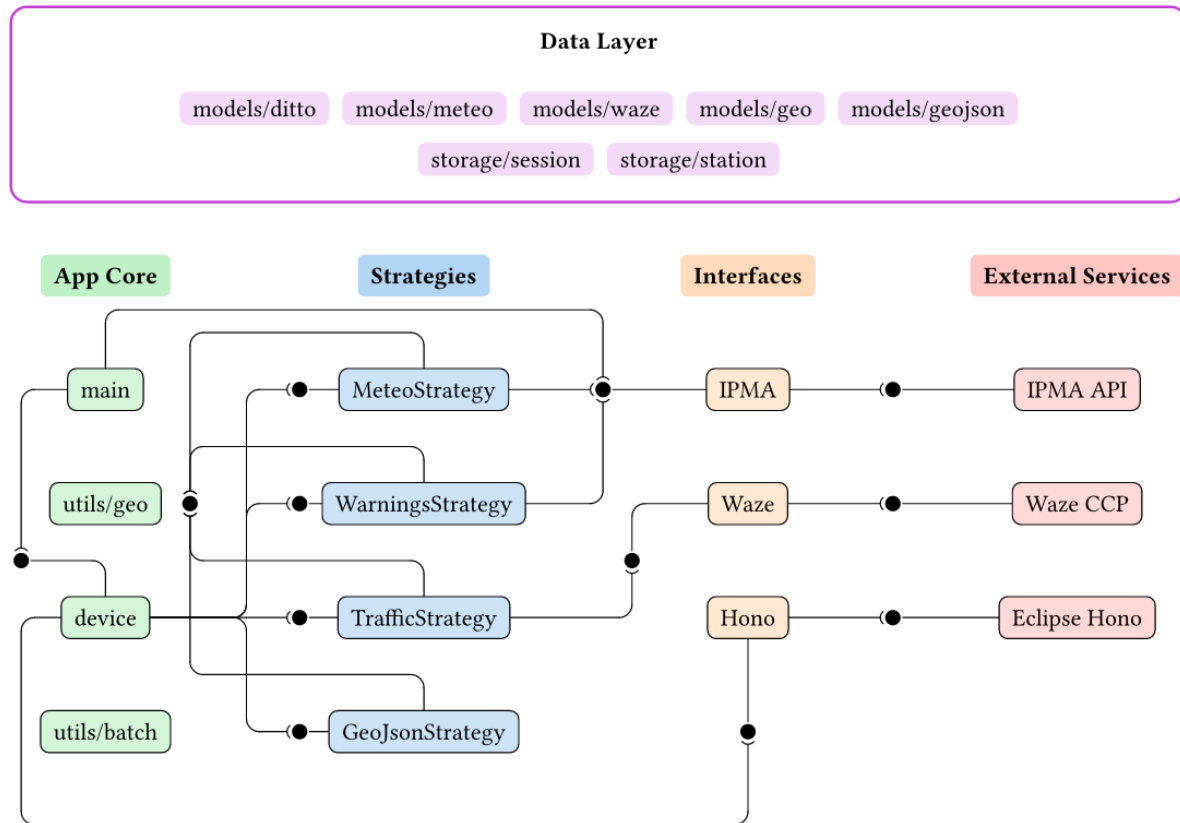


Figura 19: Grafo de dependências e estrutura de código do Data Bridge

NOTA: Para facilitar a compreensão deste diagrama: é utilizada uma notação ball and socket, onde uma bola representa o fornecimento de uma interface e uma socket representa o consumo dessa interface. É utilizada para mostrar as dependências entre componentes internos do sistema (e interação com sistemas externos). A coluna App Core é composta pelos ficheiros main.py, devices/device.py e pelos contidos em utils.py. A coluna Strategies é composta pelos ficheiros dentro do diretório strategies e a coluna Interfaces é composta pelos ficheiros contidos no diretório interfaces. Para facilitar a compreensão, as dependências na camada de dados foram removidas, mas esses ficheiros estão contidos nos diretórios models e storage.

Em main.py são criadas várias instâncias de Device, para cada um dos dispositivos configurados. Esta classe pode ser observada em devices/device.py.

Este Device contém um mecanismo de batch que espera 2 segundos entre o envio de 100 mensagens, para garantir que a instância do Eclipse Hono não seja sobrecarregada com demasiadas mensagens. Este mecanismo de batch é importado de utils/batch.py.

7.3 Singletons

Por questões de cache, ou para ter uma instância comum que necessita de ser partilhada entre

diferentes classes no projeto, como é o caso da partilha de uma `ClientSession` entre as diferentes interfaces, são utilizados Singletons. São classes que contêm internamente uma única instância de uma determinada classe e têm métodos que permitem o acesso a essa classe. No caso do `SessionSingleton` presente em `storage/session.py`, a instância media o acesso a uma única instância de `ClientSession` através do método `get_session()` definido da seguinte forma:

```
@classmethod
def get_session(cls) -> ClientSession:
    if cls.client is None:
        cls.client = ClientSession()

    return cls.client
```

O singleton é responsável por criar a instância única no caso de não existir, bem como por fechar e eliminar essa instância quando já não for necessária, através do método `close_session()`:

```
@classmethod
async def close_session(cls) -> None:
    if cls.client:
        await cls.client.close()
        cls.client = None
```

Adicionalmente, se necessário, o singleton pode também adicionar lógica antes de aceder ao recurso partilhado, como acionar um mutex ou um lock para garantir que não ocorrem condições de corrida em acesso paralelo, como é o caso do `StationSingleton`:

```
@classmethod
async def set_stations(cls, stations: List[Station]) -> None:
    async with cls._lock:
        cls._stations = set(stations)
```

Existem atualmente dois singletons implementados, no diretório `storage/`, sendo eles o `SessionSingleton` já mencionado e o `StationSingleton`.

Com o `SessionSingleton` já descrito anteriormente, o `StationSingleton` medeia o acesso a um conjunto partilhado de objetos `Station` e a um dicionário que liga o ID de uma área de aviso ao objeto `WarningArea`. Permite que outros componentes definam os itens no conjunto ou no dicionário, bem como obter um item do dicionário, obter um determinado objeto `WarningArea` pelo seu ID e também obter as estações mais próximas de um determinado ponto, dado um raio.

7.4 Interfaces

O conceito de interface neste programa não é o de uma interface padrão que define as funções obrigatórias a implementar. É apenas a definição do contacto com o mundo exterior e está, para o bem ou para o mal, fortemente acoplada à `strategy` que a utiliza. Como tal, a definição desta interface é deixada completamente ao programador. No entanto, são deixadas algumas recomendações, nomeadamente a utilização da classe `SessionSingleton` para adquirir uma `ClientSession` do `aiohttp` no caso de ser utilizada uma API REST para a fonte de dados. Espera-se que a interface retorne um Modelo personalizado que tenha sido definido na secção anterior, e que as funções definidas nesta interface sejam apenas chamadas nas respetivas estratégias.

Adicionalmente, foi seguida a diretriz de que todas as funções utilitárias definidas dentro do ficheiro devem ser prefixadas com um underscore (`_`), bem como as funções principais na estratégia devem ser prefixadas com `get`, como em `get_meteorological_data` ou `get_meteorological_warnings`, por exemplo.

Para clareza organizacional, espera-se que estas classes sejam criadas dentro de um novo ficheiro

no diretório `interfaces/`, com um nome que permita fácil reconhecimento do propósito dessa interface.

7.4.1 Hono Interface

A interface Hono é responsável por manter uma `ClientSession` com a instância fornecida do Eclipse Hono e enviar mensagens para o seu HTTP Adapter. As mensagens são enviadas para o endpoint `/telemetry` do HTTP Adapter como um pedido POST, com o conteúdo da mensagem a ser enviado no corpo desse pedido como um objeto JSON.

Adicionalmente, esta interface é também responsável por carregar o contexto SSL/TLS do dispositivo configurado, bem como o certificado x509 do Eclipse Hono, caso seja fornecido através da configuração.

Antes de fazer o pedido, a interface calcula o tamanho total da mensagem a ser enviada e, no caso de ser superior a 4k bytes, a mensagem não é enviada, pois seria maior do que o Hono aceita. No caso de o pedido retornar algum código de erro, a interface registra o erro, mas não tenta enviar o pedido novamente.

7.5 Interface HonoMock

A interface HonoMock é uma interface de depuração que, em vez de enviar o pedido HTTP para o Eclipse Hono, simplesmente registra o objeto JSON que seria enviado.

7.6 Interface IPMA

A interface IPMA é responsável por interagir com a API aberta do IPMA, adquirindo as estações, as suas medições, os avisos que o IPMA fornece e as áreas que afetam. Adicionalmente, também fornece funções para popular o `StationSingleton` com as estações meteorológicas e áreas de aviso adquiridas.

Ao adquirir os avisos meteorológicos, a interface filtra para apenas manter aqueles cujo aviso é “yellow” ou “red”, descartando os que são classificados como “gray” ou “green”.

7.7 Interface Waze

A interface Waze é responsável por interagir com a API Connected Citizens do Waze, adquirindo os Jams e Alertas registrados na plataforma do Waze.

A API CCP do Waze aceita uma bounding box, e não um ponto central e raio como o Data Bridge está configurado. Como tal, a interface, dado o centro e raio configurados, calcula os limites de uma área retangular que circunscreve a área definida. Isto é feito convertendo o raio de pesquisa na sua distância equivalente em graus decimais, que é depois adicionada e subtraída às coordenadas do ponto central. Estas coordenadas recém-calculadas definem 4 linhas, cuja interseção define a bounding box que é enviada para a API.

7.8 Dispositivos

A classe `Device`, definida em `devices/device.py`, contém o ciclo de vida de um dispositivo. A classe fornece um método assíncrono `run` que itera pelas estratégias configuradas no dispositivo e executa o seu método `get_telemetry()`. Depois, para cada `DittoProtocolEnvelope` retornado, utiliza o método `send_telemetry()` da interface `HonoDevice` para enviar a mensagem para o Eclipse Hono. Após executar todo o ciclo de vida, a classe `Device` fecha automaticamente a `ClientSession` instanciada para esse dispositivo.

7.9 Estratégias

Este Data Bridge foi construído com o conceito de facilidade de extensibilidade em mente, e tenta tornar o mais simples possível a criação de novos módulos responsáveis por adquirir dados de diferentes fontes, tais como outras APIs relevantes que possam ser necessárias para adicionar mais informação à representação do mundo do Digital Twin.

Para tal, é utilizado o conceito de Strategies, cujo nome deriva do [Strategy Pattern](#), onde uma família de algoritmos é definida mas intercambiável entre si.

O processo de criação de uma nova fonte de dados passa por criar uma nova extensão da classe BaseStrategy. Além disso, é também de notar o conceito de interface, que neste projeto é considerado o ato de interagir com serviços externos, e de modelo, que é onde os dados obtidos são armazenados. Como exemplo, temos a interface ipma (definida em interfaces/ipma.py), que é responsável por todas as interações feitas com a API do IPMA, e os modelos meteo (definidos em models/meteo.py), que contêm todos os dados relevantes para esta interação, nomeadamente a modelação de dados de estação meteorológica, medição e aviso meteorológico, juntamente com os Enums e constantes necessários.

Outro exemplo é a interface geojson, responsável por interagir com ficheiros GeoJSON no sistema de ficheiros.

Para criar uma nova fonte de dados, espera-se que primeiro sejam definidos os modelos de dados, seguindo-se as interações com o mundo exterior e, por último, a definição da própria classe de estratégia.

Devido à intercambialidade destas classes, todas estendem a mesma classe base e têm a mesma “interface” no sentido de programação padrão, onde cada subclasse deve implementar a função assíncrona `get_telemetry(self) -> List[DittoProtocolEnvelope]`.

A definição desta classe BaseStrategy pode ser vista no ficheiro [strategies/strategy.py](#). A classe deve conter obrigatoriamente os seguintes campos:

- namespace
- subject
- policyId

Que devem ser definidos no construtor da classe (função `__init__`). Adicionalmente, a classe base contém as seguintes funções utilitárias:

```
def _create_topic(
    self,
    thingName: str,
    channel: Channel = Channel.TWIN,
    criterion: Criterion = Criterion.COMMAND,
    action: Action | None = CommandAction.MODIFY,
) -> Topic:

def _create_envelope(
    self,
    topic: Topic,
    attributes: dict[str, object] | None = None,
    features: Dict[str, Feature] | None = None,
    path: str = "/",
) -> DittoProtocolEnvelope:
```

```
def _create_envelope_raw(
    self,
    message_topic: Topic,
    value: Any = None,
    path: str = "/",
) -> DittoProtocolEnvelope:
```

Que auxiliam na criação do DittoProtocolEnvelope.

Por último, após a criação da nova classe Strategy, o ficheiro [strategies/ _init_.py](#) deve ser alterado para permitir a configuração desta nova estratégia a partir do ficheiro de configuração config.toml.

Para tal, deve ser criada uma nova classe prefixada com underscore que define os campos necessários para a estratégia. Além disso, esta classe DEVE estender a classe _BaseType definida nesse ficheiro e DEVE ser adicionada à união de tipos StrategyType. É obrigatório que esta classe recém-criada contenha o campo type, que deve ser definido como uma string literal.

Como exemplo da criação de tal classe, temos a classe _GeoJSON, que permite a definição da GeoJsonStrategy no ficheiro de configuração:

```
class _GeoJson(_BaseType):
    type: Literal["geojson"] = "geojson"
    dir: str | None = None
    file: str | None = None

    @model_validator(mode="after")
    def validate_fields(self) -> Self:
        if (self.dir is None) == (self.file is None):
            raise ValueError(
                "A GeoJson strategy must either have files or directories, no
t both"
            )
        return self

StrategyType = Annotated[
    Union[_Meteo, _Traffic, _GeoJson, _MeteoWarnings], Field(discriminator="t
ype")
]
```

Por último, a instanciação da estratégia DEVE ser adicionada à função _type_to_strategy(), também definida no mesmo ficheiro. Para tal, uma nova entrada no bloco match, correspondente ao tipo da classe, tem de ser adicionada, que retorna a instanciação do objeto.

Como exemplo:

```
def _type_to_strategy(
    type: StrategyType,
    policyId: str,
    subject: str,
) -> BaseStrategy:
    match type:
        (...)
        case _GeoJson():
            return GeoJsonStrategy(
                subject, type.namespace, policyId, type.dir, type.file
            )
```


Desta forma, a configuração da estratégia no ficheiro de configuração é automaticamente tratada pelo Data Bridge, não sendo necessárias mais alterações.

A seguir, é especificada a implementação concreta de todas as estratégias.

7.10 Estratégia Meteorológica

A estratégia meteorológica é responsável por consultar a API aberta do IPMA e obter as medições realizadas nas suas estações meteorológicas.

O fluxo de execução desta estratégia é o seguinte:

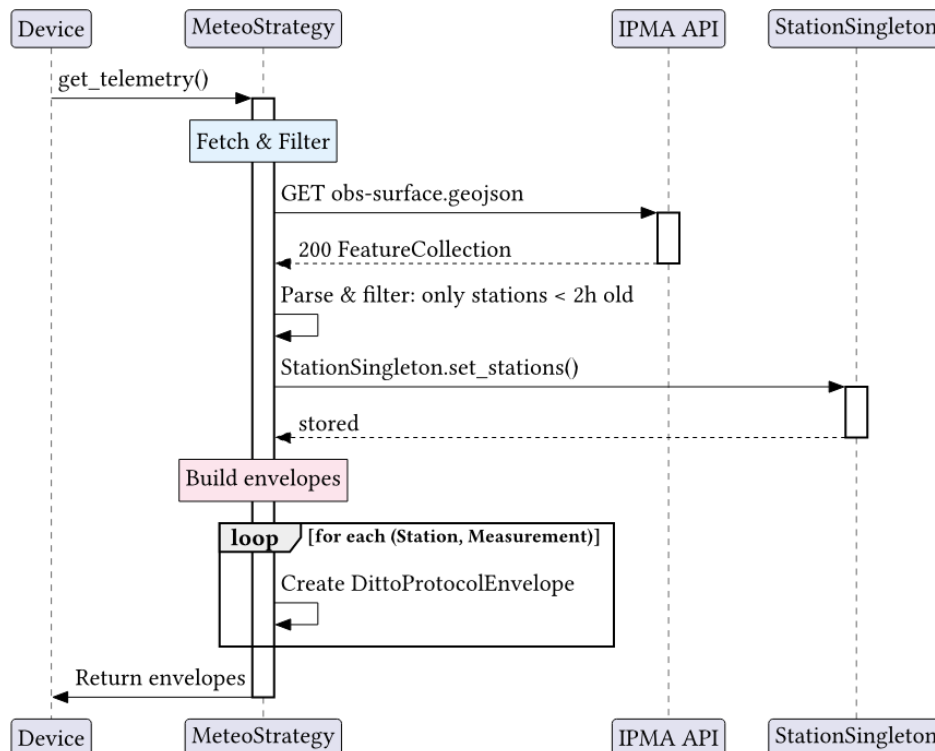


Figura 20: Diagrama de sequência da Estratégia Meteorológica

Conforme definido pela *BaseStrategy*, esta classe tem um método `get_telemetry` que, quando chamado, fará um pedido GET à API aberta do IPMA e obterá um ficheiro GeoJSON que contém não só as medições de todas as estações online, mas também alguns atributos sobre essas estações, nomeadamente a localização e a cidade onde se encontram.

A partir destes dados adquiridos, as estações são filtradas de modo a apenas manter as estações com medições recentes (aquelas realizadas há menos de 2 horas). Adicionalmente, o **StationSingleton**, que atua como cache para as localizações das estações, é atualizado.

De seguida, para cada estação contida no ficheiro GeoJSON obtido, é criada uma instância de **DittoProtocolEnvelope** e adicionada a um array, que será depois retornado ao ciclo de execução principal para uma instância de **Device** enviar para o Eclipse Hono.

Como é comum noutras implementações de Digital Twin, os campos `geotile` e `expiry_ts` são adicionados ao **Thing**, auxiliando na pesquisa de Things numa determinada área geográfica, bem como permitindo que o sistema de garbage collector elimine Things não utilizados e desatualizados.

7.11 Estratégia de Avisos Meteorológicos

A estratégia de avisos meteorológicos é responsável por consultar a API aberta do IPMA e obter os avisos que foram atribuídos a cada região do país.

O fluxo de execução desta estratégia é o seguinte:

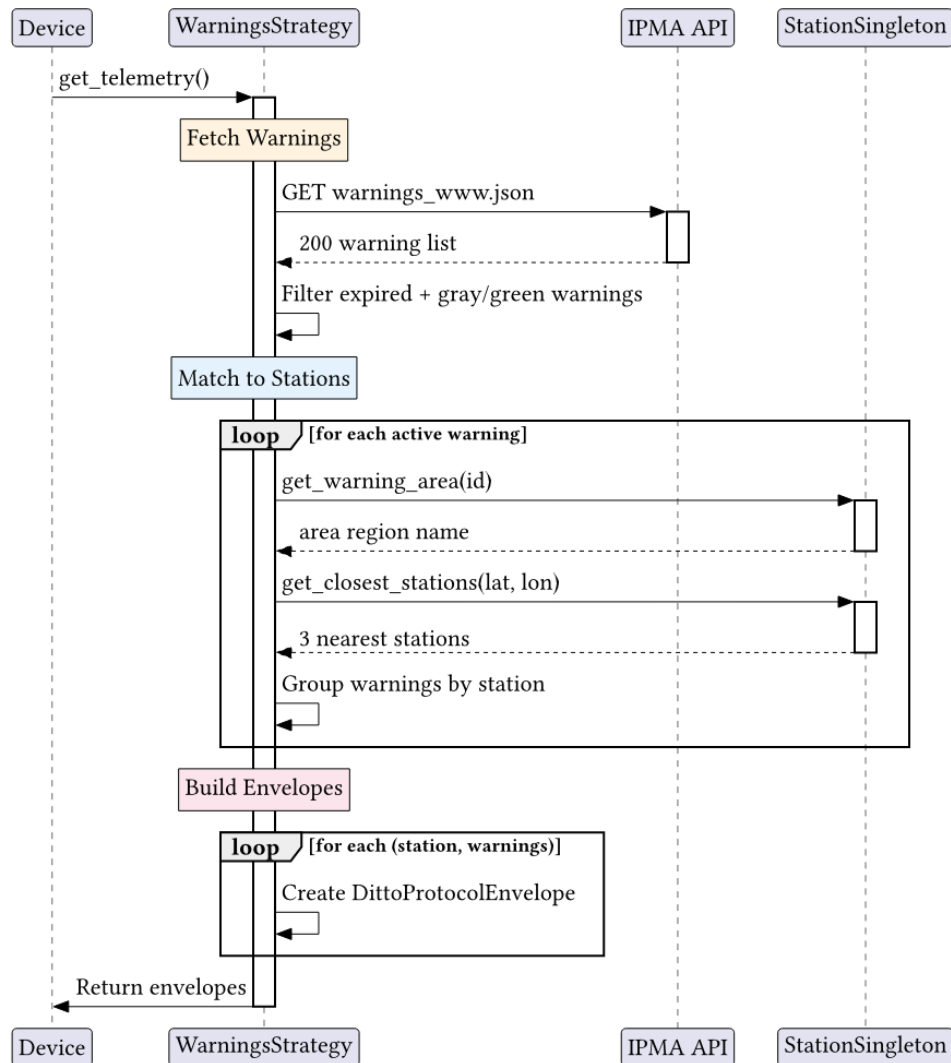


Figura 21: Diagrama de sequência da Estratégia de Avisos

Atualmente, esta estratégia assume que o **StationSingleton** já foi populado com as estratégias meteorológicas e áreas de aviso existentes, o que está a ser feito no início do programa. A partir daqui, para cada aviso ativo, a área de aviso é obtida do singleton, e a sua localização é utilizada para calcular as 3 estações mais próximas, sendo os avisos agrupados por estas estações.

De seguida, para cada estação, é instanciado um comando `modify DittoProtocolEnvelope` e adicionado a um array, que é depois retornado para o fluxo de execução principal para uma instância de **Device** enviar para o Eclipse Hono.

Como é comum noutras implementações de Digital Twin, os campos `geotile` e `expiry_ts` são adicionados ao Thing, auxiliando na pesquisa de Things numa determinada área geográfica, bem como permitindo que o sistema de garbage collector elimine Things não utilizados e desatualizados.

7.12 Estratégia de Tráfego

A estratégia de tráfego é responsável por consultar a API Connected Citizens Program do Waze e obter os alerts e jams atuais registados no Waze.

O fluxo de execução desta estratégia é o seguinte:

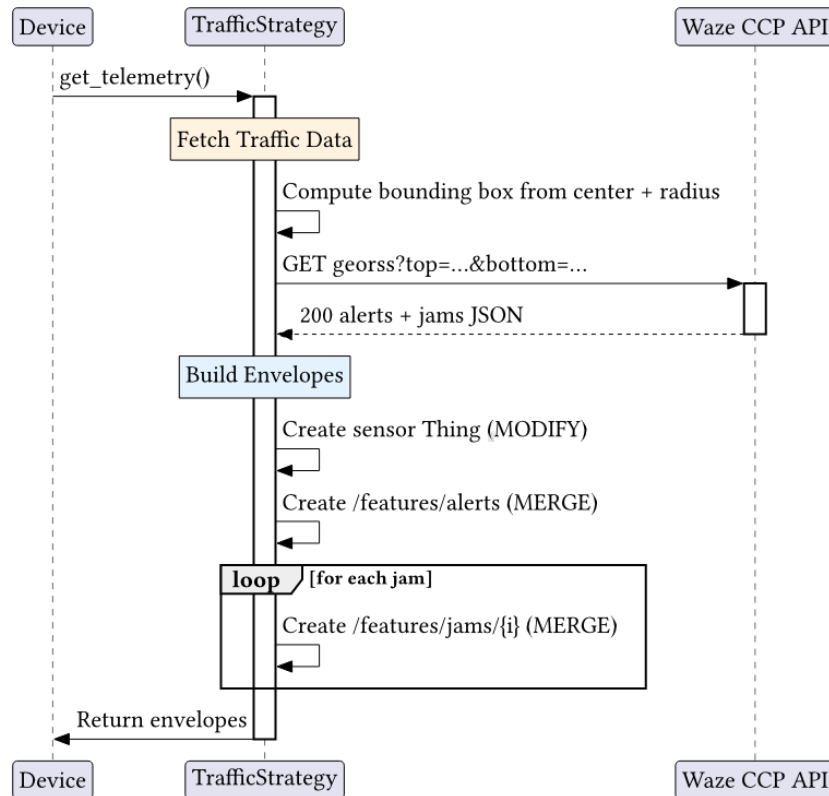


Figura 22: Diagrama de sequência da Estratégia de Tráfego

Após adquirir os dados da interface, procede-se primeiro à criação de um `DittoProtocolEnvelope` para criar ou modificar um Thing, seguido da criação de uma mensagem para modificar a feature alerts nesse Thing e, por último, iterando por todos os jams e criando uma mensagem separada para modificar a feature jams. Isto é feito desta forma porque, no caso dos jams, devido ao campo de geometria do jam, a mensagem pode tornar-se bastante grande e ultrapassar o limite de tamanho do Eclipse Hono. Fazer uma mensagem separada por jam garante que a mensagem não ultrapassa o limite e, se ultrapassar, apenas um único Jam é perdido, e não toda a atualização do Thing.

Todos os envelopes produzidos são adicionados a um array, que é depois retornado ao fluxo de execução principal para uma instância de Device enviar para o Eclipse Hono.

Como é comum noutras implementações de Digital Twin, os campos `geotile` e `expiry_ts` são adicionados ao Thing, auxiliando na pesquisa de Things numa determinada área geográfica, bem como permitindo que o sistema de garbage collector elimine Things não utilizados e desatualizados.

7.13 Estratégia GeoJSON

Esta estratégia difere das restantes porque a fonte de dados utilizada provém do sistema de ficheiros local, em vez de ser obtida através de um pedido HTTP. A esta estratégia é fornecido um

ficheiro ou diretório, e esta lerá o(s) ficheiro(s) e, a partir daí, extrairá as features contidas no ficheiro. O fluxo de execução desta estratégia é o seguinte:

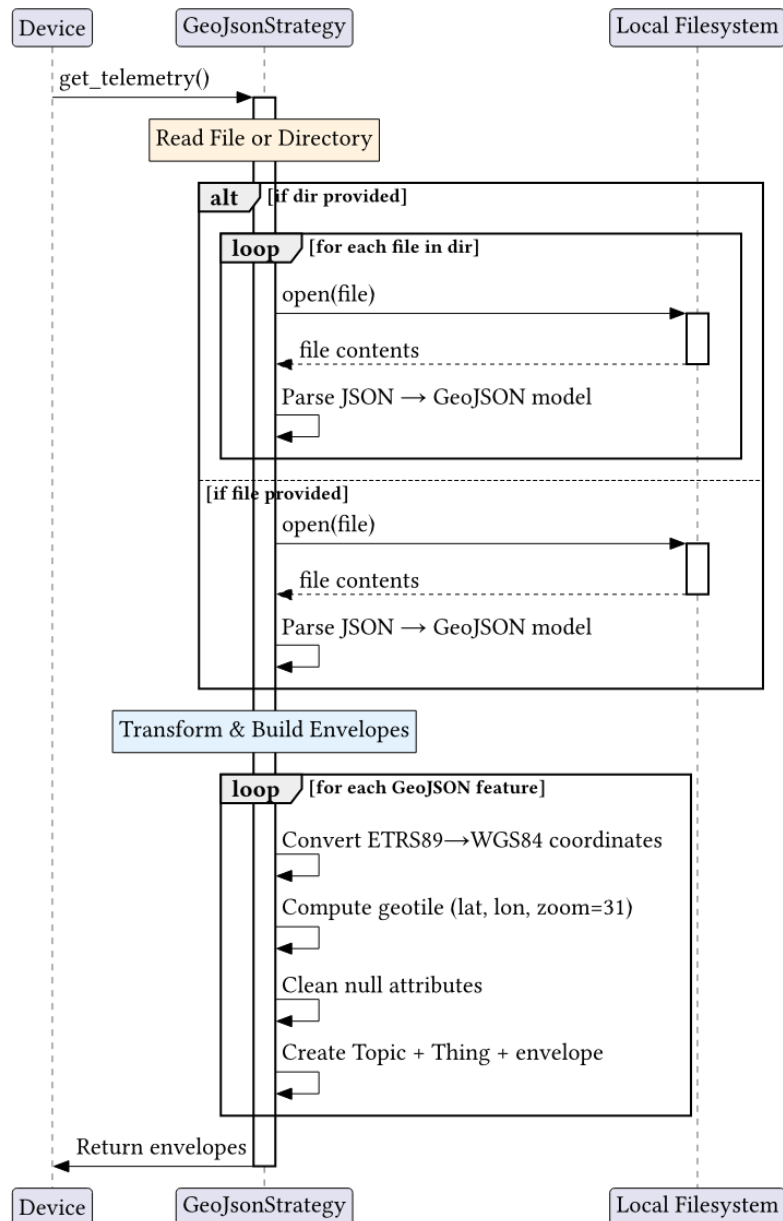


Figura 23: Diagrama de sequência da Estratégia GeoJSON

O campo name do GeoJSON é utilizado para nomear os Things que serão criados/atualizados. Do array features, os campos properties e geometry são utilizados para preencher os atributos do Thing.

Esta estratégia foi criada com a intenção de lidar com ficheiros GeoJSON cujas coordenadas estão na projeção ETRS89, significando que também converte essas coordenadas para WGS84, pois essa é a projeção que está a ser utilizada atualmente pelo sistema.

Como é comum noutras implementações de Digital Twin, os campos geotile e expiry_ts são adicionados ao Thing, auxiliando na pesquisa de Things numa determinada área geográfica, bem como permitindo que o sistema de garbage collector elimine Things não utilizados e desatualizados.

8 METEO POPULATOR

8.1 Visão Geral

O Meteo Populator é um job batch Python que popula Ditto Things com informação da estação meteorológica mais próxima. Obtém estações ativas da API do IPMA e encontra as 3 estações mais próximas para cada Ditto Thing utilizando a distância de Haversine.

O programa principal, conforme implementado atualmente, funciona da seguinte forma:

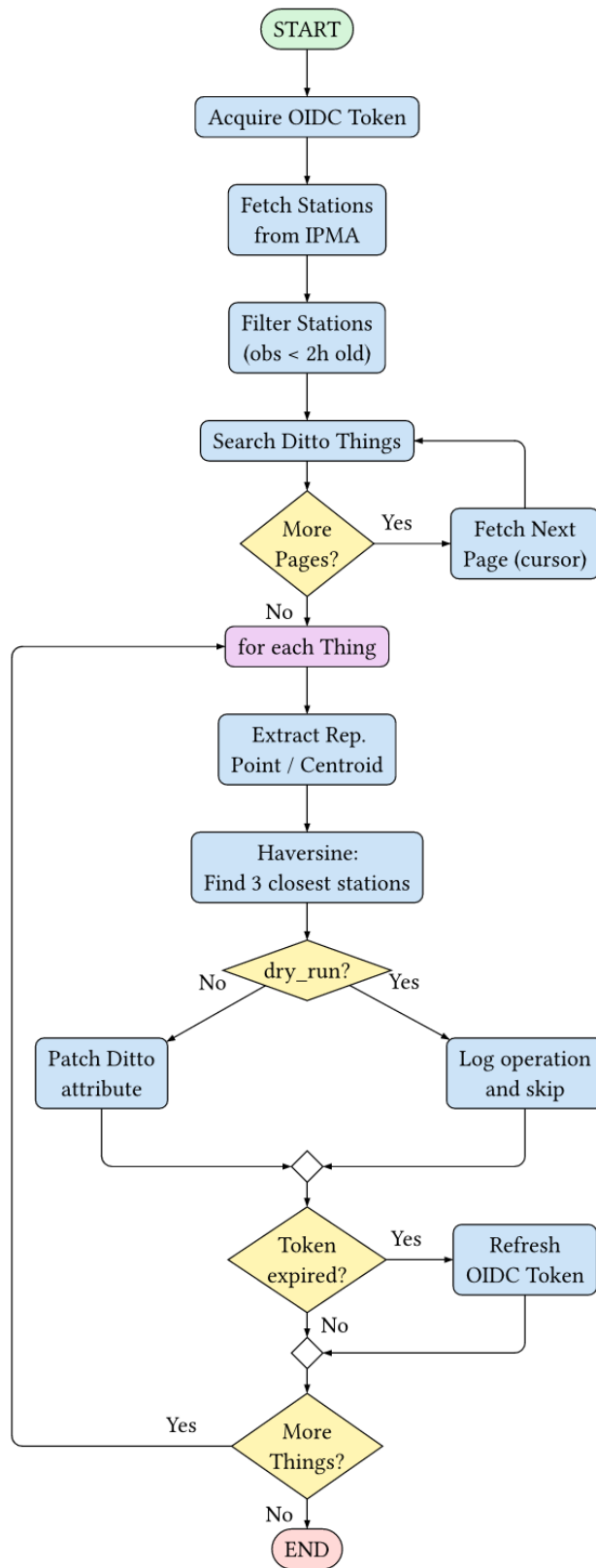


Figura 24: Grafo de fluxo de controlo do Meteo Populator

O programa começa por realizar autenticação OIDC utilizando um password grant para obter um access token e um refresh token da instância Keycloak da infraestrutura. É então criada uma `aiohttp.ClientSession` partilhada, apontada para o endpoint da API v2 do Ditto com o access token definido como cabeçalho `Authorization bearer`.

O primeiro passo de obtenção de dados obtém todas as estações meteorológicas ativas da API aberta do IPMA (`api.ipma.pt`). As estações cuja última medição é anterior a 2 horas são descartadas como desatualizadas.

O segundo passo utiliza paginação baseada em cursor para iterar por todos os Things do Eclipse Ditto que correspondem a um filtro RQL configurável. O filtro padrão obtém todos os Things que contêm um atributo `location`, `geometry` ou `coordinates` e cujo `ThingId` não contém a palavra `meteo`, para que os Things de estação meteorológica não sejam modificados.

Para cada Thing retornado, o programa extrai um único ponto geográfico representativo do atributo `location` do Thing — que pode ser uma única coordenada, uma lista de coordenadas (cujo centróide é calculado) ou ausente (o que faz com que o Thing seja ignorado). De seguida, pesquisa as estações IPMA ativas para encontrar as 3 estações mais próximas dentro de 100 km desse ponto, utilizando a fórmula de Haversine para distância de grande círculo. Se forem encontradas estações, é emitido um pedido PUT ao Ditto para definir o atributo `closest_meteo_stations` no Thing com os identificadores das estações formatados como `namespace:subject:<station_id>`.

O token é verificado quanto à expiração antes de cada Thing ser processado; se expirado, é realizado um refresh token grant de forma transparente.

O diagrama de sequência do fluxo de execução principal é o seguinte:

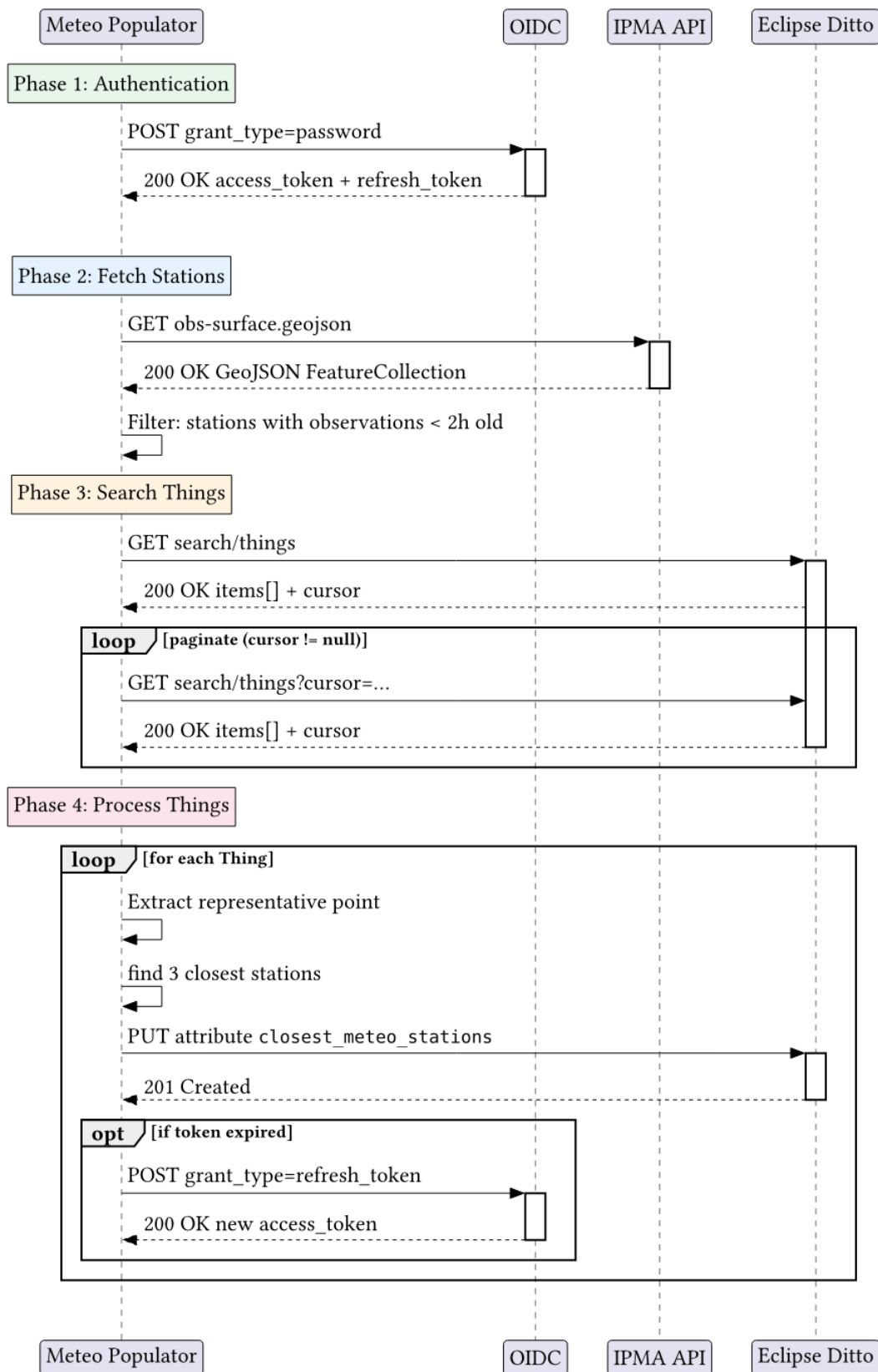


Figura 25: Diagrama de sequência do Meteo Populator

O programa executa uma única execução e termina. Destina-se a ser executado periodicamente como um Kubernetes CronJob ou uma tarefa cron do sistema.

8.2 Estrutura do Código

O programa está estruturado logicamente da seguinte forma:

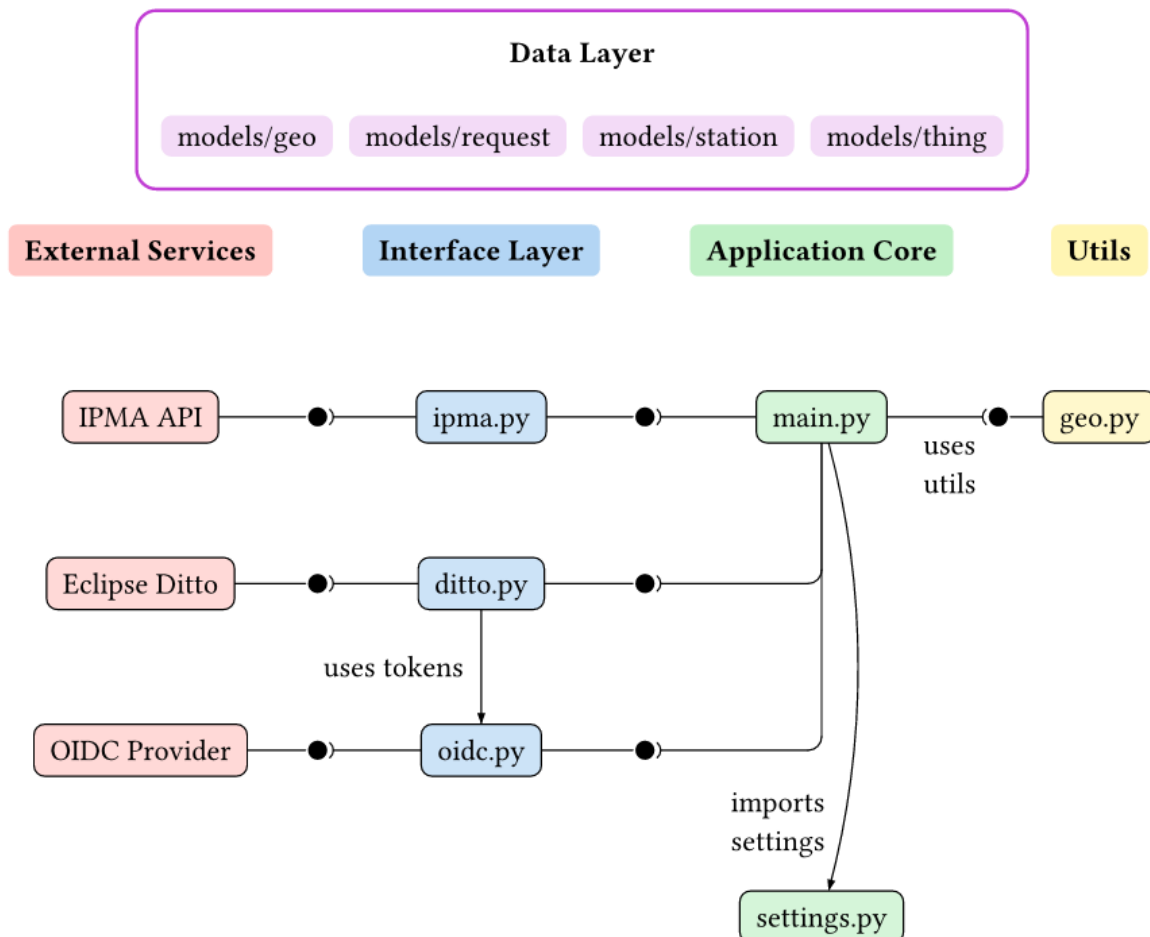


Figura 26: Grafo de dependências e estrutura de código do Meteo Populator

NOTA: Para facilitar a compreensão deste diagrama: é utilizada uma notação ball and socket, onde uma bola representa o fornecimento de uma interface e uma socket representa o consumo dessa interface. É utilizada para mostrar as dependências entre componentes internos do sistema (e interação com sistemas externos). A coluna App Core é composta pelos ficheiros main.py e settings.py. A coluna Interfaces é composta pelos ficheiros contidos no diretório interfaces/ e a coluna Data é composta pelos ficheiros contidos no diretório models/. A coluna Utils é composta pelo diretório utils/.

O ponto de entrada do programa é main.py, que importa a instância partilhada settings de settings.py e orquestra todo o ciclo de vida. O diretório interfaces/ contém módulos que cada um trata da comunicação com um serviço externo específico:

- interfaces/oidc.py — Autenticação com Keycloak

- `interfaces/ipma.py` — Obtenção de estações meteorológicas do IPMA
- `interfaces/ditto.py` — Pesquisa e modificação de Things no Eclipse Ditto

O diretório `models/` contém modelos de dados Pydantic utilizados em toda a base de código:

- `models/geo.py` — Representação de ponto geográfico
- `models/station.py` — Dados de estação meteorológica e cálculos de distância
- `models/Thing.py` — Ditto Thing e tipos relacionados
- `models/request.py` — Resposta de pesquisa do Ditto com cursor de paginação

O diretório `utils/` contém funções utilitárias puras para cálculos geográficos utilizados pelas interfaces:

- `utils/geo.py` — Extração de ponto representativo e pesquisa de estação mais próxima

8.3 Modelos de Dados

Os modelos de dados neste projeto são todos criados utilizando Pydantic's `BaseModel`, com Enums a serem criados com a classe `Enum` da biblioteca padrão do Python.

Para saber mais sobre como criar um modelo Pydantic, é recomendada a consulta da sua [documentação oficial](#). No entanto, os conceitos importantes são que uma nova classe tem de ser criada que estende a classe `BaseModel`, e os campos são definidos dentro desta nova classe, juntamente com os seus tipos. O Pydantic é então responsável pela serialização e desserialização do modelo. Validadores personalizados podem ser criados com o decorador de função `@model_validator`.

Para manter a organização, espera-se que estes modelos sejam criados dentro de um novo ficheiro no diretório `models/`, com um nome que permita fácil reconhecimento do propósito dos modelos.

Os seguintes modelos estão definidos neste projeto:

- `models/geo.py` — `Point`: uma coordenada geográfica simples com campos `latitude` e `longitude`. Este modelo é utilizado em toda a base de código sempre que é necessário representar uma localização geográfica.
- `models/station.py` — `Station`: representa uma estação meteorológica IPMA com campos `id`, `name`, `latitude` e `longitude`. Contém um método `distance_to(point)` que retorna a distância de grande círculo em quilómetros até um determinado `Point`, calculada pela função interna `_haversine`. A fórmula de Haversine aqui implementada é:

```
def _haversine(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    R = 6371.0
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlamba = math.radians(lon2 - lon1)
    a = (
        math.sin(dphi / 2) ** 2
        + math.cos(phi1) * math.cos(phi2) * math.sin(dlamba / 2) ** 2
    )
    return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
```

- `models/Thing.py` — `Thing`: representa um Ditto Thing com campos `thingId` e `attributes`. O modelo `Attributes` utiliza `AliasChoices` para aceitar campos `location`, `geometry` ou `coordinates` da resposta JSON da API, mapeando-os todos para o mesmo atributo `location`.

O campo `location` pode ser um único `Point`, um `list[Point]` (para geometrias de polígono) ou `None`. Este ficheiro também define `PopulateResult`, um `enum` de `string` com valores `SUCCESS`, `NO_STATIONS`, `NO_LOCATION` e `ERR`, utilizado para o `dispatch match/case` no ciclo principal.

- `models/request.py` — `SearchResponse`: modela a resposta paginada do endpoint `search/Things` do Ditto, contendo `items: list[Thing]` e um `cursor` opcional para a página seguinte.
- `interfaces/oidc.py` — `Tokens`: um modelo com campos `access` (a string do access token), `refresh` (a string do refresh token) e `expiry` (um `datetime` que indica quando o access token expira). Este modelo está definido dentro do módulo da interface em vez do diretório `models/` porque está fortemente acoplado à lógica de autenticação.

8.4 Configuração

A configuração é tratada através de `settings.py`, que utiliza `pydantic-settings` para carregar definições de um ficheiro `config.toml`. A classe `Settings` é uma subclasse de `BaseSettings` com as seguintes secções aninhadas:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(toml_file="config.toml")

    populator: PopulatorSettings = PopulatorSettings()
    logging: LoggingSettings = LoggingSettings()
    oidc: OidcSettings = OidcSettings()
    station: StationSettings = StationSettings()
    filter: DittoFilter = DittoFilter()
```

Um singleton ao nível do módulo é criado no momento da importação:

```
settings = Settings()
```

Este singleton é importado em toda a base de código como `from settings import settings`. Todos os componentes acedem à configuração através desta única instância, evitando a necessidade de passar objetos de configuração através de parâmetros de função.

As definições também podem ser fornecidas através de variáveis de ambiente utilizando um separador de underscore duplo (`__`) para campos aninhados. Por exemplo, o URL base do OIDC pode ser definido com `OIDC__BASE_URL`.

O método de classe `settings_customise_sources` é substituído para utilizar exclusivamente uma fonte de ficheiro TOML, ignorando variáveis de ambiente, ficheiros `.env` e outras fontes padrão que o `pydantic-settings` fornece.

O ficheiro também configura o logger `loguru` com um formatador personalizado que renderiza dicionários extra estruturados como texto magenta indentado abaixo da mensagem de log, melhorando a legibilidade da saída de depuração.

8.5 Interfaces

O conceito de interface neste programa não é o de uma interface padrão que define as funções obrigatórias a implementar. É apenas a definição do contacto com o mundo exterior. Cada interface é um módulo Python que contém funções assíncronas que utilizam `aiohttp` para comunicar com um serviço externo específico. Como tal, a definição desta interface é deixada completamente ao programador. No entanto, são deixadas algumas recomendações, nomeadamente a utilização da

`ClientSession` do `aiohttp` para todos os pedidos HTTP e a garantia de que as funções retornam modelos Pydantic tipados.

Adicionalmente, foi seguida a diretriz de que todas as funções utilitárias definidas dentro do ficheiro devem ser prefixadas com um underscore (`_`).

8.5.1 OIDC Interface

A interface OIDC, definida em `interfaces/oidc.py`, é responsável por interagir com a instância Keycloak da infraestrutura para obter e renovar tokens OIDC.

São fornecidas duas funções:

- `get_tokens()` — Utiliza o tipo de concessão password grant (`grant_type=password`) para autenticar com o nome de utilizador, palavra-passe, client ID, realm e scope configurados. Retorna um modelo `Tokens` contendo o access token, refresh token e um datetime de expiração calculado. Cada chamada cria a sua própria `ClientSession` de curta duração. Retorna `None` em caso de erro HTTP.
- `refresh_token(refresh_token)` — Utiliza a concessão refresh token para obter um novo access token antes de o atual expirar. Segue o mesmo padrão que `get_tokens()` mas utiliza o refresh token fornecido em vez de credenciais.

Ambas as funções fazem POST para o endpoint `{base_url}/auth/realms/{realm}/protocol/openid-connect/token` com dados de formulário codificados em URL.

8.5.2 IPMA Interface

A interface IPMA, definida em `interfaces/ipma.py`, é responsável por obter estações meteorológicas ativas da API pública do IPMA.

Fornece uma única função:

- `fetch_active_stations()` — Faz um pedido GET para `https://api.ipma.pt/open-data/observation/meteorology/stations/obs-surface.geojson` e analisa o GeoJSON retornado. As estações são filtradas para manter apenas aquelas cuja última medição foi realizada há menos de 2 horas. Para cada estação ativa, é criado um objeto `Station` com o ID da estação (`idEstacao`), nome (`localEstacao`) e coordenadas geográficas (extraídas de `geometry.coordinates` na ordem `[lon, lat]`, convertidas para `lat, lon`).

A função cria a sua própria `ClientSession`, pois a API do IPMA é pública e não requer autenticação.

8.5.3 Ditto Interface

A interface Ditto, definida em `interfaces/ditto.py`, é responsável por pesquisar e modificar Things no Eclipse Ditto. Ao contrário das outras interfaces, recebe uma `ClientSession` pré-autenticada de `main.py` (que já tem o cabeçalho bearer token definido).

São fornecidas três funções:

- `fetch_all_Things(session)` — Realiza paginação baseada em cursor no endpoint `search/Things` do Ditto. Cada pedido obtém 200 Things de cada vez, passando o cursor da

resposta anterior como parâmetro opcional. O filtro e os campos são lidos do singleton de definições compartilhado. A paginação pára quando o servidor retorna um cursor nulo.

- `patch_closest_stations(session, Thing_id, stations)` — Envia um pedido PUT para `Things/{Thing_id}/attributes/closest_meteo_stations` com um corpo JSON contendo uma lista de `station ThingIds` formatados como `namespace:subject:`. Em modo `dry-run`, o pedido é ignorado e apenas registrado. A verificação SSL está desativada (`ssl=False`).
- `populate_closest_stations(session, Thing, stations)` — Orquestra a lógica de população por Thing. Chama `representative_point()` de `utils/geo.py` para extrair um único ponto geográfico dos atributos do Thing, depois chama `closest_stations()` para encontrar as estações mais próximas. Se for bem-sucedido, delega em `patch_closest_stations()`. Retorna um valor do enum `PopulateResult` para o ciclo principal registrar.

8.6 Utilitários

O módulo utilitário, `utils/geo.py`, contém funções puras para cálculos geográficos que são utilizadas pela interface Ditto. Estas funções não têm efeitos secundários e não dependem de serviços externos.

São fornecidas duas funções:

- `representative_point(location)` — Aceita um `Point`, um `list[Point]` ou `None`. Se for uma lista de pontos (por exemplo, vértices de polígono), o centróide geométrico é calculado como a média das latitudes e a média das longitudes. Se for um único `Point`, é retornado como está. Se for `None`, é retornado `None`, indicando que o Thing não tem dados de localização utilizáveis.
- `closest_stations(point, stations, max_distance=100.0, n=3)` — Dado um `Point` e uma lista de objetos `Station`, ordena todas as estações pela sua distância de grande círculo ao ponto (utilizando `Station.distance_to()` que implementa a fórmula de Haversine). Retorna as top `n` estações que estão dentro de `max_distance` quilómetros. A configuração padrão retorna até 3 estações dentro de 100 km.

9 NAMESPACE REMOVER

9.1 Visão Geral

O Namespace Remover é um job batch Python que elimina em massa Ditto Things por namespace ou filtro RQL personalizado. Utiliza paginação baseada em cursor para iterar por todos os Things correspondentes e emite pedidos DELETE. Foi concebido para ser executado como uma tarefa de limpeza periódica.

O programa principal, conforme implementado atualmente, funciona da seguinte forma:

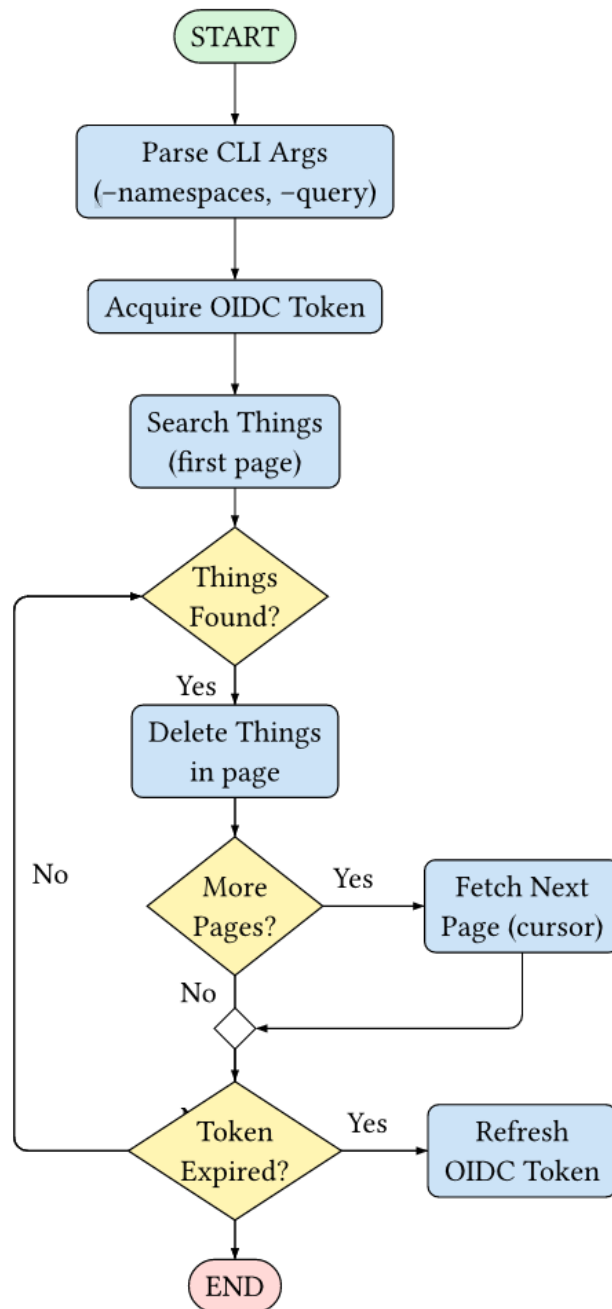


Figura 27: Grafo de fluxo de controlo do Namespace Remover

O programa começa por realizar autenticação OIDC utilizando um password grant para obter um access token e um refresh token da instância Keycloak da infraestrutura. É então criada uma `aiohttp.ClientSession` partilhada, apontada para o endpoint da API v2 do Ditto com o access token definido como cabeçalho `Authorization bearer`.

O primeiro passo de obtenção de dados obtém Things do endpoint `search/Things` do Eclipse Ditto, opcionalmente filtrados por namespace e/ou uma consulta RQL personalizada. A resposta contém uma lista de valores `thingId` e, se existirem mais resultados, uma string `cursor` para paginação.

Para cada lote de Things retornados, o programa emite um pedido `DELETE` para cada `thingId` contra a API do Ditto. Após o lote ser eliminado, o cursor é verificado: se presente, a página seguinte

é obtida e o processo repete-se. Se ausente, o ciclo termina.

O token é verificado quanto à expiração antes de cada lote paginado; se expirado, é realizado um refresh token grant de forma transparente.

O diagrama de sequência do fluxo de execução principal é o seguinte:

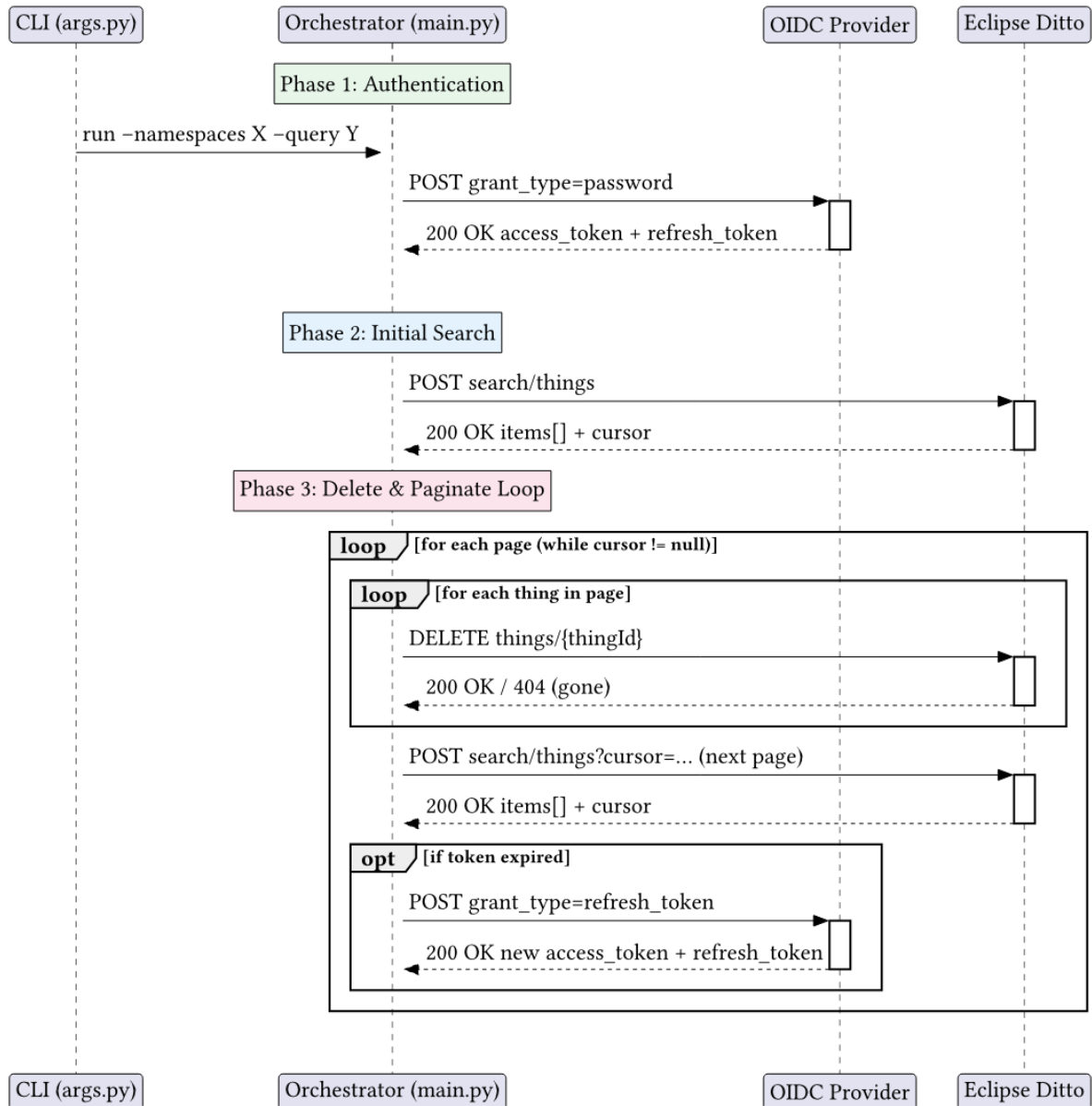


Figura 28: Diagrama de sequência do Namespace Remover

O programa executa uma única execução e termina. Destina-se a ser executado periodicamente como uma tarefa cron do sistema ou um Kubernetes CronJob.

9.2 Estrutura do Código

O programa está estruturado logicamente da seguinte forma:

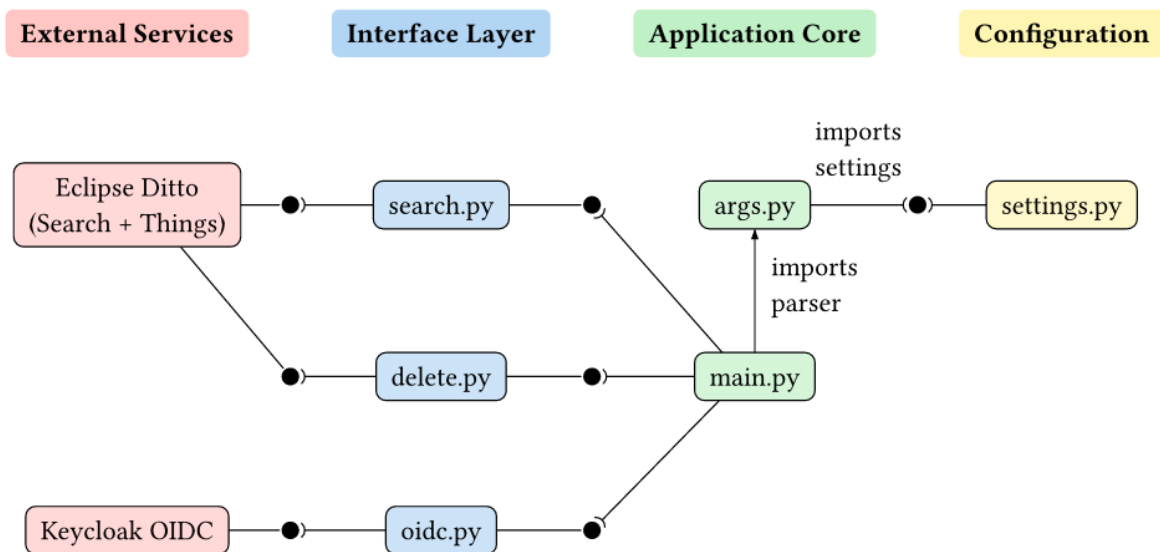


Figura 29: Grafo de dependências e estrutura de código do Namespace Remover

NOTA: Para facilitar a compreensão deste diagrama: é utilizada uma notação ball and socket, onde uma bola representa o fornecimento de uma interface e uma socket representa o consumo dessa interface. É utilizada para mostrar as dependências entre componentes internos do sistema (e interação com sistemas externos).

O ponto de entrada do programa é `main.py`, que importa a instância partilhada `settings` de `settings.py` e o parser de `args.py`, e depois orquestra todo o ciclo de vida. Os módulos de interface tratam cada um da comunicação com um serviço externo específico:

- `oidc.py` — Autenticação com Keycloak
- `search.py` — Pesquisa de Things no Eclipse Ditto
- `delete.py` — Eliminação de Things no Eclipse Ditto

Ao contrário de um projeto maior, não existe um diretório separado `models/` ou `utils/`. Os modelos de dados são definidos diretamente dentro do módulo de interface que os utiliza (por exemplo, `Tokens` em `oidc.py`, `SearchResponse` em `search.py`).

9.3 Modelos de Dados

Os modelos de dados neste projeto são todos criados utilizando `pydantic's BaseModel`. Para melhor organização, cada modelo é definido no módulo que possui a sua lógica, em vez de num diretório `models/` separado.

Os seguintes modelos estão definidos:

- `Tokens` (definido em `oidc.py`) — Representa o par de tokens OIDC retornado pelo Keycloak. Campos:
 - `access: str` — O access token (JWT) utilizado em pedidos à API.
 - `refresh: str` — O refresh token utilizado para obter um novo access token quando o atual expira.
 - `expiry: datetime` — Um datetime UTC calculado que indica quando o access

token expira, derivado da declaração `expires_in` retornada pelo Keycloak.

- `SearchResponse` (definido em `search.py`) — Representa uma resposta paginada do endpoint `search/Things` do Ditto. Campos:
 - `Things: list[str]` — Uma lista de strings `thingId` retornadas pela pesquisa.
 - `cursor: str | None` — Uma string `cursor` opcional para obter a página seguinte de resultados. Quando `None`, todos os resultados foram consumidos.

9.4 Configuração

A configuração é tratada através de `settings.py`, que utiliza `pydantic-settings` para carregar definições de um ficheiro `config.toml` e/ou variáveis de ambiente. A classe `Settings` é uma subclasse de `BaseSettings` com a seguinte estrutura:

```
class OidcSettings(BaseSettings):
    realm: str = ""
    username: str = ""
    password: str = ""
    client_id: str = ""
    scope: str = "openid"

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        toml_file="config.toml",
        env_nested_delimiter="__",
        nested_model_default_partial_update=True,
    )

    base_url: str = ""
    oidc: OidcSettings = OidcSettings()
```

Um singleton ao nível do módulo é criado no momento da importação:

```
settings = Settings()
```

Este singleton é importado em toda a base de código como `from settings import settings`. Todos os componentes acedem à configuração através desta única instância, evitando a necessidade de passar objetos de configuração através de parâmetros de função.

As definições também podem ser fornecidas através de variáveis de ambiente utilizando um separador de underscore duplo (`__`) para campos aninhados:

Variável de ambiente	Campo correspondente
<code>BASE_URL</code>	<code>base_url</code>
<code>OIDC__REALM</code>	<code>oidc.realm</code>
<code>OIDC__USERNAME</code>	<code>oidc.username</code>
<code>OIDC__PASSWORD</code>	<code>oidc.password</code>

Variável de ambiente	Campo correspondente
OIDC__CLIENT_ID	oidc.client_id
OIDC__SCOPE	oidc.scope

Tabela 31 - Variáveis de ambiente para configuração do Namespace Remover

O método de classe `settings_customise_sources` é substituído para utilizar variáveis de ambiente e uma fonte de ficheiro TOML (nessa ordem), permitindo que as variáveis de ambiente substituam valores em `config.toml`:

```
@classmethod
def settings_customise_sources(cls, ...):
    return (env_settings, TomlConfigSettingsSource(settings_cls))
```

O ficheiro `config.toml` deve ser colocado no diretório raiz do projeto (o diretório onde `main.py` está localizado).

9.5 Argumentos de CLI

A análise de argumentos de linha de comandos é tratada em `args.py` utilizando o `argparse.ArgumentParser` padrão do Python:

```
parser = ArgumentParser(
    prog="Bulk Things Eraser",
    description="...",
)

parser.add_argument("--namespaces", nargs="*", ...)
parser.add_argument("--query", type=str, ...)
```

São definidos dois argumentos opcionais:

Argumento	Tipo	Descrição
<code>--namespaces</code>	<code>nargs="*" (zero ou mais strings)</code>	Um ou mais namespaces para filtrar. Apenas os Things cujo <code>thingId</code> pertence a algum destes namespaces são eliminados. Se não for fornecido <code>--query</code> , todos os Things nos namespaces fornecidos são eliminados.
<code>--query</code>	<code>str</code>	Uma string de filtro RQL do Ditto. Apenas os Things que correspondem ao filtro são eliminados. Quando combinado com <code>--namespaces</code> , é utilizada a interseção de ambos os critérios.

Tabela 32 - Argumentos de CLI para utilização do Namespace Remover

Se ambos os argumentos forem omitidos, nenhum filtro de pesquisa é aplicado; o programa ainda será executado, mas pesquisar Things sem restrições (o comportamento depende da configuração de pesquisa da instância Ditto).

O parser é importado e utilizado em `main.py`:

```
from args import parser

if __name__ == "__main__":
    args = parser.parse_args()
    asyncio.run(main(args.namespaces, args.query))
```

9.6 Interfaces

O conceito de interface neste programa não é o de uma interface padrão que define funções obrigatórias a implementar. É simplesmente a definição do contacto com o mundo exterior — um módulo Python contendo funções assíncronas que utilizam `aiohttp` para comunicar com um serviço externo específico. Cada interface é autocontida num único ficheiro.

Para se manter organizado, espera-se que novos módulos de interface sejam criados como novos ficheiros na raiz do projeto (seguindo o padrão existente de `oidc.py`, `search.py`, `delete.py`), com um nome que permita fácil reconhecimento do propósito do módulo.

Adicionalmente, foi seguida a diretriz de que as funções ao nível do módulo devem ser tipadas com modelos Pydantic e retornar `None` em erros HTTP (em vez de lançar exceções).

9.6.1 OIDC Interface

A interface OIDC, definida em `oidc.py`, é responsável por interagir com a instância Keycloak da infraestrutura para obter e renovar tokens OIDC.

São fornecidas duas funções:

- `get_tokens()` — Utiliza o tipo de concessão password grant (`grant_type=password`) para autenticar com o nome de utilizador, palavra-passe, client ID, realm e scope configurados. Faz um pedido POST para `{base_url}/auth/realms/{realm}/protocol/openid-connect/token` com dados de formulário codificados em URL. Analisa a resposta para extrair `access_token`, `refresh_token` e `expires_in` (segundos). Retorna um modelo Tokens com um datetime `expiry` calculado, ou `None` em caso de erro HTTP. Cada chamada cria a sua própria `aiohttp.ClientSession` de curta duração.
- `refresh_token(refresh_token)` — Utiliza a concessão refresh token (`grant_type=refresh_token`) para obter um novo access token antes de o atual expirar. Segue o mesmo padrão que `get_tokens()` mas utiliza o refresh token fornecido em vez de credenciais de utilizador. Retorna um modelo Tokens ou `None`.

Ambas as funções constroem o mesmo URL:

```
url = f"{base_url}/auth/realms/{realm}/protocol/openid-connect/token"
```

9.6.2 Search Interface

A interface de pesquisa, definida em `search.py`, é responsável por consultar o endpoint Things-Search do Eclipse Ditto para encontrar Things que correspondem aos critérios fornecidos.

É fornecida uma função:

- `get_Things(session, namespaces, query, cursor)` — Envia um pedido POST para `search/Things` na sessão pré-autenticada (que já tem o bearer token e o URL base configurados). O corpo do pedido são dados de formulário codificados em URL contendo:

- `fields=thingId` — Apenas o campo `thingId` é solicitado.
- `namespaces=...` — Uma lista separada por vírgulas de namespaces (se fornecidos).
- `filter=...` — Uma string de filtro RQL do Ditto (se fornecida).
- `option=cursor(...)` — O cursor de paginação (se fornecido).
- Analisa a resposta JSON para extrair items (uma lista de objetos Thing) e cursor (uma string ou null). Retorna um modelo `SearchResponse`, ou `None` em caso de erro HTTP.

9.6.3 Delete Interface

A interface de eliminação, definida em `delete.py`, é responsável por remover Things do Eclipse Ditto emitindo pedidos DELETE.

São fornecidas duas funções:

- `delete_Thing(session, Thing)` — Envia um pedido DELETE para `Things/{thingId}` na sessão pré-autenticada. Regista e engole exceções `ClientResponseError` (por exemplo, 404 ou 403) em vez de parar o programa.
- `delete_Things(session, Things)` — Itera sobre uma lista de strings `thingId`, chamando `delete_Thing` para cada uma sequencialmente. Não é utilizado paralelismo, pois o número de Things por página é tipicamente pequeno (tamanho de página padrão do Ditto).

9.7 Orquestração

A lógica de orquestração reside em `main.py` e segue este pseudocódigo:

```
tokens ← get_tokens()
session ← ClientSession(base_url/api/2/, headers=Bearer tokens.access)

search_response ← get_Things(session, namespaces, query)
while search_response has Things:
    delete_Things(session, search_response.things)
    if no cursor: break
    search_response ← get_Things(session, namespaces, query, cursor)
    if token expired:
        tokens ← refresh_token(tokens.refresh)

close session
```

Detalhes importantes:

- O ciclo `while True` com condições `break` explícitas trata o ciclo de vida da paginação de forma limpa.
- A expiração do token é verificada após cada lote paginado utilizando `datetime.now(timezone.utc) >= tokens.expiry`.
- Se `get_tokens()` ou `refresh_token()` retornarem `None`, o programa pára, pois a autenticação é um requisito obrigatório.
- A chamada `session.close()` é alcançada apenas depois de todos os lotes paginados terem sido processados, garantindo que o pool de ligações é libertado.

10 TEXTURE CREATION

10.1 Visão Geral

O Texture Creation é um pipeline Python que cria ficheiros de textura GLTF coloridos a partir de dados de nuvem de pontos e malhas 3D. Obtém malhas PLY e nuvens de pontos CSV de uma API, calcula cores de vértices e exporta como GLTF. Os resultados são carregados para armazenamento compatível com S3 com lógica de cache.

O programa principal, conforme implementado atualmente, funciona da seguinte forma:

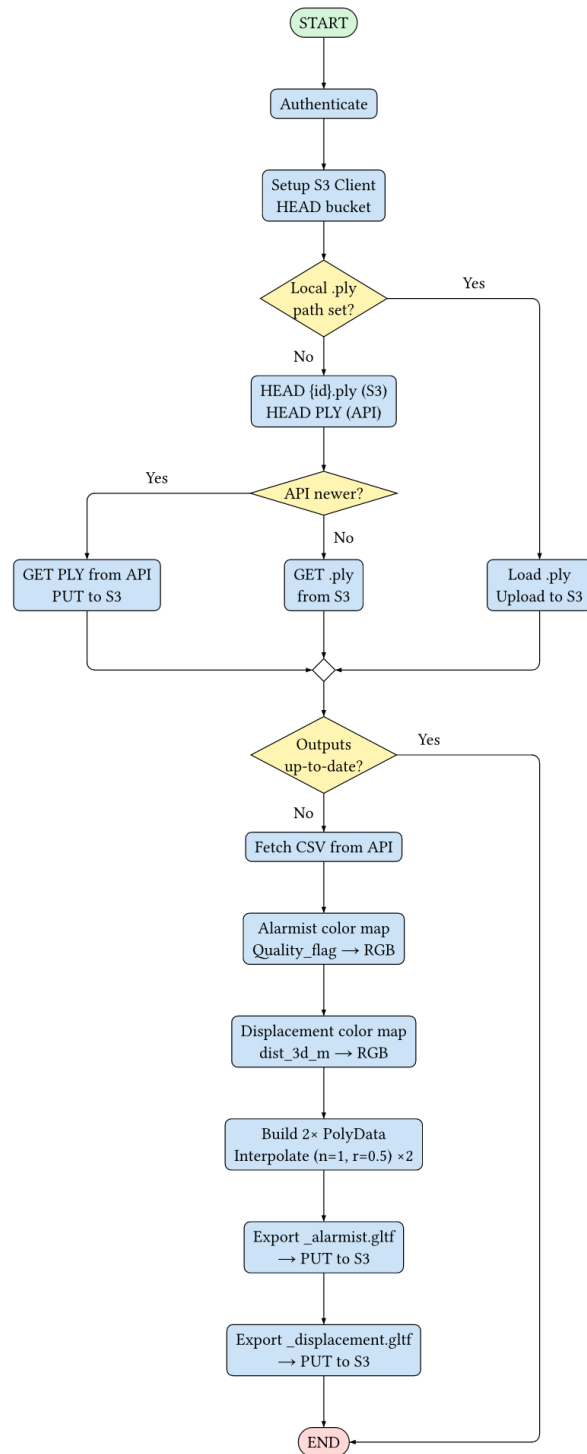


Figura 30: Grafo de fluxo de controle do Texture Creation

Além disso, o diagrama de sequência do fluxo de execução principal é o seguinte:

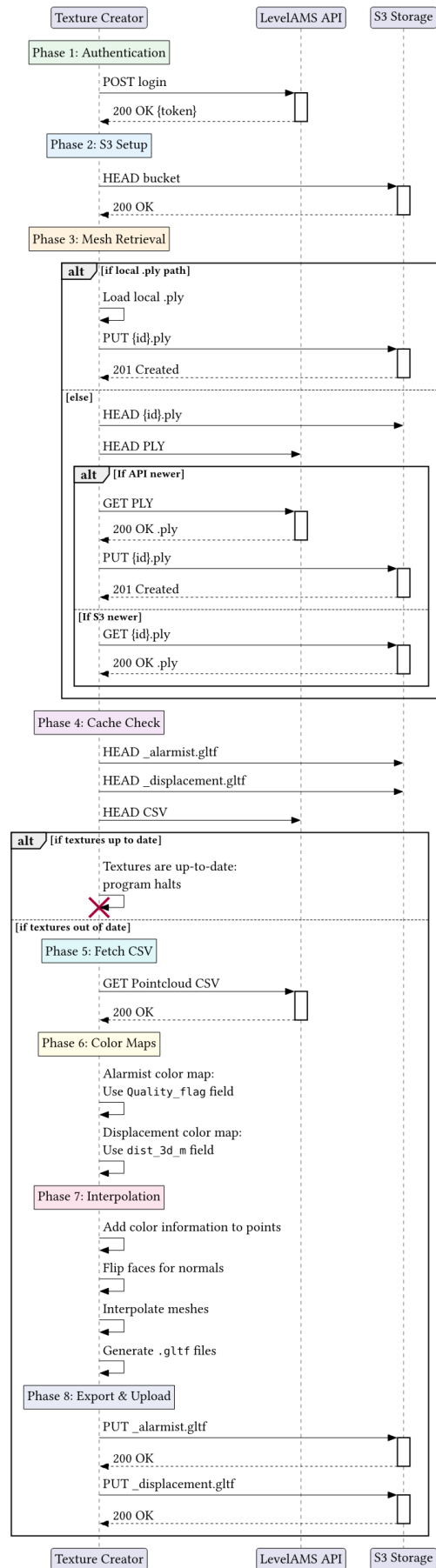


Figura 31: Diagrama de sequência do Texture Creation

10.2 Estrutura do Código

O programa está estruturado logicamente da seguinte forma:

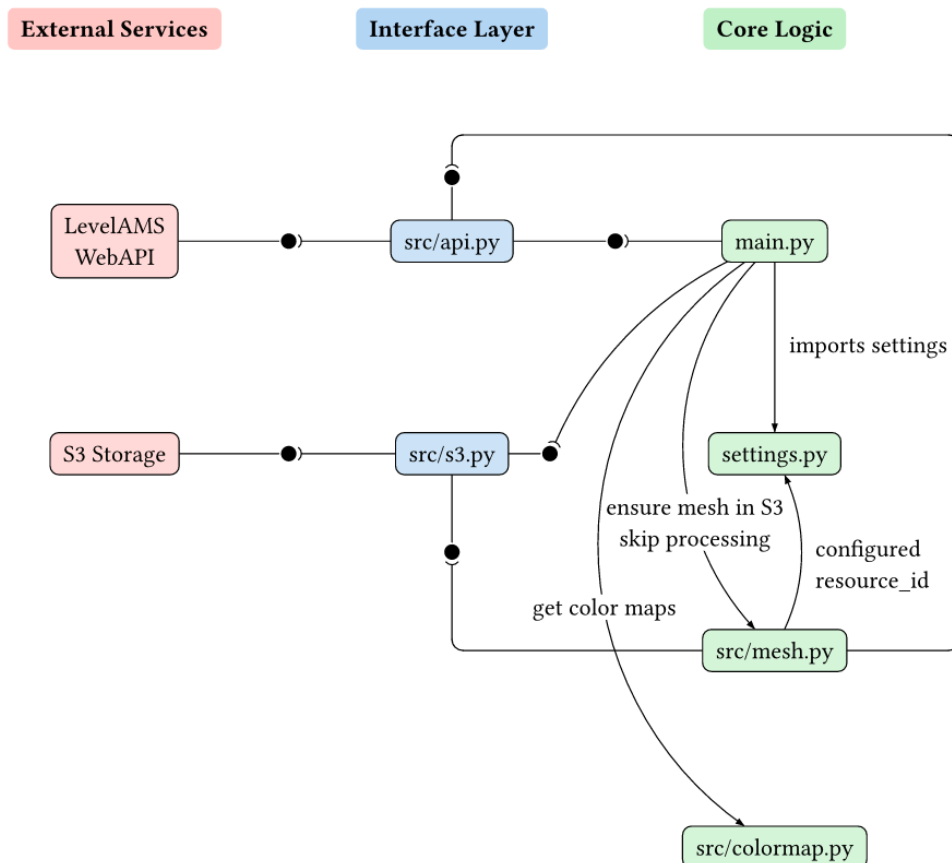


Figura 32: Grafo de dependências e estrutura de código do Texture Creator

NOTA: Para facilitar a compreensão deste diagrama: é utilizada uma notação ball and socket, onde uma bola representa o fornecimento de uma interface e uma socket representa o consumo dessa interface. É utilizada para mostrar as dependências entre componentes internos do sistema (e interação com sistemas externos). A coluna App Core é composta por main.py e settings.py. A coluna Pipeline Modules é composta pelos ficheiros dentro de src/ e a coluna External Systems é composta pelos sistemas com os quais se interage. Para facilitar a compreensão, os modelos de dados foram incluídos nos seus respetivos módulos.

Em main.py o pipeline sequencial é orquestrado, importando funções de cada um dos módulos em src/. A configuração é carregada a partir de um singleton global settings definido em settings.py, que é consumido por todos os módulos a jusante.

10.3 Modelos de Dados

Os modelos de dados neste projeto são todos criados utilizando Pydantic's BaseModel e

BaseSettings para gestão de configuração. Para saber mais sobre como criar um modelo Pydantic, é recomendada a consulta da sua [documentação oficial](#). No entanto, os conceitos importantes são que uma nova classe tem de ser criada que estende a classe BaseModel, e os campos são definidos dentro desta nova classe, juntamente com os seus tipos. O Pydantic é então responsável pela serialização e desserialização do modelo.

Em settings.py, os modelos de configuração estão estruturados como uma árvore de subclasses de BaseModel, agregadas numa classe Settings(BaseSettings) de topo:

- ApiSettings contém o campo base_url (AnyHttpUrl) para o endpoint da API que serve dados de nuvem de pontos e malha.
- AuthSettings contém tenant, resource_id, username e password para autenticação na API.
- S3Settings contém endpoint, access_key, secret_key e bucket para o armazenamento compatível com S3.
- MeshSettings contém um campo opcional path para carregar uma malha PLY local diretamente.

A classe Settings utiliza pydantic-settings para carregar de múltiplas fontes por ordem de prioridade:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        toml_file="config.toml",
        env_file=".env",
        env_nested_delimiter="__",
        extra="ignore",
    )

    s3: S3Settings = S3Settings()
    api: ApiSettings = ApiSettings()
    auth: AuthSettings = AuthSettings()
    mesh: MeshSettings = MeshSettings()
```

A ordem de resolução de fontes é config.toml (mais baixa), depois .env (média), depois variáveis de ambiente (mais alta). Campos aninhados utilizam __ como delimitador (por exemplo, S3__BUCKET). Uma única instância global é criada ao nível do módulo:

```
settings = Settings()
```

Adicionalmente, em src/api.py, dois modelos transitórios são definidos para o handshake de autenticação:

- AuthRequest — um BaseModel com campos username e password, serializado como JSON no corpo do POST.
- AuthResponse — um BaseModel com um campo token, desserializado da resposta JSON da API.

Para fins organizacionais, espera-se que quaisquer novos modelos sejam criados juntamente com o módulo a que pertencem, com um nome que permita fácil reconhecimento do propósito do modelo.

10.4 Interface API

A interface API (src/api.py) é responsável por todas as interações HTTP com o fornecedor de dados upstream, sendo este a API LevelAMS. Expõe as seguintes funções públicas:

- `fetch_token()` -> `str` — autentica contra `{base_url}/{tenant}/api/accounts/login` com um pedido POST contendo as credenciais configuradas. Retorna um bearer token utilizado para todas as chamadas subsequentes.
- `fetch_polygon(token: str)` -> `bytes` — descarrega a malha PLY de `{base_url}/{tenant}/api/v1/ativos-geotecnicos/{resource_id}/modelo/{POLYGON_TYPE}/download`. Retorna os bytes brutos do ficheiro.
- `fetch_csv(token: str)` -> `pd.DataFrame` — obtém o CSV de nuvem de pontos do mesmo padrão de URL com `{CSV_TYPE}` substituído, e analisa-o num Pandas DataFrame.
- `_get_api_last_modified(url: str, token: str)` -> `datetime | None` — emite um pedido HEAD para o URL da API fornecido e extrai o cabeçalho Last-Modified. Utilizado pela lógica de verificação de cache para comparar a frescura da fonte com as saídas S3.

Os construtores de URL internos (`polygon_url()`, `csv_url()`, `_auth_url()`) constroem os endpoints da API a partir do singleton de definições. As constantes de URL são:

```
POINT_CLOUD_TYPE = "ActivoGeotecnicoModeloModelType_CSV_Point_Cloud"
POLYGON_TYPE = "ActivoGeotecnicoModeloModelType_Ply"
```

Todas as chamadas HTTP utilizam a biblioteca `httpx`. O tratamento de erros limita-se a lançar `httpx.HTTPError` em respostas não-2xx através de `raise_for_status()`.

10.5 Interface S3

A interface S3 (`src/s3.py`) é responsável por todas as interações com o armazenamento compatível com S3 utilizado para cache tanto da malha PLY de entrada como dos ficheiros GLTF de saída. Expõe as seguintes funções públicas:

- `build_s3_client()` -> `BaseClient` — constrói um `boto3.client("s3", ...)` a partir do endpoint, access key e secret key configurados. Sai com erro se os campos obrigatórios estiverem em falta.
- `get_s3_client()` -> `tuple[BaseClient, str]` — encapsula `build_s3_client()` e realiza uma verificação de existência do bucket através de `head_bucket()`. Retorna o cliente e o nome do bucket. Regista um aviso se o bucket estiver inacessível mas não interrompe a execução.
- `download_from_s3(client, bucket, object_key, download_path)` — descarrega um objeto S3 para um ficheiro local. Levanta `botocore.exceptions.ClientError` em caso de falha.
- `upload_to_s3(client, file_path, bucket, object_key)` — carrega um ficheiro local para o S3 através de `upload_file`.
- `upload_bytes_to_s3(client, data, bucket, object_key)` — carrega bytes brutos para o S3 através de `put_object`.
- `_get_s3_last_modified(client, bucket, object_key)` -> `datetime | None` — emite um pedido HEAD ao objeto S3 e retorna o seu timestamp LastModified. Retorna None se o objeto não existir (404). Utilizado pela lógica de verificação de cache.

10.6 Processamento de Malha

O módulo de processamento de malha (`src/mesh.py`) é responsável por resolver a malha PLY de entrada a partir de uma de três fontes — ficheiro local, descarregamento da API ou cache S3 — e por decidir se o pipeline pode ser totalmente ignorado.

```
ensure_mesh_in_s3(s3_client, bucket, token, mesh_path) -> pv.DataObject.
```

Esta função implementa uma cadeia de prioridade de três níveis:

1. **Local file** — Se `mesh_path` não for `None`, a malha é carregada diretamente do sistema de ficheiros local através de `pv.read()`, carregada para o S3 como `{resource_id}.ply` e retornada imediatamente. Tanto o fallback da API como o do S3 são ignorados.
2. **API fetch** — Se não estiver definido nenhum caminho local, a função compara o timestamp `Last-Modified` do PLY em cache no S3 com o do endpoint de polígono da API. Se o S3 não tiver cópia em cache, ou a API tiver uma versão mais recente, a malha é descarregada da API através de `fetch_polygon()`, carregada para o S3 através de `upload_bytes_to_s3()` e retornada.
3. **S3 fallback** — Se a API estiver inacessível mas existir um PLY em cache no S3 (o timestamp S3 não era `None` antes da tentativa), a cópia desatualizada é descarregada e utilizada. Se nenhuma fonte estiver disponível, é levantado um `FileNotFoundError` e o programa termina.

```
need_api_fetch = s3_ply_lm is None or (
    polygon_api_lm is not None and polygon_api_lm > s3_ply_lm
)
```

```
should_skip_processing(s3_client, bucket, token) -> bool.
```

Esta função fornece idempotência. Compara o mais antigo dos dois ficheiros `.gltf` de saída no S3 com o cabeçalho `Last-Modified` do CSV da API:

```
alarmist_lm = _get_s3_last_modified(s3_client, bucket, alarmist_key)
displacement_lm = _get_s3_last_modified(s3_client, bucket, displacement_key)
s3_lastmodified = min(alarmist_lm, displacement_lm)
api_last_modified = _get_api_last_modified(csv_url(), token)
if s3_lastmodified >= api_last_modified:
    return True # skip processing
```

A função retorna `False` (ou seja, prosseguir com o processamento) se algum ficheiro de saída estiver ausente do S3 ou se o `Last-Modified` da API não puder ser determinado.

Ambas as funções dependem de `_get_s3_last_modified()` e `_get_api_last_modified()`, helpers internos que emitem pedidos HTTP HEAD ao objeto S3 e ao URL da API respetivamente, retornando objetos `datetime` ou `None`.

10.7 Cálculo de Cores

O módulo de cálculo de cores (`src/colormap.py`) fornece duas funções de mapeamento independentes dos dados de nuvem de pontos para cores de vértices, uma para cada uma das duas texturas de saída.

```
get_color(flag: str) -> list[float]
```

Mapeia a coluna `Quality_flag` do CSV para uma cor RGB fixa:

Flag	Cor	RGB
alert	Amarelo	[1.0, 1.0, 0.0]
alarm	Vermelho	[1.0, 0.0, 0.0]
OK (padrão)	Verde	[0.0, 1.0, 0.0]

Tabela 33 - Cálculo de cores para a criação da textura

A string de entrada é convertida para minúsculas e limpa antes da correspondência; qualquer valor não reconhecido assume verde como padrão. A pesquisa é uma cadeia simples if-elif-else.

```
compute_displacement_colors(dist_series: pd.Series) -> np.ndarray.
```

Mapeia o campo `dist_3d_m` através de um gradiente de cor com sinal:

1. **Valores em falta** são preenchidos com 0.0 (verde, sem deslocamento).
2. O array é **limitado** ao intervalo ± 20 mm através de `np.clip(dist, -0.02, 0.02)`.
3. Os deslocamentos negativos e positivos são tratados separadamente:
 - **Negativo** ([20 mm, 0 mm]): interpola linearmente de rosa [1.0, 0.0, 1.0] a -20 mm para verde [0.0, 1.0, 0.0] a 0 mm.
 - **Positivo** ([0 mm, +20 mm]): interpola linearmente de verde [0.0, 1.0, 0.0] a 0 mm para vermelho [1.0, 0.0, 0.0] a +20 mm.
 - As entradas com valor zero permanecem verdes.

A interpolação é uma rampa linear simples por canal de cor:

```
# Negative branch
t_neg = (d_neg + 0.02) / 0.02          # 0 at -20mm, 1 at 0mm
colors[neg_mask, 0] = 1.0 - t_neg      # R: 1 → 0
colors[neg_mask, 1] = t_neg           # G: 0 → 1
colors[neg_mask, 2] = 1.0 - t_neg     # B: 1 → 0

# Positive branch
t_pos = d_pos / 0.02                   # 0 at 0mm, 1 at +20mm
colors[pos_mask, 0] = t_pos            # R: 0 → 1
colors[pos_mask, 1] = 1.0 - t_pos      # G: 1 → 0
colors[pos_mask, 2] = 0.0             # B: stays 0
```

A função retorna um array NumPy de forma (N, 3) com `dtype float64`.

10.8 Pipeline de Cores de Vértice

A secção seguinte detalha como as cores por ponto produzidas por `colormap.py` são transferidas para a superfície da malha e exportadas como um ficheiro GLTF.

10.8.1 From Colored Points to Textured Mesh

10.8.1.1

oints receive colors

Em `main.py`, após obter o CSV, os arrays de cor são calculados em paralelo:

```
csv_colors = np.array([get_color(f) for f in df["Quality_flag"]])
cloud = pv.PolyData(df[["X", "Y", "Z"]].values)
cloud.point_data["Colors"] = csv_colors

disp_colors = compute_displacement_colors(df["dist_3d_m"])
disp_cloud = pv.PolyData(df[["X", "Y", "Z"]].values)
disp_cloud.point_data["Colors"] = disp_colors
```

Duas nuvens de pontos `pyvista.PolyData` independentes são criadas — uma para a textura alarmista (baseada em `Quality_flag`) e uma para a textura de deslocamento (baseada em `dist_3d_m`). Nesta fase, as cores residem **apenas na nuvem de pontos esparsa** (tipicamente milhares de pontos), enquanto a malha tem os seus próprios vértices (tipicamente dezenas de milhares) sem dados de cor.

10.8.1.2

olors are interpolated onto mesh vertices

```
mesh.flip_faces(inplace=True)
interpolated = mesh.interpolate(cloud, n_points=1, radius=0.5)
interpolated.set_active_scalars("Colors")
```

Internamente, `pyvista.DataSetFilters.interpolate()` executa o seguinte:

1. Constrói uma **KD-tree** a partir das coordenadas espaciais da nuvem de pontos.
2. Para **cada vértice** da malha, encontra o ponto colorido mais próximo dentro do raio de pesquisa de 0,5 m (`n_points=1, radius=0.5`).
3. Copia o valor RGB desse ponto para o array `Colors` do vértice da malha correspondente.
4. Os vértices **sem vizinho dentro de 0,5 m** recebem um valor NaN e aparecerão a preto ou sem cor na saída.

O resultado é uma **nova malha** com geometria idêntica mas agora com cores por vértice. A chamada `mesh.flip_faces()` garante que a ordem de enrolamento está correta para o renderizador GLTF a jusante.

NOTA: Isto **não** é um mapa de textura UV. Não existe imagem, coordenadas UV ou atlas de textura. As cores são embutidas diretamente como um atributo de cada vértice da malha.

10.8.1.3

meshes are exported to GLTF

```
pl = pv.Plotter(off_screen=True)
pl.add_mesh(interpolated, scalars="Colors", rgb=True, preference="point")
pl.export_gltf(tmp_path)
```

O exportador GLTF do PyVista escreve a geometria da malha e mapeia o array `Colors` ativo para o **atributo de vértice** `COLOR_0` no ficheiro GLTF. De acordo com a especificação GLTF:

- O ficheiro .gltf contém **nenhum dado de imagem** — apenas arrays de atributos de vértice.
- A renderização é feita através de **interpolação de cor de vértice**: a GPU interpola cores entre fragmentos de triângulo utilizando os três vértices coloridos de cada face.
- Qualquer visualizador compatível com GLTF (Babylon.js, Three.js, Windows 3D Viewer) carrega COLOR_0 e mapeia-o para a propriedade vertexColors do material, renderizando a malha como se tivesse uma superfície pintada.



Figura 33: Pipeline de criação de malha

Os dois ficheiros de saída são carregados para o S3 como {resource_id}_alarmist.gltf e {resource_id}_displacement.gltf.

10.9 Pipeline Principal

Conforme orquestrado por main.py, o pipeline sequencial procede da seguinte forma:

1. **Authentication** — fetch_token() é chamado para obter um bearer token do endpoint de login da API.
2. **S3 client initialization** — get_s3_client() constrói um cliente boto3 e verifica se o bucket configurado está acessível.
3. **Mesh retrieval** — ensure_mesh_in_s3() resolve a malha PLY de entrada a partir do sistema de ficheiros local, da API ou da cache S3, retornando um pyvista.PolyData.
4. **Cache check** — should_skip_processing() compara os timestamps de saída do S3 com o Last-Modified do CSV da API. Se as saídas estiverem atualizadas, o programa termina antecipadamente.
5. **CSV data fetch** — fetch_csv() descarrega o CSV da nuvem de pontos e analisa-o num Pandas DataFrame.
6. **Color computation** — Dois arrays de cor são calculados em paralelo: get_color() para cada valor Quality_flag, e compute_displacement_colors() para a coluna dist_3d_m.
7. **Interpolation** — Cada array de cor é atribuído a uma nuvem de pontos pyvista.PolyData. A malha tem as faces invertidas, depois cada nuvem de pontos é interpolada na malha utilizando interpolate(n_points=1, radius=0.5).
8. **GLTF export** — Cada malha interpolada é escrita para um ficheiro GLTF temporário através de pyvista.Plotter.export_glTF(), depois carregada para o S3 como {resource_id}_alarmist.gltf e {resource_id}_displacement.gltf.

11 SIMULAÇÃO DA FAIXA RESERVADA A5

11.1 Visão Geral

O Reserved Lane Simulation Service é uma aplicação Python desenvolvida para executar simulações microscópicas de tráfego a pedido, permitindo avaliar o impacto da implementação de uma faixa reservada com acesso pago na autoestrada A5. O serviço disponibiliza uma interface web e uma API REST através das quais o utilizador pode submeter pedidos de simulação, acompanhar o estado da execução e consultar os resultados produzidos.

A aplicação utiliza o Eclipse SUMO (Simulation of Urban Mobility) como motor de simulação microscópica. O controlo da execução é efetuado através da Traffic Control Interface (TraCI), permitindo modificar dinamicamente o comportamento dos veículos durante a simulação e recolher informação em tempo real sem necessidade de processar os ficheiros de saída gerados pelo SUMO.

Cada pedido de simulação corresponde à execução de quatro cenários independentes, executados em paralelo através do módulo `multiprocessing` de Python. A gestão dos pedidos é efetuada de forma assíncrona, permitindo que múltiplas simulações sejam submetidas.

A aplicação encontra-se organizada segundo uma arquitetura modular responsável pela gestão da API, execução das simulações, recolha das estatísticas e gestão dos pedidos.

11.2 Fluxo de Execução

O fluxo de execução (Figura 34) do serviço inicia-se quando um utilizador submete um pedido de simulação através da interface web ou da API REST. O pedido é recebido pelo endpoint `POST /api/v1/simulation`, que valida os parâmetros recebidos utilizando os modelos Pydantic e cria um novo `SimulationJob`.

Após a criação do trabalho, este é registado pelo `SimulationJobManager`, que lhe atribui um identificador único (`job_id`) e agenda a sua execução em segundo plano. O endpoint devolve imediatamente uma resposta ao cliente contendo o identificador do trabalho e o respetivo estado inicial (*pending*).

Quando existe capacidade disponível, o `SimulationJobManager` inicia a execução do trabalho através do `SimulationRunner`. Este componente cria quatro processos independentes, um para cada cenário experimental (`real`, `allOpen`, `closedLane` e `closedLaneFast`).

Cada processo executa um `ScenarioExecutor`, responsável por iniciar uma instância do SUMO, estabelecer a ligação TraCI, executar a simulação e recolher continuamente informação sobre os veículos e detetores. Os eventos observados são enviados para um `StatisticsCollector`, que calcula os indicadores finais do cenário.

Após a conclusão dos quatro cenários, o `SimulationRunner` agrega os respetivos resultados numa instância de `SimulationResponse`, atualizando o estado do trabalho para *done*. O cliente pode consultar o progresso e o resultado através do endpoint `GET /api/v1/simulation/{job_id}`.

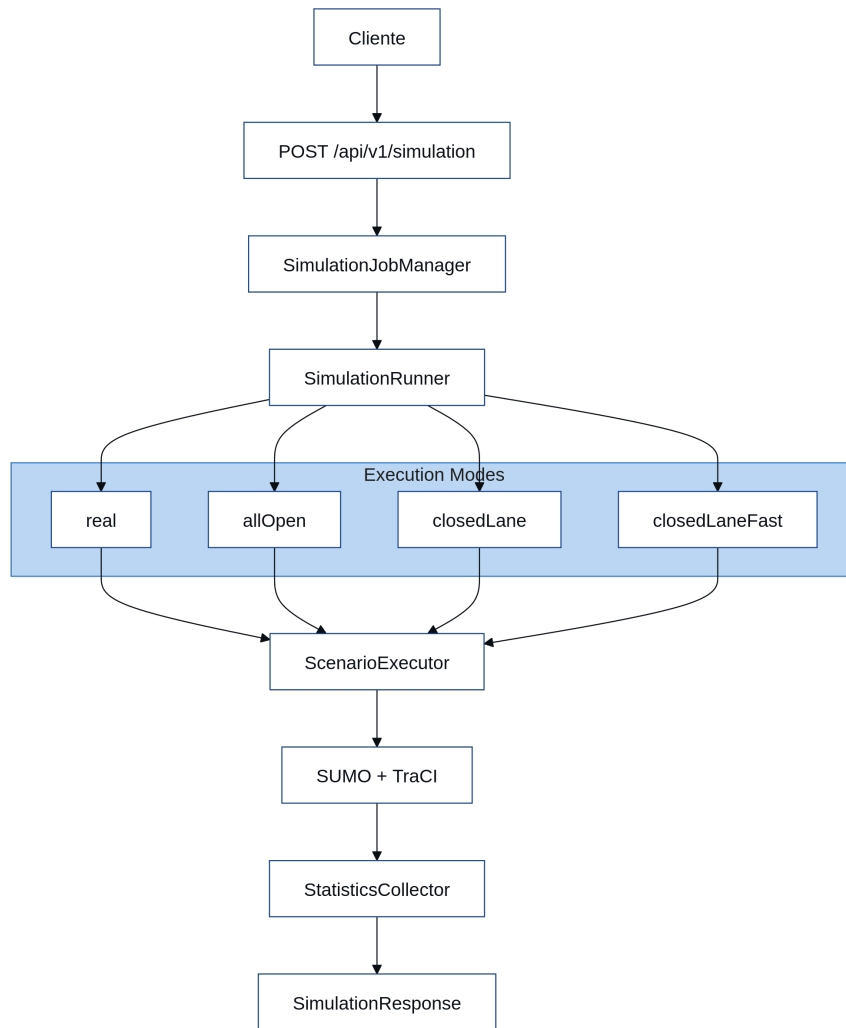


Figure 34- Diagrama do fluxo de execução

11.3 Estrutura do código

O projeto encontra-se organizado em módulos independentes.

- **main.py** – Inicialização da aplicação FastAPI e configuração dos endpoints principais.
- **api/** – Implementação dos endpoints REST responsáveis pela submissão e consulta das simulações.
- **models/** – Definição dos modelos de dados utilizados pela API.
- **services/simulation/** – Implementação do motor de simulação, gestão dos cenários, estatísticas e gestão dos trabalhos.
- **settings/** – Configuração da aplicação através de variáveis de ambiente.
- **templates/** – Interface web desenvolvida em HTML, CSS e JavaScript.
- **chart/** – Helm Chart utilizado para Kubernetes.

11.4 Configuração

A configuração do serviço é centralizada na classe `SimulatorSettings`, implementada em `settings/simulator.py`. Os parâmetros podem ser alterados através de variáveis de ambiente, permitindo adaptar facilmente a aplicação a diferentes ambientes de execução.

Entre os principais parâmetros configuráveis incluem-se:

- localização dos ficheiros da rede e das rotas;
- intervalo temporal da simulação;
- comprimento do passo temporal;
- faixas reservadas e faixas monitorizadas;
- identificador do detetor de portagem;
- intervalo da via onde é permitida a mudança para a faixa reservada;
- porta base utilizada pelas ligações TraCI;
- número máximo de simulações concorrentes;
- tempo de retenção dos resultados dos trabalhos concluídos.

11.5 Modelos de Dados

A comunicação entre a interface web, a API e os restantes componentes da aplicação é suportada por modelos de dados desenvolvidos com **Pydantic** (BaseModel). Estes modelos são responsáveis pela validação dos pedidos recebidos, pela estruturação das respostas da API e pela representação do estado dos trabalhos de simulação.

Os seguintes modelos encontram-se definidos em `models/simulation.py`.

11.5.1 SimulationRequest

Representa o pedido de submissão de uma simulação.

Campos:

- **paying_percentage** (float) – Fração de veículos elegíveis autorizados a utilizar a faixa reservada mediante o pagamento da portagem. O valor deve estar compreendido entre **0.0** e **1.0**.
- **price** (float) – Valor da portagem aplicada aos veículos que utilizam a faixa reservada. O valor deve ser superior ou igual a **0.0**.
- **traffic_scale** (float) – Fator de escala aplicado à procura de tráfego nos cenários congestionados. O valor deve ser superior a **0.0**.

11.5.2 ScenarioResult

Representa os indicadores produzidos por um cenário de simulação.

Campos:

- **total_vehicles** (int) – Número total de veículos simulados.
- **lane2_vehicles** (int) – Número de veículos que utilizaram a faixa reservada.
- **lane2_usage_perc** (str) – Percentagem de utilização da faixa reservada.
- **revenue** (float) – Receita estimada obtida através da cobrança da portagem.
- **avg_speeds** (dict[str, float]) – Dicionário contendo a velocidade média observada em cada uma das faixas monitorizadas.

11.5.3 SimulationResponse

Representa o resultado completo de um pedido de simulação.

Este modelo agrega os resultados dos quatro cenários executados:

- **real** – Tráfego histórico observado.
- **allOpen** – Cenário congestionado com todas as faixas disponíveis.
- **closedLane** – Faixa reservada encerrada ao tráfego geral.
- **closedLaneFast** – Faixa reservada com acesso condicionado ao pagamento da portagem.

Cada um destes campos corresponde a um objeto do tipo ScenarioResult.

11.5.4 JobStatus

Enumeração que representa o estado de um trabalho de simulação.

Os estados possíveis são:

- **pending** – O pedido foi submetido e aguarda execução.
- **running** – A simulação encontra-se em execução.
- **done** – A execução terminou com sucesso.
- **failed** – Ocorreu um erro durante a execução.

11.5.5 JobSubmitted

Representa a resposta devolvida após a submissão de um novo pedido de simulação.

Campos:

- **job_id** (str) – Identificador único do trabalho criado.
- **status** (JobStatus) – Estado inicial do trabalho, normalmente pending.

11.5.6 JobStatusResponse

Representa a resposta devolvida ao consultar o estado de um trabalho de simulação.

Campos:

- **job_id** (str) – Identificador do trabalho.
- **status** (JobStatus) – Estado atual da execução.
- **result** (SimulationResponse | None) – Resultados da simulação, disponíveis apenas quando o estado é done.

- **error** (str | None) – Mensagem de erro produzida durante a execução, caso o estado seja failed.

11.6 API

A comunicação com a aplicação é efetuada através de uma API REST implementada em FastAPI.

O endpoint POST /api/v1/simulation recebe os parâmetros da simulação, cria um novo trabalho e devolve imediatamente um identificador (job_id), sem bloquear a execução do cliente.

O endpoint GET /api/v1/simulation/{job_id} permite consultar o estado da execução e, após a conclusão, obter os resultados produzidos pelos quatro cenários simulados.

Adicionalmente, o endpoint /healthz disponibiliza um mecanismo de verificação do estado da aplicação, utilizado em ambientes Kubernetes.

11.7 Simulação

O motor de simulação é implementado pela classe SimulationRunner, responsável por coordenar a execução dos quatro cenários experimentais.

Cada cenário é executado numa instância independente do SUMO, recorrendo à biblioteca multiprocessing do Python. Para cada processo é atribuída uma porta TraCI distinta, permitindo que várias instâncias do simulador sejam executadas simultaneamente.

Após a conclusão de todos os processos, os resultados individuais são agregados num único objeto SimulationResponse, posteriormente disponibilizado pela API.

11.8 Gestão de Trabalhos

A execução das simulações é gerida pela classe SimulationJobManager.

Cada pedido recebido é armazenado como um trabalho contendo o identificador, o estado da execução, o pedido original, o resultado obtido e eventuais mensagens de erro.

Os estados possíveis são:

- **Pending** – trabalho submetido e em espera;
- **Running** – simulação em execução;
- **Done** – simulação concluída com sucesso;
- **Failed** – ocorreu um erro durante a execução.

Para evitar sobrecarga do servidor, a execução concorrente é limitada através de um asyncio.Semaphore, cujo número máximo de trabalhos simultâneos é configurável. Os resultados permanecem em memória durante um período definido, sendo posteriormente removidos automaticamente.

11.9 Execução dos Cenários

A execução individual de cada cenário é realizada pela classe ScenarioExecutor.

Inicialmente é construída a linha de comandos utilizada para iniciar o SUMO, incluindo a rede viária, os ficheiros de rotas, os ficheiros adicionais e os parâmetros temporais da simulação.

Durante a execução, o TraCI controla o avanço temporal da simulação e recolhe continuamente informação sobre todos os veículos presentes na rede.

Nos cenários com faixa reservada, as permissões de circulação são alteradas de forma dinâmica. No cenário **closedLaneFast**, cada veículo elegível é avaliado apenas uma vez dentro de uma determinada zona da via. Caso cumpra a probabilidade de pagamento definida pelo utilizador, o TraCI altera a sua classe de veículo e comanda a mudança para a faixa reservada.

Após o término da simulação, a ligação TraCI é encerrada e os indicadores calculados são devolvidos ao motor de simulação.

11.10 Recolha de Estatísticas

A recolha e agregação dos indicadores é efetuada pela classe `StatisticsCollector`.

Durante cada passo temporal são registados:

- os veículos presentes na rede;
- os veículos que utilizam a faixa reservada;
- as velocidades observadas nas faixas monitorizadas;
- os veículos detetados pelo laço indutivo da portagem.

No final da execução são calculados:

- número total de veículos simulados;
- número de veículos na faixa reservada;
- percentagem de utilização da faixa reservada;
- velocidade média de cada faixa monitorizada;
- receita estimada da portagem.

Os resultados são encapsulados num objeto `ScenarioResult`, sendo posteriormente agregados para os quatro cenários.

11.11 Execução

A aplicação foi desenvolvida para execução local ou em ambientes Kubernetes.

A instalação das dependências é efetuada através do **uv**, sendo posteriormente iniciado um servidor **Uvicorn** que disponibiliza a aplicação FastAPI.

Para Kubernetes é disponibilizado um Helm Chart contendo os manifestos necessários para a criação do Deployment, Service e Ingress, permitindo configurar tanto os recursos da aplicação como os parâmetros da simulação através de variáveis de ambiente.

12 VCI TRAFFIC SIMULATION

12.1 Visão Geral

A Simulação de Tráfego da VCI é desenvolvida utilizando o Eclipse SUMO e um conjunto de scripts Python responsáveis pela geração da procura, calibração e execução das simulações.

O processo encontra-se dividido em três fases principais:

- construção da rede;
- geração e calibração das rotas;
- execução da simulação.

12.2 Estrutura

12.3 Estrutura do Projeto

O projeto encontra-se organizado conforme apresentado na Tabela X.

Pasta/Ficheiro	Descrição
osm.net.xml	Rede rodoviária da VCI utilizada na simulação.
detectors.det.xml	Definição dos detetores virtuais utilizados para recolha de dados.
sampledRoutes2.rou.xml	Conjunto de rotas calibradas utilizado pela simulação.
main.py	Script principal responsável pela execução da simulação.
Dockerfile	Configuração do ambiente de execução em Docker.
randomTrips/	Scripts utilizados para geração e calibração das rotas.
stats.xml	Estatísticas produzidas pela última execução da simulação.
inductionoutput.xml	Resultados dos detetores virtuais produzidos pelo SUMO.

12.4 Rede

A rede rodoviária foi gerada a partir do OpenStreetMap utilizando o SUMO WebWizard.

Após a geração da rede foram adicionados manualmente os detetores virtuais correspondentes às localizações dos contadores de tráfego da Infraestruturas de Portugal.

Os principais ficheiros utilizados são:

Ficheiro	Descrição
osm.net.xml	Rede rodoviária
detectors.det.xml	Definição dos detetores
osm.sumocfg	Configuração da simulação

12.5 Geração das rotas

A reconstrução da procura é efetuada em duas etapas.

O script `randomTrips.py` gera um conjunto alargado de viagens sobre a rede rodoviária.

Na configuração utilizada destacam-se os seguintes parâmetros:

- `departLane="best_prob"`
- `departSpeed="avg"`
- `fringe-factor=max`
- `random-routing-factor=10`

O resultado desta etapa é um ficheiro de rotas candidatas (`randomRoutes.rou.xml`).

As rotas candidatas são posteriormente processadas pelo `routeSampler.py`. Esta ferramenta utiliza as contagens históricas dos detetores para selecionar apenas as rotas que melhor reproduzem os fluxos observados.

Como resultado são produzidos:

- ficheiro de rotas calibradas (`sampledRoutes.rou.xml`);
- ficheiro de incompatibilidades (`mismatch.xml`).

12.6 Processo de Calibração

Durante a calibração foram avaliadas múltiplas combinações de parâmetros. As variáveis analisadas incluíram:

- diferentes conjuntos de rotas;
- Step temporal da simulação;
- utilização da seleção probabilística de faixa (`best_prob`);
- extrapolação da posição inicial (`--extrapolate-departpos`);
- fatores de aleatoriedade na geração das rotas.

12.7 Execução

A simulação pode ser executada recorrendo ao Docker, construindo a imagem:

```
docker build -t vci-simulation .
```

Após a execução são produzidos diversos ficheiros contendo os resultados da simulação.

Ficheiro	Descrição
<code>stats.xml</code>	Estatísticas globais da simulação.
<code>inductionoutput.xml</code>	Contagens registadas pelos detetores virtuais.

Os resultados podem ser comparados com as contagens históricas utilizadas durante a calibração para avaliar a qualidade da simulação.

13 MODELOS DE PREVISÃO DE FLUXO DE TRÁFEGO

13.1 Visão Geral

Este módulo implementa um pipeline de *machine learning* para previsão de fluxo de tráfego horário a partir de séries temporais. O sistema suporta quatro modelos de previsão:

- SARIMA;
- Random Forest;
- XGBoost;
- Long Short-Term Memory (LSTM).

Todo o processo, desde o pré-processamento dos dados até ao treino, avaliação e armazenamento dos modelos, é automatizado. A otimização dos hiperparâmetros é realizada através do Optuna, enquanto o MLflow é utilizado para monitorização das experiências, armazenamento de métricas, modelos e artefactos produzidos durante o treino.

13.2 Arquitetura de Machine Learning

13.2.1 Pré-processamento (preprocess.py)

O módulo de pré-processamento prepara automaticamente cada conjunto de dados para treino.

As principais operações realizadas são:

- leitura dos ficheiros CSV dos detetores;
- ordenação temporal da série;
- reconstrução da série horária completa entre janeiro de 2022 e dezembro de 2024;
- imputação automática de valores em falta;
- criação de variáveis temporais;
- criação de variáveis cíclicas (seno/cosseno);
- identificação de fins de semana e feriados nacionais;
- integração das variáveis meteorológicas (temperatura média e precipitação média diária);
- criação dos desfasamentos temporais (lags);
- normalização das variáveis para modelos baseados em aprendizagem automática.

As features geradas incluem:

- hora do dia;
- mês;
- ano;
- codificação cíclica da hora;
- codificação cíclica do dia da semana;
- codificação cíclica do mês;
- indicador de fim de semana;
- indicador de feriado;
- lag de 1 hora;
- lag de 24 horas;
- lag de 168 horas;
- temperatura média;

- precipitação média.

Após o pré-processamento, os dados são divididos num conjunto de treino/validação e um conjunto de teste final (15%). Realiza-se cross validation (TimeSeriesSplit) com cinco folds.

13.2.2 Imputação de Dados (impute.py)

A imputação dos valores em falta é efetuada recorrendo a estatísticas históricas do próprio detetor. O algoritmo utiliza a seguinte prioridade:

1. média histórica para o mesmo dia da semana e hora;
2. média histórica para a mesma hora;
3. último valor válido disponível.

Para evitar séries excessivamente suavizadas é adicionada uma pequena perturbação gaussiana controlada pelo parâmetro **noiseFac**, e preservando valores de fluxo não negativos.

13.2.3 Treino dos Modelos (models.py)

O módulo de treino implementa quatro modelos independentes.

13.2.3.1 SARIMA

Modelo estatístico de previsão temporal baseado em componentes autoregressivos e sazonais. Os parâmetros otimizados incluem:

- p;
- q;
- P;
- Q.

O período sazonal utilizado corresponde a 24 horas.

13.2.3.2 Random Forest

Modelo baseado em árvores de decisão. Os principais hiperparâmetros pesquisados são:

- número de árvores;
- profundidade máxima;
- número mínimo de amostras por divisão;
- número mínimo de amostras por folha;
- número máximo de features por árvore.

13.2.3.3 XGBoost

Modelo baseado em Gradient Boosting. Durante a otimização são pesquisados:

- tipo de booster;
- learning rate;
- profundidade máxima;
- regularização L1;
- regularização L2;
- gamma;

- política de crescimento da árvore.

O treino utiliza **early stopping**.

13.2.3.4 LSTM

Rede neuronal recorrente para séries temporais. São otimizados:

- comprimento da janela temporal (lookback);
- número de neurónios nas duas camadas LSTM;
- taxa de dropout;
- learning rate;
- batch size.

O treino utiliza **EarlyStopping**, restaurando automaticamente os melhores pesos obtidos.

13.2.4 Otimização de Hiperparâmetros

Todos os modelos são calibrados automaticamente utilizando o Optuna. Cada estudo utiliza:

- otimização Bayesiana;
- cross validation temporal;
- pruning automático de experiências pouco promissoras;
- Early Stopping (quando aplicável).

O número de iterações é definido em:

Modelo	Número de trials
XGBoost	50
Random Forest	50
SARIMA	10
LSTM	20

13.3 Métricas de Avaliação

Durante a validação e teste são calculadas automaticamente as seguintes métricas:

- Mean Squared Error (MSE);
- Root Mean Squared Error (RMSE);
- Mean Absolute Error (MAE);
- Symmetric Mean Absolute Percentage Error (SMAPE);
- Coeficiente de determinação (R^2).

Para os modelos treinados com normalização (Random Forest, XGBoost e LSTM), as métricas são ainda calculadas na escala original dos dados.

13.4 Artefactos Produzidos

Para cada detetor e para cada modelo são gerados automaticamente:

- modelo treinado;
- Scalers utilizados durante o treino;
- lista de features;
- métricas de validação;
- métricas de teste;
- previsões;
- gráficos de resíduos;
- gráficos de previsão;
- diagnóstico ACF/PACF;
- gráfico Q-Q dos resíduos;
- gráfico do erro por hora;
- curva de aprendizagem (LSTM);
- gráficos SHAP (Random Forest e XGBoost).

Todos estes artefactos são armazenados na pasta output/.

13.5 Execução

Execute o seguinte para montar e iniciar o ecossistema multi-container:

```
docker compose up --build.
```

Navegue para <http://localhost:5000> no seu browser para interagir com a UI ativa do MLflow.

14 MODELO AGÊNTICO

14.1 Visão Geral

O modelo agêntico foi desenvolvido para monitorizar autonomamente o estado do tráfego na Via de Cintura Interna (VCI), através da comparação entre os fluxos de tráfego previstos pelos modelos de machine learning e os valores efetivamente observados pelos detetores. O sistema integra dados históricos de tráfego, condições meteorológicas e observações em tempo real para identificar desvios significativos, distinguir eventos localizados de alterações globais do tráfego e produzir uma avaliação contínua do estado da rede.

14.2 Arquitetura

A arquitetura multiagente, composto por um agente coordenador e por múltiplos agentes especializados, correspondendo cada um a um par detetor-direção. A coordenação entre agentes é realizada através da framework Goose, enquanto a comunicação entre o modelo de linguagem e as funcionalidades da aplicação é efetuada recorrendo ao Model Context Protocol (MCP), implementado com a biblioteca FastMCP.

O modelo agêntico encontra-se organizado segundo uma arquitetura modular composta por um agente coordenador, um conjunto de subagentes, um servidor MCP e os serviços de suporte executados em Docker.

14.3 Fluxo de Execução

O ponto de entrada da aplicação é definido pelo ficheiro `entrypoint.sh`, responsável por seleccionar o modo de execução de acordo com a variável `RUN_MODE`. Em modo de produção é executada uma única iteração utilizando a hora corrente, enquanto no modo de teste é efetuada a reprodução de um intervalo histórico, executando sucessivamente o agente coordenador para cada hora compreendida entre `TEST_START` e `TEST_END`.

O comportamento do agente coordenador e dos subagentes é definido através das receitas `coordinator_recipe.yaml` e `subagent_recipe.yaml`, respetivamente. O coordenador é responsável pela obtenção da topologia da rede, identificação dos detetores ativos, criação dos subagentes, agregação dos resultados e validação das anomalias ao nível do corredor. Cada subagente executa autonomamente a recolha dos dados, a obtenção da previsão, a comparação com os valores observados e o cálculo da métrica SMAPE.

14.4 MCP Server

As funcionalidades disponibilizadas aos agentes são implementadas no ficheiro `mcp_server.py`, recorrendo ao FastMCP. Este servidor expõe ferramentas para obtenção dos dados históricos e observados, acesso à topologia da rede, geração das previsões através dos modelos XGBoost, cálculo das métricas de avaliação e registo dos resultados no MLflow. Novas ferramentas podem ser adicionadas implementando uma nova função decorada com `@mcp.tool()`, tornando-a automaticamente disponível para utilização pelos agentes.

14.5 Modelos de Previsão

Os modelos preditivos encontram-se armazenados num repositório Amazon S3. Durante a execução, o servidor MCP descarrega dinamicamente os artefactos correspondentes ao detetor solicitado, carregando o modelo XGBoost e os respetivos scalers. A substituição ou atualização dos modelos requer a atualização dos artefactos armazenados no S3.

14.6 Configuração e Execução

A configuração dos serviços externos é efetuada através das variáveis de ambiente definidas no ficheiro `.env`, incluindo os endereços da plataforma Eclipse Ditto, do servidor MLflow, do fornecedor do modelo de linguagem, do repositório Amazon S3 e os parâmetros de funcionamento do Goose.

Após a configuração do ambiente, o sistema é iniciado através do comando `docker compose up -build`.